

Объектіге бағытталған бағдарламалау (C#)

Электрондық оқу құралы



Найзағараева А.А.

НАЙЗАҒАРАЕВА А.А.

**С# ТІЛІНДЕ ОБЪЕКТИГЕ БАҒЫТТАЛҒАН БАҒДАРЛАМАЛАУ
ЭЛЕКТРОНДЫҚОҚУЛЫҚ**

АСТАНА 2026

АЛҒЫС СӨЗ

«C# тілінде объектіге бағытталған бағдарламалау» электрондық оқулығы ақпараттық технологиялар бағытындағы білім беру бағдарламаларының студенттеріне арналған. Оқулық C# тілінің негіздерін жүйелі меңгеруге, алгоритмдік ойлауды дамытуға және объектіге бағытталған бағдарламалау дағдыларын қалыптастыруға бағытталған. Бағдарламалық жүйелерді әзірлеуде алгоритмдерді құру, деректерді ұйымдастыру және бағдарламаның құрылымын дұрыс жобалау маңызды. Объектіге бағытталған бағдарламалау күрделі есептерді өзара әрекеттесетін объектілер арқылы шешуге мүмкіндік береді. C# тілі заманауи бағдарламалық қосымшаларды әзірлеуде кеңінен қолданылатын жоғары деңгейлі бағдарламалау тілі болып табылады. Электрондық оқулықтың мақсаты — білім алушыларға C# тілінде бағдарламалау негіздерін меңгерту, теориялық білімді практикалық тапсырмалармен ұштастыру және бағдарламаларды өздігінен әзірлеу, тексеру мен жетілдіру дағдыларын дамыту. Оқу материалдары қарапайым ұғымдардан күрделі тақырыптарға қарай бірізді ұсынылған. Оқулықта C# бағдарламасының құрылымы, мәліметтер типтері, айнымалылар, енгізу және шығару құралдары, арифметикалық және логикалық операциялар қарастырылады. Шартты және циклдік операторлар, массивтер, әдістер, рекурсия және коллекциялар (List, HashSet, Dictionary) бағдарламалық мысалдар арқылы түсіндіріледі. Объектіге бағытталған бағдарламалау бөлімінде кластар мен объектілер, конструкторлар, қасиеттер, инкапсуляция, мұрагерлік және полиморфизм қағидалары қамтылады. Сонымен қатар интерфейстер, атаулар кеңістігі (namespace), .NET платформасының кітапханалары және ерекше жағдайларды (exceptions) өңдеу тәсілдері қарастырылады. Электрондық оқулық дәріс материалдарын, зертханалық жұмыстарды, әдістемелік нұсқауларды және білімді тексеруге арналған тест тапсырмаларын біріктіреді. Теориялық түсіндірмелер мен практикалық мысалдар студенттердің бағдарламалау логикасын түсінуіне және алған білімін нақты есептерді шешуде қолдануына мүмкіндік береді. Оқулықпен жұмыс барысында бағдарламалық мысалдарды орындау, бастапқы деректерді өзгерту, нәтижелерді талдау және қателерді анықтау ұсынылады. Практикалық тапсырмаларды орындау үшін Visual Studio, Visual Studio Code және .NET платформасының құралдарын пайдалануға болады. Оқулықты меңгеру нәтижесінде білім алушылар C# тілінде бағдарламалар құруды, деректер құрылымдарын пайдалануды, кластар мен объектілерді жобалауды және объектіге бағытталған бағдарламалау қағидаларын қолдануды үйренеді. Алынған білім кәсіби пәндерді игеруге, оқу жобаларын орындауға және бағдарламалық қосымшалар әзірлеуге негіз болады. Электрондық оқулық аудиториялық сабақтарда, зертханалық жұмыстарда және студенттердің өзіндік оқуында пайдалануға арналған.

Құрастырушы: Найзағарасева А.А., С. Сейфуллин

атындағы Қазақ агротехникалық

зерттеу университеті, «Ақпараттық жүйелер» білім беру бағдарламалары тобының аға оқытушысы, PhD.

МАЗМҰНЫ

Дәрістер

<u>1 дәріс. Объектіге бағытталған программалау негіздері. Кластар, объектілер және әдістерге кіріспе. Параметрленген әдістер.....</u>	<u>4</u>
<u>2 дәріс. Конструкторлар және деструкторлар. Класс мүшелеріне қол жеткізуді басқару. Қасиеттер. Индексаторлар.....</u>	<u>12</u>
<u>3 дәріс. Объектілерді әдістерге беру. ref және out модификаторлары.....</u>	<u>19</u>
<u>4 дәріс. Әдістерден объектілерді қайтару.....</u>	<u>27</u>
<u>5 дәріс. Әдістерді асыра жүктеу. Статикалық кластар және кластың статикалық мүшелері.....</u>	<u>31</u>
<u>6 дәріс. Операторларды асыра жүктеу.....</u>	<u>40</u>
<u>7 дәріс. Ноль мән қабылдайтын типтер. Бөлшектенген кластар және бөлшектенген әдістер. Кеңейтуші әдістер.....</u>	<u>52</u>
<u>8 дәріс. Мұралау. Мұралау және атауларды жасыру. Көпдеңгейлі иерархияны құру.....</u>	<u>55</u>
<u>9 дәріс. Виртуалды әдістер және оларды қайта анықтау. Абстрактілі кластар.....</u>	<u>62</u>
<u>10 дәріс. Интерфейстер. Интерфейстік сілтемелер, интерфейстік қасиеттер мен индексаторлар. Интерфейстерді мұралау.....</u>	<u>66</u>
<u>11 дәріс. Аластамалық жағдайларды өңдеу. Аластамаларды ұстамаудың салдары. Бірнеше catch операторларын пайдалану.....</u>	<u>72</u>
<u>12 дәріс. Аластамаларды өзіндік генерациялау. Аластамалардың туынды кластары...</u>	<u>77</u>
<u>13 дәріс. Делегаттар. Топтық адрестеу. Оқиғалар.....</u>	<u>82</u>
<u>14 дәріс. Анонимдік функциялар. Анонимдік әдістер және лямбда-өрнектер.....</u>	<u>89</u>
<u>15 дәріс. Жалпыламалар.....</u>	<u>96</u>

Зертханалық жұмыстар

<u>Лабораториялық жұмыс 1. Қолданушы функцияларын құру.....</u>	<u>98</u>
<u>Лабораториялық жұмыс 2. Кластарды құру және оларға әдістері қосу.....</u>	<u>103</u>
<u>Лабораториялық жұмыс 3. Конструкторларды құру. Класс мүшелеріне жабық қатынасуды жүзеге асыру.....</u>	<u>106</u>
<u>Лабораториялық жұмыс 4. Объектілерді әдістерге беруді жүзеге асыру.....</u>	<u>109</u>
<u>Лабораториялық жұмыс 5. Әдістерден объектілерді қайтаруды жүзеге асыру.....</u>	<u>111</u>
<u>Лабораториялық жұмыс 6. Әдістерді асыра жүктеуді жүзеге асыру.....</u>	<u>113</u>
<u>7 лабораториялық жұмыс. Операторларды асыра жүктеуді жүзеге асыру.....</u>	<u>115</u>
<u>8 лабораториялық жұмыс. Кеңейтуші әдістерді жүзеге асыру.....</u>	<u>117</u>
<u>9 лабораториялық жұмыс. Мұралауды жүзеге асыру.....</u>	<u>118</u>
<u>10 лабораториялық сабақ. Абстрактілі кластарды құру.....</u>	<u>120</u>
<u>11 лабораториялық сабақ. Интерфейстерді құру және жүзеге асыру.....</u>	<u>122</u>
<u>12 лабораториялық сабақ. Аластамалық жағдайларды өңдеу.....</u>	<u>123</u>
<u>13 лабораториялық сабақ. Туынды аластамалар кластарын құру.....</u>	<u>124</u>
<u>14 лабораториялық сабақ. Оқиғаларды жүзеге асыру.....</u>	<u>125</u>
<u>15 лабораториялық сабақ. Анонимдік функцияларды құру.....</u>	<u>126</u>

<u>Білімді тексеру материалдары.....</u>	<u>127</u>
<u>Қорытынды.....</u>	<u>138</u>
<u>Қолданылған әдебиеттер тізімі.....</u>	<u>139</u>

1 дәріс. Объектіге бағытталған программалау негіздері. Кластар, объектілер және әдістерге кіріспе. Параметрленген әдістер.

Мақсаты: студенттерде объектіге бағытталған программалаудың негізгі механизмдері және кластарды құру негіздері бойынша түсініктерін көрсетуге қабілет қалыптастыру.

Дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Инкапсуляция, полиморфизм, мұралау идеялары бойынша түсініктерін көрсету;
- Өрістері мен әдістері бар класты жазу форматын білетінін көрсету;
- Параметрленген әдістерді пайдалану ерекшеліктері бойынша түсініктерін көрсету.

Объектіге бағытталған программалау

C# тілінде тілінің негізгі ұғымы болып *объектіге бағытталған программалау* (ОБП) түсінігі саналады. ОБП әдістемесін C# тілі тілінен бөліп алуға болмайды, сондықтан мұндағы барлық программалар, ең төменгі дәрежеде дегеннің өзінде де, объектіге бағытталған болып табылады. Осыған орай, ең қарапайым программа жазу үшін де, ОБП-ның негізгі принциптерін қолдану өте маңызды, әрі пайдалы жұмыс болып саналады.

ОБП өзіне құрылымдық программалаудың ең жақсы идеяларын алып, оны жаңа түсініктермен толықтырды. Осының нәтижесінде программаларды ұйымдастырудың жаңа және үздік тәсілі жасалды. Ең жалпы түрде, программа екі тәсілдің біреуімен ұйымдас-тырылуы мүмкін: *код тарайналасында* (яғни, бұл іс жүзінде орындалып жатқан әрекеттер маңында) немесе *мәліметтер айналасында* (яғни кодтардың әсеріне ұшырайтын деректер маңында) атқарылатын істерді жүзеге асыру арқылы орындау. Құрылымдық программалау әдістері арқылы жасалған программалар, әдетте, кодтар айналасында ұйымдастырылады. Мұндай тәсілді "мәліметтерге әсер ететін код" деп қарауға болады.

Объектіге бағытталған программалар тіпті басқаша жұмыс істейді. Олар берілген деректерді негізге ала отырып, "мәліметтер кодқа (ішкі программаға) қол жеткізуді басқарады" деген басты қағиданы ұстанып, мәліметтер айналасында ұйымдастырылған. Программалаудың объектіге бағытталған тілінде *мәліметтер* (деректер) және соларды өңдеуге, яғни өзгертуге, түрлендіруге рұқсат етілген *кодтар* анықталады. Демек, *мәліметтер* *тіні* солармен *орындалатын операцияларды* дәлме-дәланықтай алады.

ОБП-ның принциптерін сүйемелдеу үшін, барлық объектіге бағытталған программалау тілдерінің (соның ішінде C# тілі тілі де бар) үш түрлі жалпы қасиеттері болуы тиіс, олар: *инкапсуляция*, *полиморфизм* және *мұралау* қасиеттері. Енді осы қасиеттерді әрқайсысын жеке-жеке қарастырып шығайық.

Инкапсуляция

Инкапсуляция – бұл өңделуге тиіс код пен оған берілетін мәліметтерді сырттың араласуынсыз біріктіретін механизм, ол, сонымен қатар, деректердің дұрыс пайдаланылуын да қадағалап отырады. Объектіге бағытталған тілде берілген мәлімет пен кодтың автоном-ды *қара жәшік* түрінде біріктіріліп тұруы да мүмкін. Осындай жәшіктің ішінде барлық қажетті мәліметтер мен оның өңдейтін код бірге орналасады. Код пен берілген мәліметтер осылайша бір-бірімен байланысқаннан кейін барып, объект құрылады. Басқаша айтқанда, *объект* дегеніміз – инкапсуляцияны сүйемелдейтін элемент.

Объектіде код пен мәліметтер немесе солардың әрқайсысы жеке-жеке *жабық* немесе *ашық* түрде орналасуы мүмкін. Жабық мәліметтер немесе код сол объектінің ішіндегі соның қалған бөлігіне ғана белгілі, әрі қол жетімді болады. Бұл жабық мәліметтердің немесе кодтың сол объектіден тысқары жерде қолданылуына жол бермейді, яғни оларға сырттан қол жеткізуге болмайды. Егерде мәліметтер немесе оның коды ашық болатын болса, онда олар объект ішінде анықталғанмен, программаның басқа бөліктерінде де пайдаланыла

береді, яғни оған қол жеткізуге болады. Көбінесе, объектінің ашық бөліктері оның жабық бөліктерімен басқарылатын интерфейс ұйымдастыру үшін қызмет етеді.

C# тілінде тіліндегі инкапсуляцияның негізгі бірлігі объект формасын анықтайтын *класс* болып табылады. Ол мәліметтерді және де солармен жұмыс істейтін кодты сипаттайды. C# тілінде тілі ортасындағы *класс сипаттамасы* класс экзemplяры болып табылатын объектілерді құру үшін қолданылады. Демек, класс дегеніміз, негізінде, объектіні құрудың бірсыпыра схемалық сипаттауларын беру тәсілі болып табылады.

Полиморфизм

Полиморфизм, бұл грекше "формалар жиыны" дегенді білдіреді, бұл – бір интерфейстің ұқсас іс-әрекеттердің бірнеше тобын (класын) біріктіріп, біртіндеп басқаруына мүмкіндік беретін қасиет.

Көбінесе полиморфизм ұғымының жалпы мағынасы "бір интерфейс – көптеген әдістер" түрінде айтылады. Бұл өзара байланысқан іс-әрекеттер тобы үшін жалпы бір интерфейс жасауға болады дегенді білдіреді. Полиморфизм *әрекеттердің жалпы бір класын сипаттау* үшін бір ғана интерфейс жасау арқылы программаны қарапайым түрде қысқартып жазу мүмкіндігін береді. Әрбір жеке жағдайда, белгілі бір нақты әрекетті (яғни әдісті) тандап алу бұл – компилятордың міндеті. Мұны программалаушының өзінің істеуі қажет емес. Оған жалпы интерфейс тістесек ақтай отырып, оны дұрыс пайдалана білу жеткілікті болып табылады.

Мұралау

Мұрагерлік дегеніміз жұмыс барысында бір объектінің басқа бір объектінің қасиеттерін алу (иелену) процесі болып табылады. Бұл өте маңызды процесс, өйткені ол иерархиялық жіктеу принципін қамтамасыз етеді. Егер тереңірек ойланатын болсақ, онда білімдеріміздің басым бөлігі иерархиялық жіктеу бойынша жалқыдан жалпыға қарай топтарға бөлініп жүйеленеді екен.

Егер объектілердің иерархиялық (сатылы түрдегі) ортақ қасиеттерін есепке алмасақ, онда әрбір объект үшін оның барлық қасиеттерін анықтауға тура келер еді. Ал егер мұралауды пайдалансақ, онда бұл объектінің осы кластағы басқа объектілерден ерекше болып келген қасиеттерін ғана көрсету жеткілікті болады. Ол сондай-ақ, өз класының барлық объектілеріне де ортақ жалпы қасиеттерін де (тегінің қасиеттерін) мұралай алады. Демек, мұрагерлік қасиеттерді есепке алу арқасында бір объект жалпы кластың белгілерін алған жеке бір экзemplярына (данасына) айналады.

Кластар мен әдістер

Класс объектінің формасын анықтайтын үлгі (шаблон) болып табылады, оны программада қолданылатын типтердің күрделі түрі деп те атауға болады. Онда керекті *мәліметтер* және соларды өңдейтін, яғни *түрлендіретін код* екеуі қатар көрсетіледі. Егер ең қарапайым кластар тек қана кодтан немесе мәліметтерден тұратын болса, нағыз толыққанды кластарда бұл екеуі де орын алады. C# тілінде объектілерді құруға арналған класс спецификациясы қолданылады, оларды класс даналары (экзemplяры) деп атайды. Сондықтан класты, негізінде, объект құруға арналған тізбектелген амалдар сипаттамасы деп қарауға болады. Класс оның объектісі құрылғаннан кейін ғана барып, компьютер жадынан белгілі бір орын алатын болады.

Мәліметтер (айнымалылар) класта анықталған *мәлімет-мүшелерде*, ал код (программа) – *функция-мүшелерде* орналасады. *Мәлімет-мүшелерге* (немесе өрістерге) жеке *даналар* аynamалылары және *статикалық аynamалылар* жатса, ал *функция-мүшелерге* – *әдістер* (функциялар), *конструкторлар*, *деструкторлар*, *индексаторлар*, *оқигалар*, *операторлар* және *қасиеттер* жатады.

Әдістер класта анықталған мәліметтерді өңдеуге арналған ішкі программалар болып табылады, ал көптеген жағдайларда олар сол мәліметтерге қол жеткізіп, есептеулерде пайдалануға мүмкіндік береді. Көбінесе, программаның басқа бөліктері класпен оның осы әдістері арқылы әрекеттеседі.

Әдіс бір немесе бірнеше операторлардан тұрады. C# тіліндегі сауатты жазылған кодта әрбір әдіс тек бір ғана функцияны орындайды. Әрбір әдістің оны шақыруды жүзеге асыратын арнайы аты болады. Жалпы, әдіске кез келген бір идентификаторды ат ретінде беруге болады. Дегенмен, **Main()** идентификаторы программаны бастайтын әдіске берілетін ат ретінде бекітілген. Кез келген әдістер аты ретінде C# тілінде тілінің түйінді сөздерін қолдануға да болмайды.

Төменде әдісті анықтаудың жалпы формасы келтірілген:

```
қатынасу түрі қайтарылатын_тип аты(параметрлер_тізімі) {  
  // әдіс тұлғасы  
}
```

мұндағы *қатынасу_түрі*— бұл қатынасу модификаторы, ол осы әдісті программаның қандай бөліктерінен шақыруға болатынын анықтайды. Келесі *қайтарылатын_тип* әдістің қайтаратын мәліметінің типін анықтайды. Бұл тип құрылатын кластың типін де анықтайды. Егер әдіс мән қайтармайтын болса, онда қайтарылатын типті **void** деп көрсету керек. Келесі *аты* әдіске берілетін нақты атауды көрсетеді. Аты ретінде жариялаудың осы аймағында қолдануға болатын кез келген сөзді таңдаймыз. Ең соңғы *параметрлер_тізімі*— бұл түтірмен бөліне отырып, типі мен идентификаторы қатар көрсетілген, екі сөзден тұратын анықтаулар тізбегі. Параметрлер әдісті шақыру кезінде оған берілетін *аргументтер* мәндеріне сәйкес айнымалылар тізбегі. Егер әдістің параметрлері болмаса, онда тізім бос болады.

Класс **class** деген түйінді сөз арқылы жасалады. Төменде бірнеше экземплярлар айнымалылары мен әдістерден тұратын ең қарапайым кластың жалпы құрастырылу бейнесі келтірілген.

```
class класс_аты {  
  // Экземпляр айнымалыларын жариялау  
  қатынас_түрі типі 1-айнымалы;  
  қатынас_түрі типі 2-айнымалы;  
  // ...  
  қатынас_түрі типі N-айнымалы;  
  
  // Әдістерді жариялау  
  қатынас_түрі қайтару_типі 1-әдіс (параметрлері) {  
    // әдіс операторлары  
  }  
  қатынас_түрі қайтару_типі 2-әдіс (параметрлері) {  
    // әдіс операторлары  
  }  
  // ...  
  қатынас_түрі қайтару_типі N-әдіс (параметрлері) {  
    // әдіс операторлары  
  }  
}
```

Әрбір айнымалыны немесе әдісті жариялау алдында *қатынас_түрі* көрсетілетініне назар аударыңдар. Бұл қатынас спецификаторы, мысалы, **public** сөзі, ол осы класс мәлімет-мүшесіне қатынасу, яғни қол жеткізу жолын анықтайды. Класс мүшелері өз аймағында жабық (**private**) немесе жалпы ашық (**public**) болуы мүмкін. Спецификаторды көрсету міндетті емес, егер ол жоқ болса, онда жарияланатын мүше класс аймағында жабық болып саналады. Қатынасу спецификаторы келесі тарауларда толығырақ қарастырылады.

Кластарды нақты мысалдар арқылы көрсету үшін кітап туралы ақпараттарды біртіндеп жасыратын (инкапсуляциялайтын) класс құрайық. Бұл класта кітап туралы ақпараттың бес

элементі: кітаптың аты, кітаптың авторы, жарыққа шыққан жылы, баспаның аты және беттер саны сақталатын болады.

Бір кітаптың баспа табақтарының саны класс құрамында орналасқан беттер саны өрісінің мәндеріне байланысты болады, сол себепті мұндағы есептеуді кластың өзінде орындаған дұрыс. Оған қоса, бұл есептеуді класқа енгізе отырып, біз бұл класты пайдаланатын басқа программаларды осы есептеуді өздерінің жүргізуінен босатамыз. Осыған орай кодты қайталау да орын алмайды. Және де класқа баспа табақтарының санын есептеуді қосу объектіге бағытталған құрылымды жақсартуға ықпал етеді, өйткені кітапқа тікелей байланысты шамалар класс құрамында орналасқан (басқаларынан жасырылған – инкапсуляцияланған).

```
class Kitap {
    // экземпляр айнымалылары
    public string aty; // кітаптың аты
    public string avtor; // автордың аты-жөні
    public int zhyly; // кітаптың шыққан жылы
    public string baspa; // баспаның аты
    public int bet_sany; // кітаптың беттер саны

    // класс әдістері
    public int baspa_tabagy() { return (int)(bet_sany/16); }
    //кітаптың толық баспа табақтарының санын анықтау функциясы

    public void Shygaru()
    { Console.WriteLine(aty + " кітабының авторы - " + avtor + ",
    ол " + baspa + "баспасынан " + zhyly + " жылы жарық көрді, беттер
    саны: " + bet_sany); }
}
```

Экземпляр айнымалысы қатынасу спецификаторынан бөлек, жай жергілікті айнымалы тәрізді жарияланады. **Class** сөзі мәліметтердің жаңа типі құрылғанын білдіреді.

Класс әдістерін қарастырайық.

```
public void Shygaru() { }
```

Бұл жолда **Shygaru** атты параметрлері жоқ әдіс жарияланған. Ол үшін **public** типі көрсетілген, яғни оны программаның кез келген бөлігінен шақырып пайдалануға болады. **Shygaru()** әдісі **void** типіндегі бос мән қайтарады, яғни ол оны шақырған программа бөлігіне ешқандай да мән қайтармайды. Қарастырылып отырған жол жүйелі жақшамен аяқталады, ол әдістің ішкі операторларын (тұлғасын) орналастыруға жол ашады.

Shygaru() әдісінің ішкі тұлғасы тек бір оператордан ғана тұрады.

```
Console.WriteLine(aty + " кітабының авторы - " + avtor + ", ол "
+ baspa + "баспасынан " + zhyly + " жылы жарық көрді, беттер саны:
" + bet_sany);
```

Бұл оператор кітап туралы ақпаратты экранға шығарады. Ал **Kitap** типіндегі әрбір объектіде **aty**, **avtor**, **baspa**, **zhyly**, **bet_sany** айнымалыларының өз көшірмелері болғандықтан, **Shygaru()** әдісін шақырғанда, есептеулерді орындау кезінде осы айнымалылардың шақырған объектідегі көшірмелері пайдаланылады.

Shygaru() әдісі жабылатын жүйелі жақшамен аяқталады. Бұл жақша кездескенде, басқару кері қарай программаны шақырған бөлікке беріледі.

Kitap типіндегі белгілі бір объект жасау үшін, келесі операторды пайдаланамыз.

```
Kitap kitap1 = new Kitap(); // Kitap типіндегі объект жасау
```

Осы операторды орындағаннан кейін жаңа **kitab1** объектісі **Kitap** класының бір данасы (экземпляр) болып табылады, яғни ол нақты "физикалық" түрғыда пайда болады. **new** операторы объект үшін динамикалық түрде (яғни орындалу кезінде) компьютер жадынан орын бөледі де, оған сілтеме қайтарады, сонан соң сол сілтеме айнымалыда сақталады.

kitab1 айнымалысын жариялауды ол сілтеме жасап тұратын объектіні құрудан бөлек жасауға болады, ол былайша орындалады.

```
Kitap kitab1;           // объектіге сілтеме жасауды жариялау
kitab1 = new Kitap();   // Kitap типіндегі объект үшін жадыдан
// орын бөлу
```

Бірінші жолда **Kitap** типіндегі объектіге сілтеме ретінде **kitab1** айнымалысы жарияланады. Мұндағы **kitab1** – объектіге сілтеме жасай алатын айнымалы, бірақ оның өзі объект болып табылмайды. Ал екінші жолда **Kitap** типіндегі жаңа объект құрылады да, сол объектіге сілтеме **kitab1** айнымалысына меншіктеледі.

Кластардың объектілеріне сілтеме арқылы қол жеткізуге болатын мүмкіндікке байланысты кластар *сілтемелік тип* болып аталып кетті. **Kitap1** айнымалысы объектінің өзін емес, тек оған сілтемені ғана сақтайды.

Book типіндегі әрбір объект **aty**, **avtor**, **zhyly**, **baspa**, **bet_sany** экземплярлар айнымалыларына сәйкес, өз көшірмелерін құрып сақтайтын болады. Осы айнымалыларды пайдалану үшін класс мүшесімен қатынас құратын, яғни оны қолданатын оператор керек болады, ол өз құрылымына сәйкес нүкте-оператор деп аталады. Нүкте-оператор объектатын класс мүшесінің атымен байланыстыру үшін қажет. Төменде нүкте-оператордың жалпы жазылу формасы көрсетілген.

объект.мүше

Бұл формада объект сол жақта, ал мүше – оң жақта тұрады. Мысалы, **kitab1** объектісінің **avtor** айнымалысына "**Muhtar Auezov**" мәнін меншіктеу келесідей оператор арқылы орындалады.

```
kitab1.avtor = "Muhtar Auezov";
```

Жалпы түрде, нүкте-оператор экземплярлар айнымалылары мен әдістерді (функцияларды) пайдалану үшін керек болады.

Мысал 1. **Kitap** класының объектілерін пайдаланатын программаның толық мысалы.

```
// Kitap класы пайдаланылатын программа
using System;
class Kitap {
    public string aty; // кітаптың аты
    public string avtor; // автордың аты-жөні
    public int zhyly; // кітаптың шыққан жылы
    public string baspa; // баспаның аты
    public int bet_sany; // кітаптың беттер саны

    // класс әдістері
    public int baspa_tabagy() { return (int)(bet_sany/16); }
    // кітаптың толық баспа табақтарының санын анықтау функциясы
    public void Shygaru() { Console.WriteLine("\n" + aty + "\n" + "
kitabinin avtory - " + avtor + ", ol \n" + baspa + "\n" + baspasynan "
+zhyly + " zhyly zharyk kordi, better sany: " + bet_sany + "."); }
}
// Бұл класта Kitap типіндегі объект жарияланады
class Program {
```

```

static void Main() {
    Kitap kitap1 = new Kitap(); //Kitap типіндегі объект құру

    kitap1.aty = "C# 4.0 The Complete Reference";
    kitap1.avtor = "Herbert Shildt";
    kitap1.zhyly = 2010;
    kitap1.baspa = "McGraw-Hill Education";
    kitap1.bet_sany = 976;
    kitap1.Shygaru();
    int bt = kitap1.baspa_tabagy();
    Console.WriteLine("\""+kitap1.aty + "\" kitabi " +bt + "
    baspa tabagynan turady");
    Console.ReadKey();
}
}

```

Осы программа келесідей нәтиже береді.

```

"C# 4.0 The Complete Reference" kitabinin avtory -
Herbert Shildt ol "McGraw-Hill Education" baspasynan
2010 zhyly zharyk kordi, better sany: 976.
"C# 4.0 The Complete Reference" kitabi 61 baspa
tabagynan turady

```

Бұл программа екі кластан: **Kitap** және **Program** тұрады. **Program** класында **Main()** әдісінің көмегімен алдымен **Kitap** класының **kitap1** экземпляры құрылады, содан кейін **kitap1** экземпляры айнымалыларын ары қарай пайдалану үшін, оларға мәндер меншіктеледі. Мұндағы атап айтатын бір жайт – **Kitap** және **Program** – екеуі екі бөлек кластар болып табылады. Олардың арасындағы жалғыз байланыс – біреуінде екіншісінің бір экземпляры құрылады. Олар жеке кластар болғанмен, **Program** класының коды **Kitap** класының мүшелерін пайдалана алады, өйткені олар ашық (**public**) кластар түрінде жарияланған. Егер оларды жариялағанда, **public** қатынасу спецификаторы көрсетілмесе, онда олардың қатынасуы **Kitap** класымен шектеліп, соның әсерінен оларды **Program** класында пайдалана алмайтын едік.

Ары қарай кодтың **Main()** әдісіндегі келесі жолын талдайық.

```

kitap1.Shygaru();

```

Бұл жолда **Shygaru()** әдісі **kitap1** объектісі үшін шақырылады. **Shygaru()** әдісі шақырылған соң, программаны басқару осы әдіске беріледі. Ал ол жұмысын аяқтаған соң, жұмысты басқару кері қарай программаның әдісті шақырған бөлігіне беріледі де, жұмыс сол әдісті шақырған кодтан ары қарай жалғасады.

Shygaru() әдісінде келесі жайтқа назар аудару керек: **aty**, **avtor**, **zhyly**, **baspa**, **bet_sany** экземплярылар айнымалыларын пайдалану нүкте-операторынсыз тікелей орындалады. Егер әдісте оның бір класында анықталған экземпляр айнымалылығы қолданылатын болса, онда ол тікелей объектіге сілтеме жасалмай-ақ, нүкте-операторынсыз орындалады. Әдіс әрқашанда өз класындағы белгілі бір объектіге қатысты шақырылады. Шақыру орындалысымен, объект белгілі болады.

Жалпы, әдістен кері қайту екі шарт бойынша жүзеге асырылуы мүмкін. Біріншіден, әдістің ішкі операторларын (тұлғасын) жабатын жүйелі жақша кездескенде, мысалы, ол жоғарыдағы программада **Shygaru()** әдісінің мысалында орын алып тұр. Екіншіден, ол **return** операторы орындалғанда жүзеге асады. **return** операторының да екі түрі болады: бірі – **void** типіндегі әдістер үшін, яғни олар мән қайтармайтын әдістерде орын алады, ал екіншісі – нақты мәндер қайтаратын әдістерде кездеседі.

Void типіндегі әдісті бірден аяқтау үшін **return** операторының келесі формасын пайдалану жеткілікті.

return;

Бұл оператор орындалғанда, жұмысты басқару программаның әдісті шақыратын бөлігіне қайтарылады да, әдіс ішіндегі қалған операторлар орындалмай қалады.

Қайтарылатын мәндер программалауда әртүрлі мақсаттар үшін қолданылады. Кейбір жағдайларда, мысалы, **baspa_tabagy()** әдісіндегі сияқты, қайтарылатын мән белгілі бір есептеу нәтижесі болады, ал кейде – ол жай ғана әдістің табысты немесе қатемен аяқталғанын білдіреді, ал тағы бір кездерде – жалпы жұмыс жағдайының (қалпының) кодын береді. Қойылған мақсаттарға қарамастан, мән қайтаратын әдістерді пайдалану C# тілінде тілінде программалаудың ажырамас бір бөлігі болып табылады.

Әдістен алынған мәнді программаның шақыру бөлігіне қайтару үшін **return** операторының келесі формасы қолданылады:

return мән;

мұндағы *мән* – бұл нақты қайтарылатын мән.

Мұнда мынаған назар аударыңыз: **baspa_tabagy()** әдісі шақырылған кезде, ол меншіктеу операторының оң жақ бөлігінде көрсетіледі. Ал оның сол жағында **baspa_tabagy()** әдісі қайтаратын мәнді қабылдап алатын айнымалы орналасады. Сол себепті, келесі оператор орындалған соң,

```
int bt = kitap1.baspa_tabagy();
```

bt айнымалысында кітаптың баспа табақтарының саны (**kitap1** объектісі) сақталып қалады.

Және де енді **baspa_tabagy()** әдісінің қайтарылатын мәні **int** болатынына назар аударыңыз. Бұл программаның әдісті шақыру бөлігіне бүтін мән қайтарылатынын білдіреді. Әдістің қайтаратын мәні типінің маңызы зор, өйткені әдістің қайтаратын мәні әдісте көрсетілген мәліметтер типіне сәйкес болуы тиіс.

Мұндағы **Kitap** және **Program** кластарына бір файлда орналасу міндетті емес. Бұлардың әрқайсысын жеке файлға, мысалы, Kitap.cs және Program.cs жазып, екі файлды да өз жобамызға енгізіп, сонан кейін барып компиляциялау керек.

Параметрлерді пайдалану

Әдісті шақырған кезде оған бір немесе бірнеше мәндерді беруге болады. Әдіске берілетін мән *аргумент* деп аталады. Ал аргументті қабылдайтын айнымалы *формальді параметр* немесе жай параметр деп аталады. Параметрлер әдістің атынан кейін жай жақша ішінде жарияланады. Параметрлерді жариялау синтаксисі қарапайым айнымалыларды жариялау сияқты орындалады. Ал параметрлердің әсер ету аймағы әдістің ішкі тұлғасы (денесі) болып табылады. Әдіске аргументтерді берудің ерекше жағдайларынан басқа кездерде параметрлер кез келген басқа айнымалылар тәрізді жұмыс істей береді.

Мысал 2. Төмендегі программада параметрлі әдісті пайдалану мысалы келтірілген.

```
using System;
class Shenber {
    public int xcoord; // шеңбер центрінің x осі
    // бойынша координатасы
    public int ycoord; // шеңбер центрінің y осі
    // бойынша координатасы
```

```

public double radius; // шеңбер радиусы
public double uzyndyq() { return 2 * 3.14 * radius; }

public bool ishki_nukte(int x, int y)
{
    double d = Math.Sqrt(Math.Pow(xcoord - x, 2) +
        Math.Pow(ycoord - y, 2));
    if (d < radius) return true;
    return false;
}
}
class Program {
    static void Main() {
        Shenber shenber1 = new Shenber();
        shenber1.xcoord = 2;
        shenber1.ycoord = 2;
        shenber1.radius = 3;
        int x = 3, y = 4;
        bool n = shenber1.ishki_nukte(x, y);
        Console.WriteLine("Koordinatalary (" + x + ", " + y + ")
        bolatyn nukte centri (" + shenber1.xcoord + ", " +
        shenber1.ycoord + ") nuktesinde ornalasqan, radiusy " +
        shenber1.radius + " bolatyn nuktenin ishinde ");
        if(n) Console.WriteLine("zhatady");
        else Console.WriteLine("zhatpaiady");
        Console.ReadKey();
    }
}

```

Shenber класында **ishki_nukte()** әдісі қолданылған, оған шеңбер ішінде жататын нүктенің координаталары берілсе, **true** мәнін қайтарады. Ал басқа (қарама-қарсы) жағдайда, ол **false** мәнін қайтарады.

Бұл программаның нәтижесі төмендегідей болады:

```

Koordinatalary (3,4) bolatyn nukte centri (2,2)
nuktesinde ornalasqan, radiusy 3 bolatyn nuktenin
ishinde zhatady

```

Әдісте бір немесе бірнеше параметрлер болуы мүмкін. Оның әрбір параметрі бір-бірінен үтір арқылы бөліне отырып жарияланады, олардың әрқайсысы үшін, басқаларынан өзгеше, өз типі көрсетіледі. Мысалы, **ishki_nukte(int x, int y)** коды дұрыс жазылған болып табылады.

Видеожазбалар:

Класс пен объектілер <https://youtu.be/ytExcntEBNI?si=UwcyDP3Tm9L9JE0K>

Әдістер <https://youtu.be/hmHSiffMkWQ?si=DppLIUPwV4HyVa2C>

Параметрлер https://youtu.be/6eD0i_H4-Is?si=Kyxx6iMtY_jePWM1

2 дәріс. Конструкторларжәне деструкторлар. Класс мүшелеріне қол жеткізуді басқару. Қасиеттер. Индексаторлар.

Дәрістің мақсаты: студенттерде конструкторлар, деструкторлар, қасиеттержәне индексаторлардың қызметі және оларды пайдалану ерекшеліктері туралытүсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- конструкторлар, деструкторлар, қасиеттер және индексаторлардың қызметі туралы түсініктерін көрсету;
- конструкторлар, деструкторлар, қасиеттер және индексаторларды жариялау форматы бойынша білімдерін көрсету.

Конструкторлар

Жоғарыдағы программа мысалдарында **Shenber** типіндегі әрбір объектідегі экземпляр айнымалыларын қолдап келесідей операторлартізбегі арқылыинициалдауға тура келгенеді.

```
shenber1.xcoord = 2;  
shenber1.ycoord =2;  
shenber1.radius = 3;
```

Мұндай тәсіл әдетте C# тілінде тілінде кәсіби түрде жазылған кодтарда қолданылмайды. Оның үстіне,мұнда қате де кетуі мүмкін(сіз өрістердің біріниинициалдаудыұмытып та кетуіңіз де мүмкін ғой). Дегенмен, осындай сәттердекөмектесетін жақсы бір тәсіл бар, ол: конструкторларды қолдану.

Конструктор объектіні құру кезінде бірден инициалдайды. Конструктордың аты оның класс атымен бірдей болып келеді де, синтаксис жағынан ол әдіске ұқсайды. Бірақ конструкторлардың тікелей көрсетіліп қайтарылатынтипі жоқ.

Төменде конструктордың жапы формасы келтірілген.

```
қатынасу_типі класс_аты(параметрлер_тізімі) {  
    // конструктор тұлғасы  
}
```

Көбінесе конструкторкласта анықталған экземпляр айнымалыларының бастапқы мәндерін беру үшін қолданылады немесе басқа да толық қалыптасқан объект жасау үшін қажет етілетін орнатупроцедураларын орындау үшін пайдаланылады. Одан қоса, конструкторлар көбінесе класта шақырылатындықтан, оның қатынасу типінің модификаторы **public** болып келеді. Ал *параметрлер_тізімі* кейде бос болады, ал көбінесе көрсетілген бір немесе бірнеше параметрлер тізімінен тұрады.

Программада анықтасаңыз да немесе анықтамасаңыз да, бәрі бір барлық кластарда конструкторлар болады, өйткені C# тілінде, алдын ала келісім бойынша, конструктор автоматты түрде құрылады, ол барлық экземпляр айнымалыларын берілетін мәндермен инициалдайды. Мәліметтердің көптеген типтеріүшін алдын ала келісім бойынша берілетінмән нөл болады, **bool** типі үшін – **false** мәні алынса, ал сілтемелі типтер үшін – бос мән алынады. Бірақ егер сіз программада өз конструкторыңызды анықтайтын болсаңыз, онда келісім бойынша құрылатын конструктор пайдаланылмайды.

Мысал7.3. Конструкторды қолданудың қарапайым мысалы.

```
using System;  
class Shenber {  
    public int xcoord;  
    public int ycoord;
```

```

public double radius;
public Shenber()
{
    xcoord = 2;
    ycoord = 2;
    radius = 1;
}
}
class Program {
    static void Main() {
        Shenber shenber1 = new Shenber();
        Console.WriteLine("shenber centri: " + shenber1.xcoord + ", " +
shenber1.ycoord + ", shenber radiusy - " + shenber1.radius);
        Console.ReadKey();
    }
}

```

Бұл мысалдағы **Shenber** класының конструкторымынадай түрде берілген.

```

public Shenber()
{
    xcoord = 2;
    ycoord = 2;
    radius = 1;
}

```

Мұндағы конструктордың **public** түрінде белгіленгеніне назар аударыңыз. Ол өз класы сыртындағы кодтардан да шақырылатын болуы тиіс. Ол объектіні құру кезінде **new** операторында шақырылады. Мысалы, келесі жолда:

```
Shenber shenber1 = new Shenber();
```

Конструкциялаудан кейін **shenber1** объектісінің айнымалылары сәйкесінше 2, 2, 3 мәндеріналады. Сонымен, жоғарыда келтірілген кодты орындау нәтижесімынадай болады.

```
shenber centri: 2, 2, shenber radiusy - 1;
```

Алдыңғы мысалда параметрсіз конструктор пайдаланған болатын. Кейбір жағдайларда ол жеткілікті болғанмен, көбінесе конструктор бір немесе бірнеше параметрлерді қабылдауы тиіс. Конструкторға параметрлер әдістерге енгізілетін сияқты түрде енгізіледі. Ол үшін параметрлерді конструктор атынан кейін жайжақшалар ішінде жариялау жеткілікті.

Мысал 7.4. Параметрленген **Shenber** конструкторын қолдану мысалы.

```

using System;
class Shenber {
    public int xcoord;
    public int ycoord;
    public double radius;
    public Shenber(int x, int y, int r)
    {
        xcoord = x;
        ycoord = y;
        radius = r;
    }
}
class Program {
    static void Main()
    {
        Shenber shenber1 = new Shenber(2,2,1);
        Console.WriteLine("shenber centri: " + shenber1.xcoord + ", " +
shenber1.ycoord + ", shenber radiusy - " + shenber1.radius);
        Console.ReadKey();
    }
}

```

}

Бұлкодты орындау нәтижесі мынадай болады:

```
shenber centri: 2, 2, shenber radiusy - 1;
```

Деструкторлар

Жоғарыда көрсетілгендей, new операторын қолдану кезінде жасалатын объектілер үшін компьютер жедел жадынан алынған бос орын көлемі (буфердегі) динамикалық түрде бөлінеді. Әрине, жедел жады көлемі шексіз емес, ол ертеме, кеш пе, әйтеуір бітеді. Сондайкездерде, керекті объект құру үшін бос жады мөлшерінің болмай қалуы new операторының дұрыс орындалмауына алып келеді. Міне, сондықтан да компьютер жадын динамикалық түрде бөлудің кез келген схемасының басты функциясы пайдаланылмайтын объектілерден жадыны босатып, оны келесі жұмыстарға қолдануға дайындау болып табылады.

C# тілінде "қоқысты жинау" жүйесінің өзі компьютер жадын пайдаланылып болған объектілерден байқаусыз және программалаушының қатысуынсыз автоматты түрде босатады. "Қоқысты жинау" былай жүргізіледі. Егер объектіге сілтеме болмаса, онда ондай объект керексіз деп есептеледі де, ол орналасқан жады босатылып, буферге қосылады. Осылай "тазартылған" жады басқа объектілерге бөлініп берілетін болады. "Қоқысты жинау" программаны орындау барысында, анда-санда жүргізіліп отырады. Ол жұмыстан босаған бір-екі объект үшін орындала салмайды, өз кезегін күтеді. Сонымен, "қоқысты жинау" қай кезде болатынына алдымен білуге немесе болжауға болмайды.

C# тілінде "қоқысты жинау" жүйесі арқылы қажет болмайтын объектіні жою алдында орындалатын әдісті шақыруды анықтау мүмкіндігі бар. Мұндай әдіс деструктор деп аталады және ол объектімен жұмыс істеу мерзімін аяқтауға кепілдік беретін ерекше жағдайларда пайдаланыла алады. Мысалы, деструктор босатылуға тиіс объект алып тұрған жүйелік ресурсты кепілді түрде босату үшін қолданылады. Бірақ деструкторлар тек сирек туындайтын ерекше жағдайларда ғана қолданылатын құралдар болып табылатынын айта кету керек.

Төменде деструктордың жалпы жазылу формасы келтірілген:

```
~класс_аты() {  
    // деструктор коды  
}
```

мұндағы *класс_аты* – нақты бір кластың аты. Деструктор конструктор тәрізді жарияланады, тек оның аты алдына "тильда" (~) белгісі қойылады. Деструктордың қайтарылатын типі мен оған берілетін аргументтері болмайды.

Деструкторды бір класқа қосу үшін, оны класс құрамына мүше етіп енгізу жеткілікті. Ол өз класындағы белгілі бір объектіні жою керек болған сайын іске қосылады. Деструкторда объектіні жою алдында орындалуға тиіс әрекеттерді көрсетуге болады.

Мысал 7.5. Төменде деструкторды қолдануды көрсететін программа мысалы келтірілген. Ол программада көптеген объектілер құрылады және өшіріледі. Ол процестің орындалуы барысындағы кейбір сәттерде "қоқысты жинау" іске қосылып, қажетсіз объектілерді өшіру үшін деструкторлар шақырылып жатады.

```
using System;  
class Destruct {  
    public int x;  
    public Destruct(int i) {  
        x = i;  
    }  
    // Объектіні өшіру кезінде шақырылады  
    ~Destruct() {  
        Console.WriteLine("Joiylady " + x);  
    }  
    // Объектіні жасайды да және оны бірден жояды
```

```

public void Generator(int i) {
    Destruct o = new Destruct (i);
}
}

class Program {
    static void Main() {
        int count;
        Destruct ob = new Destruct(0);
        /* Ал енді көптеген объектілер жасалады.
Белгілі бір сәттерде "қоқыс жинау" орын алады.
Ескерту: "қоқыс жинауды" екпінді ету үшін жасалатын объектілер
санын көбейту керек шығар */
        for (count=1; count < 100000; count++)
            ob.Generator(count);
        Console.WriteLine("Dayin!");
    }
}

```

Программаорындалуының нәтижесі:

```

...
Joiylady 4403
Joiylady 4402
Joiylady 4401
Joiylady 4400
Joiylady 4399
Joiylady 4398
Joiylady 4397
Joiylady 4396
Joiylady 4395
Joiylady 4394
Joiylady 4393
...

```

Бұл программа былай жұмыс істейді. Конструктор **x** айнымалысын белгілі бір мәнмен инициалдайды. Бұл мысалда **x** айнымалысы объект идентификаторы рөлін атқарады. Ал деструктор объект жұмысын бітірген соң, **x** айнымалысының мәнін шығарады. **Destruct** типіндегі объектіні жасап және бірден жоятын **Generator()** әдісі ерекше қызығушылық туғызады. Алдымен **Program** класында **Destruct** типіндегі **ob** бастапқы объектісі жасалады, ал сонан соң біртіндеп кезекпен 100 мың объект жасалып өшіріледі. Осы процестің әртүрлі сәттерінде "қоқысты жинау" орын алады. Оның қаншалықты жиі орындалуы – бірнеше факторларға байланысты болады, оның ішінде бастапқы бос жады көлемі, пайдаланылатын операциялық жүйе типі және т.с.с. бар. Дегенмен белгілі бір сәтте деструктор арқылы қалыптасатын хабарламалар пайда бола бастайды. Егер де олар программа жұмысының соңына дейін, яғни "Дайын!" деген хабарлама шыққанша пайда болмаса, онда жасалынатын объектілердің шектеулі санын **for** цикліндегі қадамдар санын көбейте отырып арттыру керек.

Тағы бір маңызды ескерту: **~Destruct()** деструкторында **WriteLine()** әдісі тек осы мысалдың көрнекті болып көрінуі үшін ғана шақырылады. Көбінесе деструктор тек өз класында анықталған экземпляр айнымалыларына ғана әсер етуі керек.

Деструкторларды шақыру реттілігі дәлме-дәл анықталмағандықтан, оларды программаның орындалуы барысындағы белгілі бір сәтте жүзеге асатын әрекеттерді орындау үшін қолдануға болмайды. Дегенмен, "қоқысты жинауды" қолдап инициалдау көптеген жағдайларда ұсынылмайды, өйткені бұл программаның тиімділігін төмендетуге әкеліп соқтыруы мүмкін. Оның үстіне, "қоқысты жинау" жүйесінің өз ерекшеліктері бар – егер тіпті "қоқысты жинауды" тікелей орындауды сұратсақта, нақты объектінің қашан жойылатынын бәрі бір алдын ала білуге болмайды.

Мысал 7.6. Кесінді класын құрыңыз. Класс кесіндінің екі ұшының координаталарын анықтайтын 4 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. кесіндінің ұзындығын анықтау;
2. өрістерді инициалдауға арналған конструкторлар құру.

```

using System;
class Kesindi {
    public int x1, y1, x2, y2;
    public Kesindi() { x1 = y1 = 0; x2 = y2 = 1; }
    public Kesindi(int a, int b, int c, int d) {
        x1 = a; y1 = b; x2 = c; y2 = d;
    }
    private double uzyndygy() { return Math.Sqrt
        (Math.Pow(x2 - x1, 2) + Math.Pow(y2 - y1, 2)); }
    public void Shygaru()
    {
        Console.Write("kesindi koordinatalary: ({0}, {1}) zhane
        ({2}, {3}), onyn uzyndygy - {4:###.##}", x1, y1, x2,

```

```

        y2, uzyndygy());
    }
}
class Program {
    static void Main() {
        Kesindi A = new Kesindi();
        Kesindi B = new Kesindi(2, 2, 7, 8);
        Console.WriteLine("A ");
        A.Shygaru();
        Console.WriteLine();
        Console.WriteLine("B ");
        B.Shygaru();
        Console.WriteLine();
        Console.ReadKey();
    }
}

```

Программаның орындалу нәтижесі:

```

A kesindi koordinatalary: (0, 0) zhane (1, 1), onyn uzyndygy -
1,41
B kesindi koordinatalary: (2, 2) zhane (7, 8), onyn uzyndygy -
7,81

```

Класс мүшелеріне қол жеткізу(қатынасу) модификаторлары

Класс мүшелеріне қол жеткізуді (қатынасу) басқару жұмысы төрт түрлі *қатынасу модификаторы* арқылы: **public**, **private**, **protected** және **internal** ұйымдастырылады. **protected** модификаторы тек мұралауға байланысты іс-шараларды атқарады. Ал **internal** модификаторынегізіненкеңейтілетін программаларға немесе кітапханаларға байланысты жинақтау (*сборки*) мақсатында қолданылады.

Егер класс мүшесі **public** спецификаторымен белгіленсе, оны программадағы кез келген басқа кодтарда, мысалы, басқа класта анықталған әдістерде де қолдануға болады.

Ал егер класс мүшесі **private** спецификаторымен белгіленсе, онда оны осы кластың басқа мүшелерінде ғана пайдалануға болады. Сондықтан, басқа кластардың әдістері бұл кластың жабық мүшелерін (**private**) пайдалана алмайды. Ал егерде ешбір спецификатор көрсетілмесе, онда класс мүшесі, келісім бойынша, жабық болып саналады. Сол себепті кластың жабық мүшелерін құрған кезде **private** спецификаторын жазу міндетті емес.

Қатынасу спецификаторы жеке мүшенің типін сипаттау бөлігінің алдында көрсетіледі. Бұл класс мүшесін жариялау операторы осыдан басталуы тиіс дегенді білдіреді. Сонымен, программаларда қатынас түрлері былай көрсетіледі:

```

public string avtor;
private int zhyl;
private bool ishki_nukte(int x, int y) { // ...}

```

public және **private** модификаторларының айырмашылығын көрсету үшін келесі мысалды қарастырамыз.

Мысал 9.1. Класс мүшелерін қолданудағы **public** және **private** қатынасу түрлерінің (модификаторлары) айырмашылықтары.

```

using System;
class Shenber {
    private int xcoord; // private арқылы жабық қатынас
    int ycoord;         // келісім бойынша жабық қатынас
    public int radius;  // public арқылы ашық қатынас
    // Класс мүшелері осы класс ішіндегі
    // жабық (private) мүшелерді пайдалана алады
}

```


- Егер объектіге зиян келтіре алатын мүшелер дұрыс пайдаланылмайтын болса, олар жабық болуы тиіс. Ондай мүшелерді пайдалануды, қате болмайтындай етіп, ашық әдістер арқылы ұйымдастыру керек.
- Жабық мәліметтердің мәндерін тағайындайтын (орнататын) және қабылдайтын әдістер ашық болуы тиіс.
- Экземпляр айнымалыларының жабық болуына ешқандай негіз болмайтын жағдайда ғана оларды ашық етіп алуға болады.

Видеожазбалар:

Конструкторлар мен деструктуралар https://youtu.be/lk8nrSambjw?si=9fZ91fCNpcQy_tgF
Қасиеттер <https://youtu.be/negZJdIzNRs?si=0wNuNg9rOuQCWy1e>

3 дәріс. Объектілерді әдістерге беру. ref және out модификаторлары.

Дәрістің мақсаты: студенттерде объектілермен әдістердің құрамында жұмыс істеу ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Әдістерге объектілерді беру тәсілдері бойынша түсініктерін көрсету;
- ref және out модификаторларын пайдалану жағдайларын ажырату.

Бұған дейін әдістерге параметрлер бергенде `int` немесе `double` типтері қолданылды. Бірақ әдістерге сілтемелік типтегі параметрлерді де беруге болады, бұл үрдіс ОБП-да кең таралған. Келесі мысалда объектілерді сілтеме бойынша әдіске беру қарастырылады.

Мысал 1. Әдістерге сілтеме бойынша объектілер беру мысалы.

```
using System;
class Shenber {
    public int xcoord;
    public int ycoord;
    public double radius;
    public Shenber(int x, int y, int r)
    { xcoord = x;
      ycoord = y;
      radius = r;
    }
    public void Shygaru() { Console.WriteLine("shenber centrinin
koordinatalary: (" + xcoord + ", " + ycoord + "),
shenber radiusy = " + radius);
    public bool qiylysu(Shenber ob) {
        double d = Math.Sqrt(Math.Pow(xcoord - ob.xcoord, 2) +
Math.Pow(ycoord - ob.ycoord, 2));
        if (d < radius + ob.radius) return true;
        return false;
    }
}
class Program {
    static void Main() {
        Shenber shenber1 = new Shenber(2,2,1);
        Shenber shenber2 = new Shenber(6,6,2);
        Console.WriteLine("birinshi shenber parametrleri: ");
        shenber1.Shygaru();
        Console.WriteLine("ekinshi shenber parametrleri: ");
        shenber2.Shygaru();
        bool b = shenber1.qiylysu(shenber2);
        if(b) Console.WriteLine("berilgen shenberler qiylysady");
        else
            Console.WriteLine("berilgen shenberler qiylyspaidy");
        Console.ReadKey();
    }
}
```

Программа орындалу нәтижесі :

```
birinshi shenber parametrleri:
shenber centrinin koordinatalary: (2, 2), shenber radiusy
= 1
ekinshi shenber parametrleri:
shenber centrinin koordinatalary: (6, 6), shenber radiusy
```

Бұл программадағы **qiylysu()** әдісі аргумент ретінде **Shenber** типіндегі объектіге сілтеме алады. Бұл әдіс шақырған объекті мен **ob** параметрі арқылы берілетін объектінің қиылысатынын тексереді. Егер екі объекті қиылысатын болса, бұл әдіс **true** мәнін қайтарады.

Параметрдің ref модификаторын қолдану

Параметрдің ref модификаторы әдісті мән бойынша емес, мәжбүрлі түрде сілтеме бойынша шақырады. Бұл модификатор әдісті жариялау кезінде де және оны шақырғанда да көрсетіледі.

Келесі программада Sqr() әдісі құрылады, ол оған берілген бүтін аргументтің квадратын қайтарады. Осы мысалдағы ref модификаторының қалай қолданылғанына назар аударыңыздар.

```
// ref модификаторының типті сілтеме бойынша беруі.
using System;
class RefTest {
    // Бұл әдіс ref сөзі арқылы өз аргументін өзгертеді.
    public void Sqr(ref int i) {
        i = i * i;
    }
}

class RefDemo {
    static void Main() {
        RefTest ob = new RefTest(); // ob енді сілтеме бойынша

        int a = 10;

        Console.WriteLine("shaqyruga deingi a: " + a);

        ob.Sqr(ref a);           // ref модификаторы қолданылды
        Console.WriteLine("shaqyrudan keingi a: " + a);
    }
}
```

Сонымен, ref модификаторы әдісті жариялаудағы параметр алдында және әдісті шақырғанда да аргумент алдында көрсетіледі екен. Программа нәтижесінде де аргумент мәні Sqr() әдісі арқылы нақты өзгергені көрініп тұр.



Енді ref модификаторы арқылы аргументтердің мәндерін алмастыратын Swap() әдісін жазып көрсетейік.

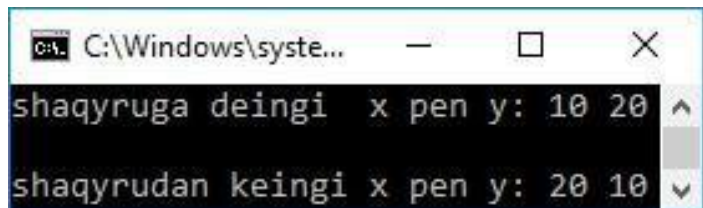
```
// Мәндердің орнын ауыстыру.
using System;
```

```

class ValueSwap {
    // Бұл әдіс өз аргументтерінің мәндерін алмастырады.
    public void Swap(ref int a, ref int b) {
        int t; t = a; a = b; b = t;
    }
}
class ValueSwapDemo {
    static void Main() {
        ValueSwap ob = new ValueSwap();// сілтеме бойынша
        int x = 10, y = 20;
        Console.WriteLine("shaqyruga deingi x pen y: " + x + " " + y);
        ob.Swap(ref x, ref y);
        Console.WriteLine("shaqyrudan keingi x pen y: " + x + " " + y);
    }
}

```

Бұл программа нәтижесі:



ref модификаторы жайлы мынаны білу керек:

- бұл модификатор арқылы сілтеме бойынша берілетін аргументке *әдіс шақырылғанға дейін* мән берілуі тиіс.
- параметр ретінде мұндай аргумент алатын әдісте параметр белгілі бір мәнге сілтеме жасайды деп есептеледі.
- сонымен, ref модификаторын қолданған кезде әдіс ішінде аргументтің бастапқы мәнін беруге болмайды.

Параметрдің out модификаторын қолдану

Кейдесілтемеліпараметр әдіске мән беру үшін емес, одан мән алу үшін қолданылады. Мысалы, белгілі бір функцияны орындап, тек мән қайтаратын әдіс бар делік (желілік сокетті ашып, одан желі операциясының дұрыс немесе қате орындалғаны жайлы сілтемелік параметртүрінде мәлімет алу керек). Мұнда әдіске мәлімет берілмейді, бірақ ол белгілібір жұмысты орындап, мән қайтаруы тиіс. Бұған дейінгі ref типіндегі параметр әдісті шақыру үшін оған мән беруді талап ететін болатын. Сондықтан, осы айтылған шектеулерді болдырмас үшін, C# тілінде – параметрдің out модификаторын пайдалану керек.

Параметрдің out модификаторы ref түйінді сөзіне ұқсас, тек бір айырмасы бар: ол әдістен мән қайтарып береді, бірақ мән қабылдап алмайды. Сондықтан out параметрі ретінде қолданылатын айнымалыға мән беру қажет емес. Оған қоса, мұндағы әдісте out параметрі *инициалданбайтын болып саналады*. Бұл әдістің қайтаратын параметріне ол *жұмысына яқтағанша мән берілуі тиіс*. Сонымен, әдіс шақырылған соң, out параметріне белгілі бір мәні берілуі керек.

Келесі программа мысалында нақты санның бүтіні мен бөлшегін жеке-жеке бөліп алу үшін Decompose класындағы GetParts() әдісі пайдаланылады. Берілген санның әрбір бөлігінің әдістен қалай қайтарылатынына назар аударыңыздар.

```

// Параметрдің out модификаторын қолдану.
using System;
class Decompose {

```

// Нақты санның бүтіні мен бөлшегін ажыратып алу.

```
public int GetParts(double n, out double frac) {
    int whole;

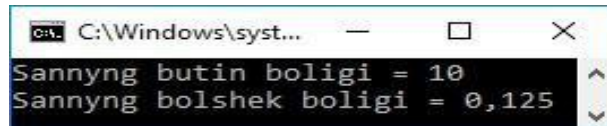
    whole = (int)n;
    frac = n - whole; // frac параметрі санның бөлшегін береді
    return whole; // санның бүтін бөлігін қайтару
}
}
```

```
class UseOut {
    static void Main() {
        Decompose ob = new Decompose();
        int i; double f;

        i = ob.GetParts(10.125, out f);

        Console.WriteLine("Sannyng butin boligi = " + i);
        Console.WriteLine("Sannyng bolshek boligi = " + f);
    }
}
```

Бұл программа нәтижесі.



GetParts() әдісі n санының бүтін бөлігін return операторы арқылы қайтарады. Ал санның бөлшегін out типіндегі frac параметрі арқылы бөліп алады. Мысал көрсеткендей, out модификаторын қолдана отырып, әдістен екі мән қайтарып алдық. Осы тәсілмен әдістерден екі, үш, төрт, т.с.с. мәндер қайтара аламыз, оған шек қойылмайды.

Енді out арқылы екі параметр қолданатын мысал келтірейік. Мұндағы HasComFactor() әдісі екі түрлі функцияны орындайды.

- 1) Ол екі бүтін санның ортақ көбейткішін анықтап (ондайлар бар болса), логикалық true мәнін қайтарады, әйтпесе – false мәнін береді.
- 2) out типіндегі параметрлер арқылы екі санның ең кіші және ең үлкен ортақ бөлгіштерін (ондай сандар бар болса) анықтайды.

// out типіндегі екі параметрді пайдалану.

```
using System;
```

```
class Num {
    /* x және y айнымалыларының ортақ көбейткішін анықтау.
    * Егер олар бар болса, солардың ішіндегі ең үлкен
    * және ең кіші көбейткіштерін out типіндегі параметр
    * арқылы анықтап қайтару. */
    public bool HasComFactor(int x, int y, out int least, out int greatest) {
        int max = x < y ? x : y; int i;
        bool first = true; least = 1; greatest = 1;
        // Ең кіші және ең үлкен көбейткіштерді табу.
        for(i=2; i <= max/2 + 1; i++) {
            if( ((y%i)==0) & ((x%i)==0) ) {
                if (first) { least = i; first = false; }
                greatest = i;
            }
        }
    }
}
```

```

if(least != 1) return true;

else return false;
}
}

class DemoOut {
static void Main() {
Num ob = new Num();
int lcf, gcf;

if(ob.HasComFactor(231, 105, out lcf, out gcf)) {
Console.WriteLine("231 jane 105 sandarynyng " +
"eng kishi orta q kobeitkishi = " + lcf) ;
Console.WriteLine("231 jane 105 sandarynyng " +
"eng ulken orta q kobeitkishi = " + gcf);
}
else
Console.WriteLine("35 jane 49 sandarynyng orta q kobeitkishi joq.");
if(ob.HasComFactor(35, 51, out lcf, out gcf)) {
Console.WriteLine("35 jane 51 sandarynyng eng kishi " +
"orta q kobeitkishi = " + lcf);
Console.WriteLine("35 jane 51 sandarynyng eng ulken " +
"orta q kobeitkishi = " + gcf); }
else Console.WriteLine("35 jane 51 sandarynyng orta q kobeitkishi joq.");
}
}

```

Бұл программадағы Main() әдісінде lcf және gcf айнымалыларына мәндер HasComFactor() әдісін шақырғанға дейін меншіктеледі. Егер HasComFactor() әдісінің параметрлері out болмай, ref болғанда, бұл қате болар еді. Мұндағы әдіс екі бүтін санның ортақ көбейткішінің болуы/болмауына байланысты true не false мәндерінің бірін қайтарады.

Егер ортақ көбейткіш табылса, out типіндегі параметр арқылы осы сандардаң ең кіші және ең үлкен ортақ көбейткіштері қайтарылады.

Бұл программаның орындалу нәтижесі:

```

C:\Windows\system32\cmd.exe
231 jane 105 sandarynyng eng kishi orta q kobeitkishi = 3
231 jane 105 sandarynyng eng ulken orta q kobeitkishi = 21
35 jane 51 sandarynyng orta q kobeitkishi joq.

```

Аргументтердің айнымалы санын пайдалану

Әдіс құрған кезде оған берілетін параметрлер саны алдын ала белгілі болады, бірақ кейде олай болмай қалады. Әдіске берілетін аргументтер саны айнымалы болып келетін жағдайлар туындағанда, арнайы типтегі параметр қолданылады. Мысалы, бірсыпыра мәндер ішінен кішісін табатын әдіс құрылған болса, ол берілген мәндер санына қарамай, бәрі бір солардың ең кішісін анықтауы тиіс.

Осындай кездерде қолданылатын арнайы тип params модификаторы арқылы жасалады. Мұнда аргумент ретінде жиым қолданылады да, әдіске берілетін параметрлер саны жиым элементтерінің санымен анықталады. Ал программадағы аргументтерді анықтап алу үшін жиымға қатынас құру, яғни оны пайдалану ісі ұйымдастырылады.

Төмендегі мысалда params модификаторы MinVal() әдісін құруда қолданылып, ол

берілген мәндер ішінен ең кішісін анықтайды.

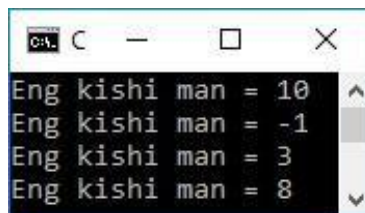
// params модификаторын пайдалану мысалы.

using System;

```
class Min {  
    public int MinVal(params int[] nums) {  
        int m;  
  
        if(nums.Length==0) {  
            Console.WriteLine("Kate: argumentter joq.");  
            return 0;  
        }  
  
        m = nums[0];  
        for(int i=1; i < nums.Length; i++)  
            if(nums[i] < m) m = nums[i];  
        return m;  
    }  
}
```

```
class ParamsDemo {  
    static void Main() {  
        Min ob = new Min();  
        int min;  
        int a = 10, b = 20;  
  
        // Екі мәні бар әдісті шақыру.  
        min = ob.MinVal(a, b);  
        Console.WriteLine("Eng kishi man = " + min);  
  
        // Үш мәні бар әдісті шақыру.  
        min = ob.MinVal(a, b, -1);  
        Console.WriteLine("Eng kishi man = " + min);  
  
        // Бес мәні бар әдісті шақыру.  
        min = ob.MinVal(18, 23, 3, 14, 25);  
        Console.WriteLine("Eng kishi man = " + min);  
  
        // Бүтін сандық жиымнан тұратын әдісті шақыру.  
        int[] args = { 45, 67, 34, 9, 112, 8 };  
        min = ob.MinVal(args);  
        Console.WriteLine("Eng kishi man = " + min);  
    }  
}
```

Бұл программа нәтижесі:



Бұл программада MinVal() әдісі шақырылғанда, оған nums жиымынан аргументтер беріледі. Жиым ұзындығы берілетін аргументтер санына тең. Сондықтан MinVal() әдісі арқылы кез келген көлемдегі мәндер арасынан ең кішісін таба аламыз.

MinVal() әдісінің ең соңғы шақырылуына назар салыңыздар. Бұл жолы жеке мәндер орнына көптеген элементтері бар жиым беріледі. Аргументтерді осындай түрде беру дұрыс болып саналады. Params типіндегі параметр құрылғаннан кейін, ол ұзындығы айнымалы аргументтер тізімін немесе жиым элементтерін қабылдайды.

Params типіндегі параметрге кез келген аргументтер санын беруге болады, олардың барлығы да осы параметр көрсетіп тұрған жиым типінде болуы тиіс.

Мысалы, MinVal() әдісін мынадай түрде шақыру қате:

```
min = ob.MinVal(1, 2.2); // Дұрыс емес!
```

өйткені double типін (2.2) автоматты түрде MinVal() әдісінде көрсетілген nums жиымының int типіне айналдыра алмаймыз.

Params модификаторын байқап пайдалану керек, өйткені ол кез келген аргументтер санын, тіпті нөлді де қабылдай береді. Мысалы, төмендегідей түрде MinVal() әдісін шақыру синтаксис бойынша дұрыс болғанымен айтылған ережеге қайшы келіп тұр.

```
min = ob.MinVal(); // аргументтер жоқ
```

```
min = ob.MinVal(3); // 1 аргумент
```

Осындай себептерге байланысты MinVal() әдісінде оны пайдалану алдында nums жиымының кем дегенде, бір элементінің болуы тексеріледі. Егер мұндай тексеру болмаса, MinVal() әдісін аргументтерсіз шақырғанда, аластама (ерекше жағдай) туындайтын еді.

MinVal() әдісінде бір ғана элемент те болуы мүмкін. Әдісте қарапайым параметр де, айнымалы параметр де бола береді. Келесі мысалда ShowArgs() әдісі String типіндегі бір параметр қабылдайды және де params типіндегі параметр ретінде бүтін сан түріндегі ды қабылдайды.

```
// params типіндегі ұзындығы айнымалы параметрмен бірге  
// қарапайым параметрді де қолдану.
```

```
using System;
```

```
class MyClass {
```

```
public void ShowArgs(String msg, params int[] nums) {  
    Console.Write(msg + ": ");
```

```
    foreach (int i in nums)  
        Console.Write(i + " ");  
    Console.WriteLine();  
}
```

```
class ParamsDemo2 {
```

```
    static void Main() {
```

```
        MyClass ob = new MyClass(); // конструктор
```

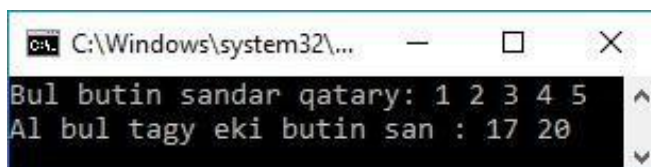
```
        ob.ShowArgs("Bul butin sandar qatary", 1, 2, 3, 4, 5);
```

```
        ob.ShowArgs("Al bul tagy eki butin san ", 17, 20);
```

```
    }
```

```
}
```

Бұл программаның нәтижесі төмендегідей түрде болады.



Әдісте қарапайым параметрлермен бірге params типіндегі ұзындығыайнымалы параметр бар болса, ол параметрлер тізімінде ең соңғыболып көрсетілуі керек. Бірақ кез келген жағдайда params типіндегі параметр жалғыз болуы тиіс.

Видеожазбалар:

ref және out модификаторлары <https://youtu.be/amCv4hAGf84?si=tyPXI2xnYiTmi12O>

4 дәріс. Әдістерден объектілерді қайтару.

Дәрістің мақсаты: студенттерде әдістердің құрамында объектілермен жұмыс істеу ерекшеліктері бойынша түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Әдістен объектіні қайтару форматын білетінін көрсету;
- Әдістен ағымдағы объектіні қайтару форматынбілетінін көрсету.

Әдіс кез келген типтегі мәліметті қайтара алады, ол класс типін де қайтара алады. Төменде құрамында Enlarge() әдісі бар Rect класының бір нұсқасы берілген, онда тіктөртбұрыш тұрғызылады, оның қабырғалары әдісті шақыратын объектідегі тіктөртбұрышпен бірдей, бірақ тұрғызылатын тіктөртбұрыш көрсетілген коэффициентке көбейтіліп, үлкейтіледі.

```
// Әдістен объектіні қайтару.
```

```
using System;
```

```
class Rect {  
    int width;  
    int height;
```

```
    public Rect(int w, int h) {  
        width = w;  
        height = h;  
    }
```

```
    public int Area() {  
        return width * height;  
    }
```

```
    public void Show() {  
        Console.WriteLine(width + " " + height);  
    }
```

```
    /* Әдіс шақырылған объектідегі төртбұрышпен салыстырып  
    карағанда, қабырғалары берілген коэффициентке  
    пропорционал үлкейтілген төртбұрышты қайтарады. */  
    public Rect Enlarge(int factor) {  
        return new Rect(width * factor, height * factor);  
    }  
}
```

```
class RetObj {  
    static void Main() {  
        Rect r1 = new Rect(4, 5);
```

```
        Console.WriteLine("r1 tortburysh qabyrgalary: ");  
        r1.Show();  
        Console.WriteLine("r1 tortburysh audany: " + r1.Area());
```

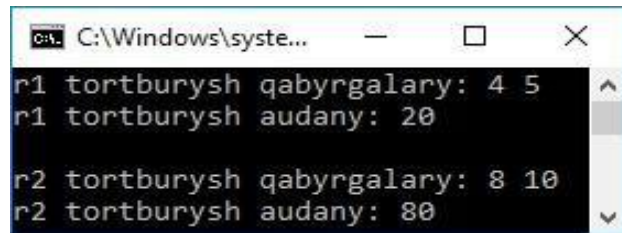
```
Console.WriteLine();
```

```
// r1 tortburyshynan eki ese ulken tortburysh turgyzu.  
Rect r2 = r1.Enlarge(2);  
Console.Write("r2 tortburysh qabyrgalary : ");  
r2.Show();  
Console.WriteLine("r2 tortburysh audany : " + r2.Area());  
}  
}
```

Бұл программа нәтижесі:

```
r1 tortburysh qabyrgalary: 4 5  
r1 tortburysh audany : 20
```

```
r2 tortburysh qabyrgalary: 8 10  
r2 tortburysh audany : 80
```



Әдіс объектіні қайтарғанда, объект оған сілтеме болғанша жұмыс істей береді. Сонан кейін объект "қоқыс" ретінде жойылады. Сол себепті объектіні құрған әдіс аяқталғанмен, объект жойылмайды. Объект типіндегі қайтарылатын мәліметтердің практикалық мысалы ретінде *класс фабрикасы* қолданылады. Ол өз класындағы объектілер тұрғызуға арналған әдіс болып табылады.

Кейбір кездерде класс қолданушыларына сол класс конструкторларын пайдалануға мүмкіндік беру қауіпсіздік талаптарына қарай қажет болмайды, ол объектіні тұрғызудың кейбір сыртқы факторларға тәуелді болатындығына байланысты да болып жатады. Мұндай жағдайларда объект құру үшін класс фабрикасы қолданылады. Мысал қарастырайық.

// Класс фабрикасын пайдалану.

```
using System;
```

```
class MyClass {  
    int a, b;           // кластың жабық мүшелері
```

```
    // MyClass класы үшін фабрика құру.
```

```
    public MyClass Factory(int i, int j) {  
        MyClass t = new MyClass();
```

```
        t.a = i;
```

```
        t.b = j;
```

```
        return t;       // объектіні қайтару  
    }
```

```
    public void Show() {  
        Console.WriteLine("a және b: " + a + " " + b);  
    }  
}
```

```
class MakeObjects {  
    static void Main() {
```

```

MyClass ob = new MyClass();

int i, j;
// Класс фабрикасы арқылы объектілер қалыптастыру.
for(i=0, j=10; i < 10; i++, j --) {
    MyClass anotherOb = ob.Factory(i, j); // объект құру
    anotherOb.Show();
}
Console.WriteLine();
}
}

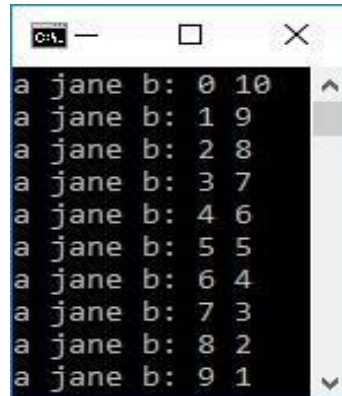
```

Программа нәтижесі:

```

a jane b: 0 10
a jane b: 1 9
a jane b: 2 8
a jane b: 3 7
a jane b: 4 6
a jane b: 5 5
a jane b: 6 4
a jane b: 7 3
a jane b: 8 2
a jane b: 9 1

```



Мұнда MyClass класы үшін конструктор анықталмайды, сондықтан тек келісім бойынша құрылатын конструктор қолданылады. Бұл a және b айнымалыларының мәнін конструктор арқылы беруге болмайтынын білдіреді. Бірақ Factory() класы фабрикасында a және b айнымалыларының мәндері берілетін объектілер құруға болады. Оған қоса, a және b айнымалылары жабық болғандықтан, олардың мәндерін тек қана Factory() класы фабрикасының көмегімен беруге болады.

Main() әдісінде MyClass класы объектісінің экземпляры алынады, ал оның фабрикалық әдісі for циклінде басқа 10 объектіні құру үшін қолданылады.

Төмендегі жолда сол объектілер құрылатын код көрсетілген.

```
MyClass anotherOb = ob.Factory(i, j); // объект құру
```

циклдің әрбір қадамында anotherOb объектісіне сілтеме жасайтын айнымалы жасалады, оған класс фабрикасында қалыптасатын объектіге сілтеме меншіктеледі. Циклдің әрбір қадамы аяқталған сайын anotherOb айнымалысы өз әсер ету аймағынан шығып кетеді де, ол сілтеме жасап тұрған объект жойылады.

Әдістен жиым қайтару

C# тілінде жиымдар объектілер түрінде жасалған, бұл әдістердің жиымды қайтара алатындығын білдіреді. Басқа тілдердің көбісінде әдістерден жиымдар қайтарылмайды. Төменде FindFactors() әдісі оған берілген аргументтің көбейткіштерін жиым түрінде қайтаратын мысал берілген.

// Әдістен жиым қайтару.

```
using System;
```

```

class Factor {
    /* Әдіс num аргументінің көбейткіштерінен тұратын facts жиымын қайтарады. Әдістен
    out типіндегі numfactors параметрі - анықталған көбейткіштер санын қайтарады. */
    public int[] FindFactors(int num, out int numfactors) {

```

```

int[] facts = new int[80];      //жиым ені алдын ала
                                // артығымен 80 болып алынған
int i, j;

// Көбейткіштерді тауып, facts жиымына орналастыру.
for(i=2, j=0; i < num/2 + 1; i++)
    if( (num%i)==0 ) {
        facts[j] = i;
        j++;
    }

numfactors = j;
return facts;
}
}

```

```

class FindFactors {
    static void Main() {
        Factor f = new Factor();
        int numfactors;
        int[] factors;

        factors = f.FindFactors(1000, out numfactors);

        Console.WriteLine("1000 sanynyngkobeitkishteri: ");
        for(int i=0; i < numfactors; i++)
            Console.Write(factors[i] + " ");
        Console.WriteLine();
    }
}

```

Жоғарыдағы Factor класындағы FindFactors() әдісі былай жарияланған:

```

public int[] FindFactors(int num, out int numfactors) {

```

Мұндағы int типіндегі қайтарылатын жиым қалай көрсетілгеніне назар салыңыздар. Бұл синтаксисті жалпылама етуде болады. Әдіс жиым қайтарған сайын, ол осылай көрсетіледі, бірақ оның типі мен өлшемі есепке алынуы тиіс. Мысалы, келесі жолда double типіндегі екі өлшемді жиым қайтаратын someMeth() әдісі жарияланып отыр:

```

public double[,] someMeth() { // ...

```

5 дәріс. Әдістерді асыра жүктеу. Статикалық кластар және кластың статикалық мүшелері.

Дәрістің мақсаты: студенттерде әдістерді асыра жүктеудің қызметі және кластың статикалық мүшелерін пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Әдістерді асыра жүктеудің қызметі туралы түсініктерін көрсету;
- Әдістерді асыра жүктеу шарттарын көрсету;
- Кластың статикалық мүшелерін құру және пайдалану ерекшеліктері туралы түсініктерін көрсету.

Бірклассішіндегі бірнеше әдістердің аттары бірдей болып, теколардың параметрлері әртүрлі болып жариялануы әдістердің асыра жүктелуі деп аталады. C# тіліндегі әдістердің асыра жүктелуі ОБП-дағы полиморфизмді жүзеге асырудың бір жолы болып табылады.

Жалпы, әдістерді асыра жүктеу үшін олардың әртүрлі нұсқаларын жариялау жеткілікті, ал қалған жағын компилятор орындайды. Мұнда мынадай маңызды шартты орындау керек: асыра жүктелетін әрбір әдістің параметрлерінің типтері немесе олардың сандары әртүрлі болуы керек. Екі әдістің тек қайтаратын мәндерінің типтерінің әртүрлі болуы жеткіліксіз. Олар өз параметрлерінің типтерімен немесе сандарымен өзгеше болуы керек.

Әрине, асыра жүктелген әдістердің қайтарылатын типтері де әртүрлі бола береді, бірақ олар қай әдісті таңдап алып, орындау керектігі жайлы толық мәлімет бере алмайды. Асыра жүктелген әдіс шақырылғанда, оның параметрлерінің әдіске берілетін аргументтерге сәйкес келген (типтері және сандары арқылы) нұсқасы орындалады.

Төменде әдістерді асыра жүктеуді көрсететін қарапайым мысал берілген.

Мысал 1. Әдістерді асыра жүктеуді көрсету.

```
using System;
class Tortburysh {
    public double a;
    public double b;
    public Tortburysh(double a, double b)
    { this.a=a; this.b = b; }
    public void Shygaru() {
        Console.WriteLine("tortburysh qabyrgalary: "+a+", "+b);
    }
    // Бір бүтін параметрлі eseleu әдісін асыра жүктеу.
    public Tortburysh eseleu(int k) {
        return new Tortburysh(a*k, b*k);
    }
    // Бір нақты параметрлі eseleu әдісін асыра жүктеу.
    public Tortburysh eseleu(double k) {
        return new Tortburysh(a*k, b*k);
    }
    // Екі бүтін параметрлі eseleu әдісін асыра жүктеу.
    public Tortburysh eseleu(int k, int i) {
        return new Tortburysh(a*k, b*i);
    }
}
class Program {
```

```

static void Main() {
    Tortburysh ob1 = new Tortburysh (4,5);
    Console.WriteLine("bastapqy tortburysh: ");
    ob1.Shygaru();
    Console.WriteLine("eseleu adisin butin parametrmen
        s h a q y r u " ) ;
    Tortburysh ob2 = ob1.eseleu(2);
    ob2.Shygaru();
    Console.WriteLine("eseleu adisin naqty parametrmen
        s h a q y r u " ) ;
    Tortburysh ob3 = ob1.eseleu(2.3);
    ob3.Shygaru();
    Console.WriteLine("eseleu adisin eki butin parametrmen
        s h a q y r u " ) ;
    Tortburysh ob4 = ob1.eseleu(2,3);
    ob4.Shygaru();
}
}

```

Жоғарыдағы программа жұмысының нәтижесі:

```

bastapqytortburysh:
tortburysh qabyrgalary: 4, 5
eseleu adisinbutin parametrmen shaqyru
tortburysh qabyrgalary: 8, 10
eseleu adisinnaqty parametrmen shaqyru
tortburysh qabyrgalary: 9.2, 11.5
eseleu adisineki butin parametrmen shaqyru
tortburysh qabyrgalary: 8, 15

```

Мұнда **eseleu()** әдісі 3 рет асыра жүктеледі.

- 1) Бір бүтін типті параметрмен – 2;
- 2) Бір нақты типті параметрмен – 2.3;
- 3) Екі бүтін типті параметрмен – 2 және 3.

Әдісті асыра жүктеуде қайтарылатын мәннің типі ешқандай рөл атқармайды. Бірақ бір әдіске екі түрлі қайтару типін қолдану қате береді. Төменде **eseleu()** әдісіне екі түрлі (қайтарылатын мәннің типі бойынша) мән қайтару тәсілін қолдану мысалы көрсетілген.

// **eseleu(int)** әдісін бір рет жариялау жеткілікті

```

public Tortburysh eseleu(intk) {
    return new Tortburysh(a*k,b*k);
}

```

/* Қате! Әртүрлі мән қайтаратын болғанмен, **eseleu(int)** әдісін екінші рет жариялауға болмайды. */

```

public void eseleu(int k) {
    Console.WriteLine("tortburysh parametrleri : "
        + a * k + ", " + b * k);
}

```

Комментарийден көрінгендей, **eseleu()** әдісінің екі нұсқасы екі түрлі мән қайтаратын болғанмен, ол асыра жүктеуге жеткіліксіз болып табылады.

Әдістерді асыра жүктеу программалау тілінің полиморфизм қасиетін сүйемелдейді, өйткені полиморфизмнің басты қағидасы: бір интерфейс – бірнеше әдіс мұнда жүзеге асырылады. Бұл түсініктілеу болуы үшін, тағы бір нақты мысал қарастырайық. Егер программалауда асыра жүктеу болмаса, әрбір әдістің өзіндік аты болуы тиіс. Бірақ программалауда мәліметтердің әртүрлі типтері үшін бір әдіс қажет болып жатады. Бізге санның абсолюттік мәнін анықтайтын функция қажет болсын делік. Асыра жүктеу сүйемелденбейтін тілде мұндай функцияның әртүрлі аты бар бірнеше түрін құру керек болар еді.

Мысалы, C тілінде **abs()** функциясы – бүтін сандардың абсолюттік мәні үшін, **labs()** функциясы – ұзын бүтін санның абсолюттік мәні үшін, **fabs()** – нақты санның абсолюттік мәні үшін болып кете береді. C# тілінде әдістер асыра жүктеледі, өйткені **System.Math** класында әртүрлі типтегі мәліметтерді өңдейтін бір ғана **Abs()** әдісі бар.

C# тілінде әдістің аты мен оның параметрлері тізімін белгілейтін *сигнатура* түсінігі анықталған. Асыра жүктеуге байланысты бұл түсінік бір класта сигнатуралары бірдей екі әдіс болмауы тиіс дегенді білдіреді. Сигнатура түсінігіне әдістен қайтарылатын мән типі кірмейді, өйткені компилятор асыра жүктеу жайлы шешім қабылдағанда, қайтарылатын мән типі есепке алынбайды. Сигнатураға **params** модификаторы да енгізілмеген.

Конструкторларды асыра жүктеу

Әдістер секілді конструкторлар да асыра жүктеледі. Ол объектілерді әртүрлі жолдармен құру мүмкіндігін береді. Мысал қарастырайық.

Мысал 2. Конструкторды асыра жүктеу.

```
using System;
class Tortburysh {
    public double a;
    public double b;
    public Tortburysh() {
        Console.WriteLine("Tortburysh() konstruktorynda");
        a = b = 1;
    }
    public Tortburysh(double a) {
        Console.WriteLine("Tortburysh(double) konstruktorynda");
        this.a = a; this.b = a;
    }
    public Tortburysh(double a, double b) {
        Console.WriteLine("Tortburysh(double, double)
        konstruktorynda");
        this.a = a; this.b = b;
    }
    // Көшіру конструкторы
    public Tortburysh(Tortburysh T) {
        Console.WriteLine("Tortburysh(Tortburysh)
        konstruktorynda");
        this.a = T.a; this.b = T.b;
    }

    public void Shygaru() {
        Console.WriteLine("tortburysh qabyrgalary: " + a + ", " + b);
    }
}
class Program {
```

```

static void Main() {
    Tortburysh ob1 = new Tortburysh();
    ob1.Shygaru();
    Tortburysh ob2 = new Tortburysh(2.3);
    ob2.Shygaru();
    Tortburysh ob3 = new Tortburysh(2.3, 4.5);
    ob3.Shygaru();
    Tortburysh ob4 = new Tortburysh(ob3);
    ob4.Shygaru();
}
}

```

Бұл программаның нәтижесі:

```

Tortburysh()konstruktorynda
tortburysh qabyrgalary: 1, 1
Tortburysh(double)konstruktorynda
tortburysh qabyrgalary: 2.3, 2.3
Tortburysh(double,double) konstruktorynda
tortburysh qabyrgalary: 2.3, 4.5
Tortburysh(Tortburysh) konstruktorynda
tortburysh qabyrgalary: 2.3, 4.5

```

Бұл мысалда **Tortburysh()** конструкторы 4 рет асыра жүктеледі, әр орындалған сайын ол объектіні басқаша құрады. Қажетті конструктор new операторында көрсетілген аргументтерге байланысты шақырылады. Аргументретінде класс объектісін қабылдайтын конструктор көшіру конструкторы деп аталады. Осылай кластың конструкторын асыра жүктеу қосымша мүмкіндіктер береді екен.

Статикалық кластар және кластың статикалық мүшелері.

Кейде бір кластың басқа объектілерінен тәуелсіз түрде қолданылатын сол класс мүшесін анықтау керек болып жатады. Көбінесе класс мүшесіне қол жеткізу осы класс объектісі арқылы ұйымдастырылады, бірақ сонымен қатар объектінің нақты экземплярына сілтеме жасамай-ақ, жеке қолданыла беретін класс мүшесін құруға болады. Осындай мүше жасау үшін оны жариялаудың ең басында **static** түйінді сөзін көрсету жеткілікті. Егер класс мүшесі **static** болып жарияланса, онда басқа ешбір объектіге сілтеме жасамай-ақ, ол өз класының кез келген объектісін жасау үшін қолданыла алады. **Static** түйінді сөзі арқылы айнымалыларды да және сонымен қатар әдістерді де жариялауға болады. **static** типіндегі мүшенің қалыпты мысалы ретінде операциялық жүйе арқылы шақырылып, программаның ең басында жарияланатын **Main()** әдісін есептеуге болады. **static** типіндегі мүшені кластан тыс қолдану үшін, осы класс атын нүкте-оператормен көрсету жеткілікті. Бірақ бұл үшін объект құру қажет емес. Негізінде, **static** типіндегі мүшеге қол жеткізу объектіге сілтеме жасау жолымен емес, өз класының аты бойынша жүзеге асады. Сонымен, егер **Timer** класының мүшесі болып табылатын **static** типіндегі **Count** айнымалысына 10 мәнін беру керек болса, онда келесі код жолын қолдануға болады.

```
Timer.Count = 10;
```

Бұл жазу формасы объект арқылы қарапайым экземпляр айнымалысын пайдалану кезіндегі сияқты болып жазылады, бірақ онда объектінің аты емес, класс аты көрсетіледі. Осындай тәсіл арқылы класс аты мен нүкте-операторды пайдаланып, **static** типіндегі әдісті де шақыруға болады.

static типінде жарияланған айнымалылар, негізінде, глобальді айнымалылар болып табылады. Объектілер өз класында жарияланған кезде, **static** типіндегі айнымалылардың көшірмесі жасалмайды. Оның орнына кластың барлық экземпляры **static** типіндегі бір

айнымалымен бірге қолданылады. Мұндай айнымалы оны класта пайдалану алдында инициалданады.

Айнымалының инициализаторы тікелей көрсетілмеген жағдайда, ол сандық типте болса, нөл мәнімен, сілтемелік типте болса, бос мәнмен инициалданады, ал егер ол bool типінде болса, оған false мәні беріледі.

Сонымен, static типіндегі айнымалы әрқашанда белгілі бір мәнге ие болуы тиіс. static типіндегі әдістің қарапайым әдістен айырмашылығы оны бұл кластың объектісі экземплярын жасамай-ақ, өз класының атымен шақыруға болады. Мұндай шақыру мысалы бұрын келтірілген болатын. Ол C# кластары кітапханасындағы System.Math класына жататын static типіндегі Sqrt() әдісі болатын.

Төменде static типіндегі айнымалы және әдіс жарияланған программа мысалы көрсетілген.

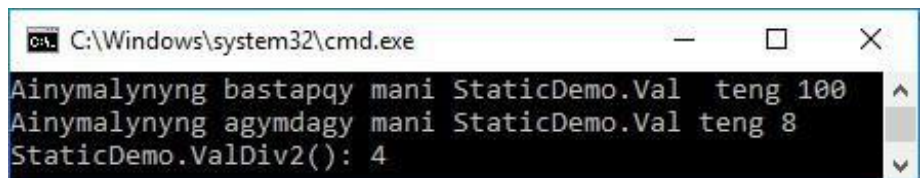
// static модификаторын пайдалану.

using System;

```
class StaticDemo {  
    // static типіндегі айнымалы.  
    public static int Val = 100;  
    // static типіндегі әдіс.  
    public static int ValDiv2() { return Val/2; }  
}
```

```
class SDemo {  
static void Main() {  
    Console.WriteLine("Ainymalynyngbastapqy mani " +  
        "StaticDemo.Val teng " + StaticDemo.Val);  
  
    StaticDemo.Val = 8;  
    Console.WriteLine("Ainymalynyngagymdagy mani " +  
        "StaticDemo.Val teng " + StaticDemo.Val);  
    Console.WriteLine("StaticDemo.ValDiv2(): " + StaticDemo.ValDiv2());  
}  
}
```

Бұл программа
нәтижесі:



Мұнан көретініміз static типіндегі айнымалы оның класын-дағы кез келген объект құрылғанша, инициалданады.

static типіндегі әдістерді қолдануға бірсыпыра шектеулер қойылады:

- static типіндегі әдістерде this сілтемесі болмауы тиіс, өйткені мұндай әдіс белілі бір объектіге қатысты орындалмайды.
- static типіндегі әдісте осы кластың экземпляры әдісін емес, тек басқа static типіндегі әдістерді тікелей шақыруға рұқсат етілген. Мұның себебі – экземпляр әдістері нақты объектілермен жұмыс істейді, ал static типіндегі әдіс объект үшін шақырылмайды. Сондықтан, мұндай әдістің жұмыс істейтін объектілері болмайды.
- Дәл осындай шектеулер static типіндегі мәліметтерге де қойылады. static типіндегі әдіс үшін оның класында анықталған басқа static типіндегі мәліметтер тікелей қол жетімді болып табылады.

Ол, мысалы, өз класының экземпляры айнымалысымен жұмыс істей алмайды, өйткені онда оның өңдей алатын объектілері болмайды. Төмендегі мысалда static типіндегі ValDivDe-nom() әдісі жұмыс істей алмайды.

```
class StaticError {
```

```

public int Denom = 3;           // экземплярдың қалыпты айнымалысы
public static int Val = 1024;    // статикалық айнымалы

/* Қате! Статикалық әдісте статикалық емес айнымалыны тікелей қолданбайды. */
static int ValDivDenom() {
    return Val/Denom;           // компиляциядан өтпейді!
}
}

```

Бұл мысалда Denom жай қалыптағы айнымалы болып табылады, оны static типіндегі әдісте қолдана алмаймыз. Бірақ мұнда Val айнымалысын пайдалана аламыз, өйткені ол static болып жарияланған.

Осындай қателік төмендегі мысалда да бір класс ішіндегі статикалық әдістен статикалық емес әдісті шақыру кезінде де туындайды.

```

using System;
class AnotherStaticError {

    // Статикалық емес әдіс.
    void NonStaticMeth() {
        Console.WriteLine("NonStaticMeth() adisinde.");
    }
}

```

```

/* Қате! Статикалық әдістен статикалық емес әдісті тікелей шақыруға болмайды. */
static void staticMeth() {
    NonStaticMeth(); // компиляциядан өтпейді!
}
}

```

Мұнда компиляция кезінде статикалық әдістен статикалық емес әдісті (экземпляр әдісін) шақыру тәсілі қате береді.

static типіндегі әдістен экземпляр әдістерін шақырып, оның класындағы экземпляр айнымалыларын сол класс объектілеріндегі тәрізді қолдануға болмайды.

Мұның себебі – нақты объект көрсетілмесе, экземпляр айнымалысын немесе әдісін қолдануға болмайды. Мысалы, төмендегі код фрагменті дұрыс жазылған болып саналады.

```

class MyClass {

    // Статикалық емес әдіс.
    void NonStaticMeth(){
        Console.WriteLine("NonStaticMeth() adisinde. ");
    }

    /* Объектіге сілтеме жасау арқылы статикалық әдістен статикалық емес әдіс
       шақырыла алады. */
    public static void staticMeth(MyClass ob) {
        ob.NonStaticMeth(); // бәрі дұрыс!
    }
}

```

Бұл мысалда NonStaticMeth() әдісі staticMeth() әдісінен MyClass типіндегі ob объектісіне сілтеме бойынша шақырылады.

static типіндегі өрістер нақты объектіге тәуелді болмайды, сондықтан олар бүкіл осы класта қолданылатын ақпарат сақтауға ыңғайлы.

Төменде осындай жағдайды көрсететін программа мысалы келтірілген. Бұл программада

static типіндегі өріс бұрыннан бар объектілердің санын сақтау үшін қолданылады.

// static типіндегі өрісті бұрыннан бар объектілер

// экземплярыларын санау үшін пайдалану.

using System;

public CountInst() { count++; }

class CountInst {

static int Count = 0;

// Объект құру кезіндегі санауышты инкременттеу.

public CountInst() { count++; }

// Объектіні жою кезінде санауышты декременттеу.

~CountInst() { count--; }

public static int GetCount() { return count; }

}

class CountDemo {

static void Main() {

CountInst ob;

for(int i=0; i < 10; i++) {

ob = new CountInst();

Console.WriteLine("Agymdagь esepreu: " +

C o u n t I n s t . G e t C o u n t ()) ;

}

}

}

Бұл программа нәтижесі:

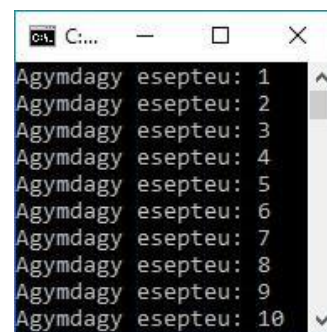
Agymdagь esepreu: 1

Agymdagь esepreu: 2

Agymdagь esepreu: 3

...

Agymdagь esepreu: 10



CountInst типіндегі объект құрылған сайын static типіндегі Count өрісі инкременттеледі, бірақ осындай объектіні жою кезінде Count өрісі декременттеледі. Сол себепті Count өрісі осы сәттегі объектілер санын сақтап тұрады. Ал бұл static типіндегі өрісті қолдану арқылы жүзеге асатын болады. Осындай санау тәсілінэкземпляр айнымалысы арқылы ұйымдастыруға болмайды, өйткені ол бұл кластағыобъектінің нақты экземпляры емес, бүкіл класты камтиды.

Төменде кластың статикалық мүшелерін қолданудың тағы бір мысалы келтірілген. Бұдан бұрынырақ кластар фабрикасы арқылы объектілерді құруға болатыны айтылған болатын. Ол мысалда фабрика статикалық емес әдіс болып жасалған еді, сондықтан ол фабрикалық әдіс тек алдын ала құрылған объектіге сілтеме жасау арқылы шақырылатын болаын.

Бірақ класс фабрикасын static типіндегі әдіс ретінде жүзеге асыру дұрысырақ болады, өйткені ол осы фабрикалықәдісті керекті объект құрмай-ақ шақыру мүмкіндігінберетін еді. Міне осы тәсіл келесі класс фабрикасын жүзеге асыратын мысалда көрсетілген.

// Статикалық класс фабрикасын қолдану.

using System;

class MyClass {

int a, b; // MyClass класы үшін фабрика құру.

static public MyClass Factory(int i, int j) {

```

MyClass t = new MyClass();

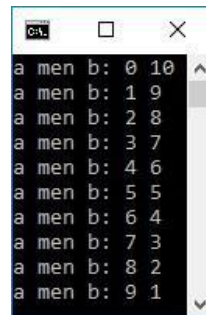
t.a = i;
t.b = j;

return t; // объектіні қайтару
}

public void Show() { Console.WriteLine("a men b: " + a + " " + b); }
}

class MakeObjects {
    static void Main() {
        int i, j;
        // Фабриканы қолданып, объектілерді қалыптастыру.
        for(i=0, j = 10; i < 10; i++, j--) {
            MyClass ob = MyClass.Factory(i, j); // объект жасау
            ob.Show();
        }
        Console.WriteLine();
    }
}

```



Программаның нәтижесі:

Программаның бұл нұсқасында фабрикалық Factory() әдісі оның класының аты арқылы былай шақырылады:

```
MyClass ob = MyClass.Factory(i, j); // объект жасау
```

Енді бұл кластың фабрикасын қолдану алдында MyClass класы объектісін құру қажет емес.

Статикалық конструкторлар

Конструкторды да static түрінде жариялауға болады. Көбінесе статикалық конструктор кластағы объектінің жеке экземпляры үшін емес, бүкіл класс үшін қолданылатын компоненттерді инициалдау үшін пайдаланылады.

Сондықтан класс мүшелері осы кластың кез келген объектілерін құрудан бұрын статикалық конструктор арқылы инициалданады.

Төменде статикалық конструкторды пайдаланудың қарапайым мысалы келтірілген.

// Статикалық конструкторды қолдану.

```

using System;
class Cons {
    public static int alpha;
    public int beta;

    // Статикалық конструктор.
    static Cons() { alpha = 99;
        Console.WriteLine("Statikalyq konStruktorda.");
    }
}

```

// экземпляр конструкторы.

```

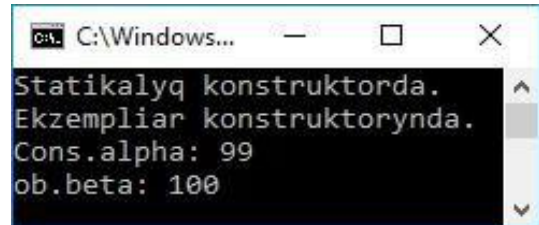
public Cons() {

```

```
beta = 100; Console.WriteLine("Ekzemplier konStruktorynda.");
```

```
class ConsDemo {  
    static void Main() {  
        Cons ob = new Cons();  
        Console.WriteLine("Cons.alpha: " + Cons.alpha);  
        Console.WriteLine("ob.beta: " + ob.beta);  
    }  
}
```

Бұл программа нәтижесі мынадай болады:



Мұнда static типіндегі конструктор класс алғашқы рет жүктелгенде, экземпляр конструкторына дейін автоматты түрде шақырылатынына назар салыңыздар.

Бұдан шығатын жалпы қорытынды: статикалық конструктор кез келген экземпляр конструкторына дейін орындалуы тиіс. Оған қоса, статикалық конструкторларда қол жеткізудимодификаторларыболмайды – оларға келісім бойынша қол жеткізіледі, сондықтан оларды программаданшақыруға болмайды.

6 дәріс. Операторларды асыра жүктеу.

Дәрістің мақсаты: студенттерде операторларды асыра жүктеудің қызметі мен синтаксисі туралы түсініктерін көрсету қабілетін қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Арифметикалық операторларды асыра жүктеу ерекшеліктері бойынша түсініктерін көрсету;
- Логикалық операторларды асыра жүктеу ерекшеліктері бойынша түсініктерін көрсету;
- Типтерді келтіру операторларын асыра жүктеу ерекшеліктері бойынша түсініктерін көрсету.

C# тілінде құрылатын класқа байланысты оператордың атқаратын жұмысын анықтауға болады. Бұл процесс *операторларды асыра жүктеу* деп аталады. Асыра жүктеуге орай класстағы оператордың қолданылу аймағы кеңейтіледі. Мұнда оператордың атқаратын жұмысы толық бақыланады да, оның істейтін қосымша жұмысы нақты класқа байланысты өзгеруі мүмкін. Мысалы, + операторы бір класста объектіні бір байланысқан тізімдерге енгізуі мүмкін, ал басқа класста ол басқа бір әрекет орындауы мүмкін.

Оператор асыра жүктелген кезде оның бұрынғы атқарып отырған жұмыстары жалғаса береді, ол нақты объектіге байланысты тағы бір жаңа операция орындайтын болады. Сондықтан + операторын тізімдерді өңдеуге байланысты асыра жүктеу, мысалы, оның бұрынғы бүтін сандармен орындайтын жұмысын (оларды қосуды) өзгертпейді.

Операторларды асыра жүктеу үшін *операторлық әдісті* анықтайтын `operator` түйінді сөзін қолданамыз, ал ол, өз кезегінде, қолданылатын класына байланысты оператор жұмысын анықтайды.

Операторлық әдістің (**operator**) екі формасы бар: олардың бірі – унарлық операторлар үшін болса, екіншісі – бинарлық операторлар үшін қолданылады. Төменде осы әдістердің әрқайсысының жазылу формасы көрсетілген.

```
// Унарлық операторды асыра жүктеу түрінің жазылуы
public static қайтарылатын_тип operator
    op (параметр_типi операнд)
{
    // операциялар
}
// Бинарлық операторды асыра жүктеу түрінің жазылуы
public static қайтарылатын_тип operator op (параметр1_типi
    о п е р а н д 1 ,   параметр2_типi операнд2)
{
    // операциялар
}
```

Мұнда `op` орнына асыра жүктелетін оператор қойылады, мысалы, + немесе /; ал *қайтарылатын_тип* көрсетілген операция қайтаратын мәннің типін білдіреді. Бұл мән кез келген типте бола береді, бірақ көбінесе ол асыра жүктелген оператор пайдаланылатын кластың типіндей болады. Унарлық операторлар үшін операнд берілетін операндты білдіреді, ал бинарлық операторлар үшін – олар *операнд1* және *операнд2* болып белгіленген. Операторлық әдістердің екеуі де **public** және **static** типінде болатынына назар салыңыздар.

Унарлық операторлардың операнд типі оператор асыра жүктелетін класс типіндей болуы тиіс, ал бинарлық операторларда кемдегенде оның операндтардың біреуі өз класындағыдай типте болуы керек.

Бинарлық операторларды асыра жүктеу

Мысал 1. Операторларды асыра жүктеуді көрсету үшін, + пен - сияқты екі операторды асыра жүктейтін қарапайым мысал қарастырайық. Төмендегі программада комплекс санның нақты және жорамал бөліктері арқылы берілген Complex класы құрылады. Асыра жүктелген "+" операторы комплекс сандардың нақты және жорамал бөліктерін сәйкесінше қосады. Ал асыра жүктелген "-" операторы комплекс сандардың нақты және жорамал бөліктерін сәйкесінше азайтады.

```
// Бинарлық операторларды асыра жүктеу мысалы
using System;
// Комплекс сандармен жұмыс істеуге арналған класс
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // + бинарлық операторын асыра жүктеу
    public static Complex operator +(Complex op1, Complex op2)
    {
        Complex natizhe = new Complex();
        // екі комплекс санның бөліктерін қосып,
        // нәтижесін қайтару
        natizhe.a = op1.a + op2.a;
        natizhe.b = op1.b + op2.b;
        return natizhe;
    }

    // - бинарлық операторын асыра жүктеу.
    public static Complex operator -(Complex op1, Complex op2)
    {
        Complex natizhe = new Complex();
        /* Операндтардың орналасу реттілігіне назар салыңыз:
        op1 - сол жақ операнд, ал op2 - оң жақ операнд. */
        natizhe.a = op1.a - op2.a;
        natizhe.b = op1.b - op2.b;
        return natizhe;
    }
    // комплекс санды шығару
    public void Shygaru()
    {
        Console.WriteLine(a + " + " + b + "i ");
    }
}

class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2 = new Complex(10, 2);
        Complex C3;
        Console.WriteLine("C1 complex sany: ");
        C1.Shygaru();
        Console.WriteLine("C2 complex sany: ");
        C2.Shygaru();
        C3 = C1 + C2; // C1 және C2 комплекс сандарын қосу
        Console.WriteLine("C3 = C1 + C2 qosu natizhesi: ");
        C3.Shygaru();
        C3 = C1 + C2 + C3; // үш комплекс санды қосу
        Console.WriteLine("C3 = C1 + C2 + C3 qosu natizhesi: ");
        C3.Shygaru();
        C3 = C3 - C1; // нәтижелік комплекс саннан бірінші
        // комплекс санды азайту
        Console.WriteLine("C3 = C3 - C1 azaitu natizhesi: ");
        C3.Shygaru();
        C3 = C1 + C3 - C2; // операцияларды аралас пайдалану
        Console.WriteLine("C3 = C1 + C3 - C2 amalyunyg natizhesi: ");
        C3.Shygaru();
    }
}
```

```

Console.WriteLine();
Console.ReadKey();
}
}

```

Бұл программаны орындау нәтижесі:

```

C1 complex sany: 3 + 8i
C2 complex sany: 10 + 2i
C3 = C1 + C2 qosu natizhesi: 13 + 10i
C3 = C1 + C2 + C3 qosu natizhesi: 26 + 20i
C3 = C3 - C1 azaitu natizhesi: 23 + 12i
C3 = C1 + C3 - C2 amalyng natizhesi: 16 + 18i

```

operator+() әдісінің **Complex** типіндегі объект қайтаратынына назар салыңыздар, осы себеппен **+** операторын мынадай **C1 + C2 + C3** құрамалы өрнектерде қолдана аламыз. Мұнда **C1 + C2** өрнегі **Complex** типіндегі нәтиже береді, оны тағы да осы типтегі объектімен қосуға болады. Егер **C1+C2** өрнегі басқа типтегі нәтиже беретін болса, онда **C1 + C2 + C3** құрамалы өрнегі есептеу мүмкін болмас еді. “-” операторы “+” операторы сияқты жұмыс істейді, бірақ бұл оператор үшін операндтардың берілу реттілігі маңызды болып табылады. Барлық екілік операторлар үшін операторлық әдістің бірінші параметрі – сол жақ операнд, ал екінші параметр – оң жақ операнд болып табылады. **operator-()** операторы да **Complex** типіндегі объект қайтаратын болғандықтан, **C3 + C1 - C2** өрнегін де есептеуге болады.

Унарлық операторларды асыра жүктеу

Унарлық операторлар да бинарлық операторлар тәрізді асыра жүктеледі. Бұлардың басты айырмашылығы – бір ғана операндының болуы.

Мысал 2. **Complex** класы үшін унарлық минус операторын асыра жүктеу.

```

using System;
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // - унарлық операторын асыра жүктеу
    public static Complex operator -(Complex op1)
    {
        Complex natizhe = new Complex();
        // комплекс санның нақты және жорамал бөліктерін терістеу
        natizhe.a = -op1.a;
        natizhe.b = -op1.b;
        return natizhe;
    }
    public void Shygaru()
    {
        if(b >= 0) Console.WriteLine(a + " + " + b + "i ");
        else Console.WriteLine(a + " - " + b + "i ");
    }
}
class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2;
        Console.Write("C1 complex sany: "); C1.Shygaru();
        C2 = -C1;
    }
}

```

```

Console.WriteLine("-C1 complex sany: "); C2.Shygaru();
Console.ReadKey();
}
}

```

Бұл программаны орындау нәтижесі:

```

C1 complex sany: 3 + 8i
-C1 complex sany: -3 -8i

```

Бұл мысалда асыра жүктелетін унарлық оператордың өрістерінде операндтың теріс мәндері сақталатын жаңа объект жасалады, соңынан бұл объект операторлық әдіс арқылы қайтарылады. Мұнда операндтың өзгертілмейтініне назар аударыңыздар. Ол унарлық минус операторының кәдімгі атқаратын қызметінің сақталатынын білдіреді. Мысалы, мынадай өрнектің нәтижесі

$a = -b$

b операндының теріс мәні болып табылады, бірақ b операндының мәні өзгермейді.

C# тілінде ++ және -- операторларының асыра жүктелуі қарапайым орындалады. Ол үшін инкременттелген немесе декременттелген мәнді қайтару жеткілікті, бірақоны шақырған объектіні өзгертпеу керек. Бұл операторлардың префикстік және постфикстік формаларын ажырата отырып, қалғандарының барлығын компилятор орындайды.

Мысал 3. Complex класы үшін operator++() операторлық әдісі келтірілген.

```

// ++ унарлық операторын асыра жүктеу
using System;
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // ++ унарлық операторын асыра жүктеу
    public static Complex operator ++(Complex op)
    {
        Complex natizhe = new Complex();
        // Инкременттеу нәтижесін қайтару
        natizhe.a = op.a + 1; natizhe.b = op.b + 1;
        return natizhe;
    }

    // комплекс санды шығару
    public void Shygaru()
    {
        if(b >= 0) Console.WriteLine(a + " + " + b + "i ");
        else Console.WriteLine(a + " - " + b + "i ");
    }
}

class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2;
        Console.WriteLine("C1 complex sany: ");
        C1.Shygaru();
        C2 = C1++;
        Console.WriteLine(++ postfixtik operaciya natizhesi: ");
        C2.Shygaru();
        C2 = ++C2;
        Console.WriteLine(++ prefixtik operaciya natizhesi: ");
        C2.Shygaru();
        Console.ReadKey();
    }
}

```

Бұл программаны орындау нәтижесі:

```
C1 complex sany: 3 + 8i
++ postfixtik operaciya natizhesi: 3 + 8i
++ prefixtik operaciya natizhesi: 4 + 9i
```

C# құрамындағы мәліметтер типтерімен операциялар орындау

Кез келген класс және оператор үшін операторлық әдісті асыра жүктеу мүмкіндігі бар. Бұл, мысалы, класс типімен және де басқа мәліметтер типтерімен (құрамдас типтермен де) операциялар орындау кезінде қажет болады.

Мысал 4. 11.1-ші мысалда + операторының бір **Complex** объектісінің нақты және жорамал бөліктерін басқа бір объектінің нақты және жорамал бөліктеріне сәйкесінше қосу үшін қалай асыра жүктелетіні көрсетілген болатын. Бірақ бұл **Complex** класы үшін қосу операциясын анықтаудың жалғыз тәсілі емес. **Complex** типіндегі объектінің әрбір бөлігіне бүтін сан қосуды орындаған дұрыс болар еді. Бірақ ол үшін + операторын төменде көрсетілгендей тағы да бір рет асыра жүктеу керек болады.

```
using System;
// Комплекс сандармен жұмыс істеуге арналған класс
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // + бинарлық операторын асыра жүктеу
    public static Complex operator +(Complex op1, Complex op2)
    {
        Complex natizhe = new Complex();
        // екі комплекс санның бөліктерін қосып, нәтижесін қайтару
        natizhe.a = op1.a + op2.a;
        natizhe.b = op1.b + op2.b;
        return natizhe;
    }

    public static Complex operator +(Complex op1, int op2)
    {
        Complex natizhe = new Complex();
        // комплекс санға бүтін санды қосып, нәтижесін қайтару
        natizhe.a = op1.a + op2;
        natizhe.b = op1.b + op2;
        return natizhe;
    }
    // комплекс санды шығару
    public void Shygaru()
    {
        Console.WriteLine(a + " + " + b + "i ");
    }
}

class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2 = new Complex(10, 2);
        Complex C3; int x = 5;
        Console.Write("C1 complex sany: ");
        C1.Shygaru();
        Console.Write("C2 complex sany: ");
        C2.Shygaru();
    }
}
```

```

C3 = C1 + C2; // C1 және C2 комплекс сандарын қосу
Console.WriteLine("C3 = C1 + C2 қосу нәтижесі: ");
C3.Shygaru();
C3 = C1 + x; // комплекс санға бүтін санды қосу
Console.WriteLine("C3 = C1 + x қосу нәтижесі: ");
C3.Shygaru();
Console.WriteLine();
Console.ReadKey();
}
}

```

Бұл программаны орындау нәтижесі:

```

C1 complex sany: 3 + 8i
C2 complex sany: 10 + 2i
C3 = C1 + C2 қосу нәтижесі: 13 + 10i
C3 = C1 + x қосу нәтижесі: 8 + 13i

```

Мұндағы операторлық әдістің екінші параметрі `int` типінде. Сондықтан, бұл әдісте бүтін мәнді **Complex** типіндегі объектінің әрбір өрісімен қосуға рұқсат етіледі. Мұндай операция орындала алады, өйткені бұрын айтылғандай, бинарлық операторды асыра жүктегенде, осы (асыра жүктелу керекті) класта оның бір операнды класс типінде болуы тиіс. Бірақ оның екінші операнды кез келген типте бола береді.

+ операторын асыра жүктеу, **Complex** класының пайдалы функцияларын кеңейтеді, дегенмен, оны ол аяғына дейін орындамайды. Оның себебі `operator + (Complex, int)` әдісі келесідей операцияларды орындауға мүмкіндік береді:

```
C3 = C1 + 10;
```

бірақ ол келесідей операцияларды орындай алмайды,

```
C3 = 10 + C1;
```

Бұл әдісте екінші бүтін аргумент + бинарлық операторының оң жақ операндын белгілейді, бірақ жоғарыда көрсетілген жолда бүтін аргумент сол жақта көрсетіледі. Мұндай қосу операциясын орындауға рұқсат беру үшін, + операторындағы да бір рет асыра жүктеу керек. Мұндағы жағдайда операторлық әдістің бірінші параметрі `int` типінде болуы керек, ал екінші параметрі – тип **Complex** типінде болуы тиіс.

Қатынас операторларын асыра жүктеу

`==` және `<` тәрізді қатынас операторлары да өте оңай асыра жүктеледі. Бұлар көбінесе асыра жүктелу кезінде логикалық **true** және **false** мәндерін қайтарады. Бұл мүмкіндік осылай операторларды шартты операторларда қолдану мүмкіндігін береді. Егерде нәтиже басқа типте болса, қатынас операторларын қолдану аясы кішірейеді.

5-мысал. `<` және `>` операторларын **Complex** класы арқылы асыра жүктеу керек. Бұл мысалдағы операторлар комплекс сандардың модульдерін салыстырады. Комплекс санның модулі оның нақты және жорамал бөліктерінің квадраттары қосындысын түбір астынан шығару арқылы анықталады. Егер де бір де бір оператор логикалық **true** мәнін қайтармаса, онда екі комплекс санның модульдері тең болып саналады. Әрине, бұдан басқа да реттеу алгоритмдері болуы мүмкін.

```

using System;
// Комплекс сандармен жұмыс істеуге арналған класс
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // < қатынас операторын асыра жүктеу
    public static bool operator <(Complex op1, Complex op2)
    {

```

```

double mop1 = Math.Sqrt(Math.Pow(op1.a,2) +
    M a t h . P o w ( o p 1 . b ,      2 ) ) ;
double mop2 = Math.Sqrt(Math.Pow(op2.a, 2) +
    M a t h . P o w ( o p 2 . b ,      2 ) ) ;
// екі комплекс санның модульдерін салыстыру
if(mop1 < mop2) return true;
else return false;
}
// > қатынас операторын асыра жүктеу
public static bool operator >(Complex op1, Complex op2)
{
double mop1 = Math.Sqrt(Math.Pow(op1.a, 2) +
    M a t h . P o w ( o p 1 . b ,      2 ) ) ;
double mop2 = Math.Sqrt(Math.Pow(op2.a, 2) +
    M a t h . P o w ( o p 2 . b ,      2 ) ) ;

if (mop1 > mop2) return true;
else return false;
}
// комплекс санды шығару
public void Shygaru()
{
Console.WriteLine(a + " + " + b + "i ");
}
}
class Program {
static void Main() {
Complex C1 = new Complex(3, 8);
Complex C2 = new Complex(10, 2);
Console.Write("C1 complex sany: ");
C1.Shygaru();
Console.Write("C2 complex sany: ");
C2.Shygaru();
if(C1 > C2) // C1 және C2 комплекс сандарын
// > шартына салыстыру
Console.WriteLine("C1 complex sany C2 complex sanynan
u l k e n " ) ;
else if (C1 < C2) // C1 және C2 комплекс сандарын
// < шартына салыстыру
Console.WriteLine("C1 complex sany C2 complex sanynan
k i s h i " ) ;
e l s e
Console.WriteLine("C1 zhane C2 complex sandary ten");
Console.WriteLine();
Console.ReadKey();
}
}

```

Бұл программаның нәтижесі:

```

C1 complex sany: 3 + 8i
C2 complex sany: 10 + 2i
C1 complex sany C2 complex sanynan kishi

```

Қатынас операторларының асыра жүктелуіне мынадай бір шектеу қойылады: олар жұп-жұбымен асыра жүктелуі тиіс. Егер < операторы асыра жүктелетін болса, онда > операторын да асыра жүктеу керек. Төменде жұптасып асыра жүктелетін қатынас операторлары көрсетілген.

==	!=
<	>
<=	>=

true және false операторларынасыра жүктеу

true және **false** түйінді сөздерін асыра жүктеу кезінде унарлық операторлар ретінде қолдануға болады. Бұлардың асыра жүктелу нұсқалары құрылатын кластар үшін **true** және **false** түйінді сөздерінің атқаратын қызметін анықтай алады. Бұл түйінді сөздерді нақты класс үшін унарлықоператорлар ретінде асыра жүктегеннен кейін бұл класс объектілерін **if**, **while**, **for**, **do-while** операторларын басқару мен ? шартты өрнегінде қолдану мүмкіндігі пайда болады.

true және **false** операторлары жеке-жеке емес, жұбымен (парымен) асыра жүктеледі. Бұл екеуінде де **bool** типіндегі мән қайтарылады. Төменде осы унарлық операторлардың асыра жүктелуінің жалпы формасы келтірілген.

```
public static bool operator true(параметр _ типі операнд) {  
    // true немесе false логикалық мәндерін қайтару  
}  
public static bool operator false(параметр _ типі операнд) {  
    // true немесе false логикалық мәндерін қайтару  
}
```

6-мысал. **Complex** класында **true** және **false** операторларын қолдану. Егер **Complex** типіндегі объектінің нақты және жорамал бөліктерінің кемінде біреуі нөлге тең болса, онда бұл объект жалған болады деп тұжырымдаймыз.

```
using System;  
// Комплекс сандармен жұмыс істеуге арналған класс  
class Complex {  
    int a, b; // санның нақты және жорамал бөліктері  
    public Complex() { a = b = 0; }  
    public Complex(int x, int y) { a = x; b = y; }  
  
    // true операторын асыра жүктеу  
    public static bool operator true(Complex op1)  
    {  
        if (op1.a != 0 && op1.b != 0) return true;  
        else return false;  
    }  
    public static bool operator false(Complex op1)  
    {  
        if (op1.a == 0 || op1.b == 0) return true;  
        else return false;  
    }  
    public static Complex operator --(Complex op1)  
    {  
        Complex natizhe = new Complex();  
        natizhe.a = op1.a - 1;  
        natizhe.b = op1.b - 1;  
        return natizhe;  
    }  
    // комплекс санды шығару  
    public void Shygaru()  
    {  
        Console.WriteLine(a + " + " + b + "i ");  
    }  
}  
  
class Program {  
    static void Main() {  
        Complex C1 = new Complex(3, 8);  
        Complex C2 = new Complex(0, 2);  
        Console.WriteLine("C1 complex sany: ");  
    }  
}
```

```

C1.Shygaru();
Console.WriteLine("C2 complex sany: ");
C2.Shygaru();
if(C1) // C1 және C2 комплекс сандарын қосу
{
    while (C1) {
        Console.WriteLine("C1 complex sany aqiqat");
        - - C 1 ;
        C1.Shygaru();
    }
    Console.WriteLine("C1 complex sany zhalgan");
}
else Console.WriteLine("C1 complex sany zhalgan");

if(C2) // C1 және C2 комплекс сандарын қосу
Console.WriteLine("C2 complex sany aqiqat");
else Console.WriteLine("C2 complex sany zhalgan");
Console.WriteLine();
Console.ReadKey();
}
}

```

Программаның нәтижесі:

```

C1 complex sany: 3 + 8i
C2 complex sany: 0 + 2i
C1 complex sany aqiqat
2 + 7i
C1 complex sany aqiqat
1 + 6i
C1 complex sany aqiqat
0 + 5i
C1 complex sany zhalgan
C2 complex sany zhalgan

```

If операторында **Complex** типіндегі объект **true** операторы арқылы тексеріледі. Егер мұндағы нәтиже ақиқат болса, онда **if** операторы орындалады. Ал **while** операторында **c1** объектісі циклдің әрбір қадамында декременттеледі. Сондықтан, цикл **c1** объектісінің тексерілуі ақиқат мән беріп тұрса, орындала береді. Егер **c1** объектісінің кем дегенде бір бөлігі 0-ге тең болса, онда **true** операторы жалған нәтиже береді де, цикл аяқталады.

Түрлендіру операторлары

Кейде белгілі бір класс объектісін басқа типтегі мәліметтері бар өрнектерде қолдануға тура келеді. Осымақсатта кей кезде бір немесе бірнеше операторды асыра жүктеу жеткілікті болса, кейде – класс типін керекті типке қарапайым түрлендіру керек болып жатады.

C# тілінде осындай жағдайлар үшін операторлық әдістің арнайы *түрлендіру операторы* деп аталатын бір түрі қарастырылған. Осындай оператор берілген класс объектісін басқа типке түрлендіреді. Түрлендіру операторлары, типтерді түрлендіру тәртібі анықталған жағдайда, кластарды басқа типтермен бірге еркін қолдануға мүмкіндік беріп, кластар типін программалау ортасына толық енгізу ісін атқара алады.

Түрлендіру операторларының екі формасы бар: *тікелей* және *жанамалы*. Олардың жазылу жолдары:

```

public static explicit operator керекті_тип(бастапқы_тип v)
{ return мән; }
public static implicit operator керекті_тип(бастапқы_тип v)
{ return мән ; }

```

мұндағы *керекті_тип* түрлендірілетін жаңа типті білдіреді; *бастапқы_тип* – өзгертілетін бұрынғы тип; *мән* – класс үшін түрлендіруден кейін орнатылатын нақты мән. Түрлендіру операторлары *керекті_типке* көшірілген мәліметтерді қайтарады, мұнда қайтарылатын басқа типтер көрсетуге рұқсат етілмейді.

Егер түрлендіру операторы жанамалы (**implicit**) формада көрсетілсе, онда түрлендіру автоматты түрде шақырылады, бұл, мысалы, объект өрнектегі керекті типтегі мәнмен бірге пайдаланылған кездерде орын алады. Егерде түрлендіру операторы тікелей формада (**explicit**) көрсетілсе, онда түрлендіру, типтерді өзгертуді (келтіруді) орындау кезінде шақырылады. Мәліметтердің берілген бір бастапқы және керекті типтері үшін түрлендіру операторын тікелей және жанамалы түрде қатарластыра көрсетуге болмайды.

7-мысал. Түрлендіру операторын **Complex** класында қолдану. **Complex** типіндегі объектіні нақты мәнге түрлендіру (оны кейін нақты сандық өрнекте қолдану үшін) қажет болсын делік. Мұндай әрекет комплексанның модулін есептеу арқылы орындалады. Түрлендіруді орындаудың жанамалы формасын құрайық.

```
using System;
// Комплекс сандармен жұмыс істеуге арналған класс
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // түрлендіру операторын асыра жүктеу
    public static implicit operator double(Complex op1)
    {
        return Math.Sqrt(Math.Pow(op1.a, 2) + Math.Pow(op1.b, 2));
    }

    // комплекс санды шығару
    public void Shygaru()
    {
        Console.WriteLine(a + " + " + b + "i ");
    }
}
class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2 = new Complex(0, 2);
        Console.Write("C1 complex sany: ");
        C1.Shygaru();
        Console.Write("C2 complex sany: ");
        C2.Shygaru();
        double x = C1;
        Console.WriteLine("C1 complex sanynyn moduli = " + x);
        x = x + C2;
        Console.WriteLine("C1 zhane C2 complex sandarynyn
        modulderinin kosyndysy = " + x);
        Console.WriteLine();
        Console.ReadKey();
    }
}
```

Бұл программаның нәтижесі:

```
C1 complex sany: 3 + 8i
C2 complex sany: 0 + 2i
C1 complex sanynyn moduli = 8.54400374531753
C1 zhane C2 complex sandarynyn modulderinin
kosyndysy = 10.5440037453175
```

Бұлмысалда **Complex** типіндегі объект нақты сандық типте (**x = c1**) пайдаланылған кезде, операторды түрлендіру ісі орындалады. Бірақ жұмыс барысында өрнекті есептеу үшін **double** типіне түрлендіру қажет етілмесе, онда операторды түрлендіру әрекеті шақырылмайды.

Жанамалы түрлендіру операторы келесідей жағдайларда:

- А) өрнектерде типтерді түрлендіру қажет болғанда;
- Ә) әдіске объект беру кезінде;
- Б) меншіктеу әрекеті орындалып, керекті типке тікелей көшу ісін атқару керек болған сәттерде автоматты түрде жүзеге асады.

Басқа жағынаналатын болсақ, тек тікелей типті өзгерту ісі керек болған кездерде шақырылатын тікелей түрлендіру операторын да жасауға болады. Мұндайда тікелей түрлендіру операторы автоматты түрде шақырылмайды.

8-мысал. Тікелей түрлендіру операторын **Complex** класында тікелей қолдану нұсқасын құрайық.

```
using System;
// Комплекс сандармен жұмыс істеуге арналған класс
class Complex {
    int a, b; // санның нақты және жорамал бөліктері
    public Complex() { a = b = 0; }
    public Complex(int x, int y) { a = x; b = y; }

    // тікелей түрлендіру операторын асыра жүктеу
    public static explicit operator double(Complex op1)
    {
        return Math.Sqrt(Math.Pow(op1.a, 2) + Math.Pow(op1.b, 2));
    }

    // комплекс санды шығару
    public void Shygaru()
    {
        Console.WriteLine(a + " + " + b + "i ");
    }
}

class Program {
    static void Main() {
        Complex C1 = new Complex(3, 8);
        Complex C2 = new Complex(0, 2);
        Console.WriteLine("C1 complex sany: ");
        C1.Shygaru();
        Console.WriteLine("C2 complex sany: ");
        C2.Shygaru();
        double x = (double)C1;
        Console.WriteLine("C1 complex sanynyng
            moduli = { 0 : # . ## } " , x );
        x = x + (double)C2;
        Console.WriteLine("C1 zhane C2 complex sandarynyng
            modulderining qosyndysy = {0: # . ## } " , x);
        Console.WriteLine();
        Console.ReadKey();
    }
}
```

Бұл программаның нәтижесі:

```
C1 complex sany: 3 + 8i
C2 complex sany: 0 + 2i
C1 complex sanynyng moduli = 8.54
C1 zhane C2 complex sandarynyng modulderining qosyndysy = 10.54
```

Тікелей немесе жанамалы түрлендірулерді операторларын таңдау кезінде бұл шектеулерге қосымша бірсыпыра ұсыныстар беріледі. Жанамалы түрлендірулердің барлық артықшылықтарына қарамастан, оларды тек түрлендіруде қателер болмайтындай жағдайларда қолдану керек.

Қателерді болдырмас үшін жанамалы түрлендірулер келесідей шарттар орындалатын кездерде ғана ұйымдастырылуы тиіс.

1) алдымен, мысалы, қысқарту, толып кету немесе таңбаны жоғалту нәтижелерінде мәлімет жоғалмайтындай болуы керек.

2) түрлендіру ерекше жағдайды туындатпауы тиіс.

Егер де жанамалы түрлендіру осы екі шартты қанағаттандырмаса, онда тікелей түрлендіруді таңдау қажет.

Видеожазбалар:

Өрнектер мен операторлар

<https://youtu.be/6g91zl5xC50?si=7MZ0XDnMi0SIInkvF>

Операторлардың басымдылығы

https://youtu.be/uo8GyPz_8JA?si=yoRnvGnU3iSYon5l

7 дәріс. Нуль мән қабылдайтын типтер. Бөлшектенген кластар және бөлшектенген әдістер. Кеңейтуші әдістер.

Дәрістің мақсаты: студенттерде нуль мән қабылдайтын және бөлшектенген типтердің, кеңейтуші әдістердің қызметі және оларды пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Нуль мән қабылдайтын типтердің қызметі және оларды пайдалану ерекшеліктері туралы түсініктерін көрсету;
- Бөлшектенген типтер мен әдістердің қызметі және оларды пайдалану ерекшеліктері туралы түсініктерін көрсету;
- Кеңейтуші әдістердің қызметі және оларды пайдалану ерекшеліктері туралы түсініктерін көрсету.

Нуль мән қабылдайтын типтер

Ағымдағы қалдықпен шот-фактуралар және Тапсырыс берушінің мекен-жайына, нөмірі, сәйкестендіру нөмірлері тапсырыс берушілердің атаулы жазба деректер базасын шотыңызда сақталады. Осындай жағдайда тапсырыс берушінің толық емес деректер нүктесі құрылды, онда мүмкін осы бір немесе бірнеше өріс инициалданған. Мысалы, жай ғана пайдаланылмаған болса және бұл жағдайда, тапсырыс беруші өнім каталогы шот-фактураның нөмірі, демек, оны өріс сұрау қажет болмайды.

Өңдеу үшін мәндерді немесе жай ғана өріс немесе өрістер қолдануға тура келді, олар өздерінің көрсетпеген бұрын пайдаланылмайтын пайдаланылады қосымша өрістер жоқ. Сөз жоқ, олай болмаған жағдайда, егер олар өздерінің маңызы бар жағдайда ғана жарамды тосылып жатыр, бірақ әрқашан осылай болады, ол орнына маңызы бар жарамсыз болып жатыр. Ал қолмен енгізу және өңдеу өрісі пайдаланылады, жоқ па, бірақ олардың барлық жағдайларда көрсететін қосымша өрістерді жарамды деген шаруа аз емес. Осы екі түрі көп қиыншылықтар нуль мән беруге мүмкіндік береді.

Нуль мән қабылдайтын типтерді қолдану негіздері

Ол ерекше нуль мән қабылдайтын типтер нұсқасы бір түрінің маңызы бар құрылым. Айқындалатын басқа базалық нуль мән қабылдайтын типтер мәндердің түрі бос мәндерге сақтауға мүмкіндік береді (бос). Демек, осындай ауқым бар сандарды ұсыну мен сипаттамаларын және оның негізгі түрі ретінде нуль мән қабылдайтын типтер. Ол қосымша мүмкіндік береді, яғни айнымалы мәнін көрсетуге маңызы осы түрдегі баптандырылмаған. System нуль мән қабылдайтын типтер объектілері болып табылады. T - түрі маңызы бар, онда nullable < T > нуль мән қабылдайтын тип болмауға тиіс.

Нуль мән қабылдайтын типтер екі тәсілмен көрсетілуі мүмкін. Біріншіден, мүмкін, аттар кеңістігіндегі түрінің нысандарын Nullable < T > белгілі бір System, әбден айқын түрде жарияланды. Мәселен, өзгермелі үлгідегі bool int және төменде келтірілген мысалда нуль мән қабылдайтын типтер құрылады.

```
System.Nullable < int > Count;
```

```
System.Nullable < bool > done;
```

Екіншіден, нуль мән қабылдайтын типтер жиі жарияланады және қысқа болуы және сондықтан көрсете отырып тәсілімен пайдаланылатын таңбаның? базалық атындағы кейін түрі. Төменде келтірілген сақталуы тиіс және одан да көп таралған түрі int bool нуль мән қабылдайтын айнымалы хабарландыру тәсілі арқылы көрсетіп жатыр.

```
int? count; bool? done;
```

Әдетте төмендегідей түрлері нуль мән қабылдайтын нысан кодында нуль мән қабылдайтын типтер құрылатын кезде қолданылады.

```
int?count = null;
```

Бұл жолда айнымалы мәні анық инициалданатын кодын count бос (бос). Қабылданған Ереже бұл толық сәйкес келеді: Айнымалы пайдалануға, оған бұрын тағайындау қажетмән. Бұл жағдайда екенін білдіреді берілетін айнымалы мәні анықталмаған.

Бөлшектенген типтер

C # құрылымының немесе екі немесе одан да көп бөліктерге бөлу 2,0 нұсқасынан бастап, сыныбы, олардың әрқайсысының сақтай отырып жеке файлда интерфейс айқындау мүмкіндігі пайда болды. Бұл кілт сөз partial мәтінмәндік көмегімен жасалады. Осы бөлімдергебірге алғанда құрастыру кезінде барлық бағдарламалары.

Егер partial өзгерткіші бөлшектенген тип жасау үшін пайдаланылады, онда ол мынадай жалпы нысанын қабылдайды:

```
partial тип_аты түрі{ //.
```

Класс аты білдіреді, онда тип_атыинтерфейс, құрылымының немесе орайда бөлінетін бөліктерге. Әр бөлігі бөлшектенген үлгідегі partial модификаторыменбірге көрсетілуі тиіс.

Қарапайым мысал қарастырайық, үш кезеңге бөлу координаттары бар бөлшектенген класс, XY файл. Файл мазмұнын қайтарылған бірінші.

```
{partialclass XY
publicXY (int int a, b) {
X = a;
Y = b;
}
}
```

Бұдан әрі екінші файлдың мазмұнын керек.

```
{partialclass XY
public{int X get; set; }
}
```

Файл мазмұнын, ақыры, үшінші.

```
{partialclass XY
public{int Y get; set; }
}
```

Бастапқы мәтін файл класс XY көрсетіп жатыр бағдарламасын қолдану төменде келтірілген сақталуы тиіс.

```
Көрсету ішінара
// айқындауқажет.
using System;
class Test {
static {void Main (
XY xy = new XY (1, 2);
Console.WriteLine (xy.X xy. Y);
}
}
```

Пайдалану үшін класпен XY компиляцияны қосу қажет ету үшін оның барлық файлдар. Мәселен, егер XY файлдарды сыныпты ху1 деп аталады. cs, ху2. cs және ху3. cs класы бар, ал Test файлында test. Пәрмен жолында енгізу үшін жеткілікті болса, онда оның CS құрастыру хабарлайды.

```
csc test.cs ху1.cs ху2.cs ху3.cs
```

Және ең соңғы ескертпе: Жаңа C # жинақталған сыныптарішінаражол беріледі. Бірақ оның екі жақ сәйкес болуы керек, бұлхабарландырудатүрінің тағайындауын әрбір осындай сынып қалғанбөліктерінде көрсетіледі.

Бөлшектенген әдістер

Алдыңғы Бөлімде түрлендіргішті partial класы ретінде түсіндірді көмегімен ішінара түрін жасауға болады. C # және ішінара әдісін пайдалану мүмкіндігін жасау үшін деректер пайда болды, бұл 3,0 нұсқасынан бастап өзгерткіші элементінде ішінара түрі. Оның бір бөлігінде жарияланады, ал басқа ішінара әдісі іске асырылады. Демек, оны сатудан немесе құрылымы ішінара сыныпта түрлендіргішті көмегімен хабарландыру partial әдісін бөлу керек болады.

Әдісін частичного басты ерекшелігі мынада, оны жүзеге асыру талап етілмейді! Егер іске асырылмай отыр, онда оның барлық шақырулар басқа бөлігіндегі сыныбынан немесе құрылымын ішінара әдісі үнсіз ескермеуде. Бұл қосымша міндетті функцияларын айқындауға мүмкіндік береді, бірақ ол болмаса да, ізденіп алу қажет. Егер бұл функцияларды іске асырылмаған болса, онда олар жай ғана ескермеуде.

Төменде келтірілген, онда кеңейтілген нұсқасы алдыңғы бағдарламаның ішінара құрылады Show әдісі (). Бұл әдіс басқа әдіспен іске қосылады ShowXY (). Барлық бөлігі ыңғайлы болу үшін бір файлда ұсынылды, бірақ олар көрсеткендей, алдыңғы Бөлімде көрсетілген жекелеген фацлам сынып XY бөлінуі мүмкін.

// Көрсету ішінара әдісін қолдану.

```
using System;
{partialclass XY
publicXY (int int a, b) {
X = a;
Y = b;
}
Void // ішінара жариялау әдісі, partial Show ();
}
{partialclass XY
public{int X get; set; }
Ішінарасатуға // partial Show() void {әдіс
Console.WriteLine (" {0}, {1}", X, Y);
}
}
{partialclass XY
public{int Y get; set; }
Ішінараәдісі, public void тудыруы // {ShowXY (
Show ();
}
}
class Test {
static {void Main (
XY xy = new XY (1, 2); xy. ShowXY ();
}
}
```

Екінші сыныпты жарияланады, ал оның бір бөлігін XY Show () бір бөлігі екенін ескеріңіз әдісі іске асырылады. Бұл дегеніңіз осы әдісті іске асыруда координаттар әдісі () ішінен ShowXY Show кезде X пен Y мәндері шығарылады, онда бұл әдісін шақыру шынында да нақты іске қосылады. () бар зардаптар: Бірақ егер Show (), онда оның координаттары X пен Y мәндерінің шығару әдісін іске асыруға ShowXY әдісін шақыру ештеңеге әкел меңді. () ішіненТүсініктеме қылу.

Ішінара әдістеріне бірқатар тән мынадай шектеулер. Void түрінің мәні қайтаруы керек. Оларда түрлендіргіштерді виртуалды мүмкін емес және олар қол жеткізу мүмкін емес. Сонымен қатар оларда out параметрлерін пайдалану мүмкін емес.

8 дәріс. Мұралау. Мұралау және атауларды жасыру. Көпдеңгейлі иерархияны құру.

Дәрістің мақсаты: студенттерде кластар иерархиясын құру мақсатында мұралау механизмінің идеясы туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Мұралау механизмінің идеясы бойынша түсініктерін көрсету;
- Көпдеңгейлі иерархияны құру ерекшеліктері бойынша түсініктерін көрсету;
- Мұралау кезінде атауларды жасыру механизмі бойынша түсініктерін көрсету.

Мұралау – объектіге бағытталған программалаудың негізгі қағидаларының бірі, ол иерархиялық жіктеуді құруға мүмкіндік береді. Мұралаудың арқасында байланысқан элементтер жиынына тән ерекшеліктер анықталатын жалпы класты құруға болады. Бұл кластан кейіннен өздерінің ерекшеліктерін қосу арқылы басқа, айтарлықтай нақтыланған кластар мұралай алады.

C# тілінде мұраланатын класс базалық, ал мұралайтын класс туынды класс деп аталады. Ендеше, туынды класс базалық кластың арнайы нұсқасы болып табылады. Ол базалық класта анықталатын барлық айнымалылар, әдістер, қасиеттер және индексаторларды мұралайды да, оларға өзіндік элементтерін қосады.

C# тілінде мұралауды қолдау бір кластың жариялануына екінші класты енгізу арқылы жүзеге асырылады. Ол үшін туынды класты жариялау кезінде базалық класс көрсетіледі. Төменде базалық кластан мұралайтын туынды класты жариялаудың жалпы формасы берілген:

```
class туынды_класс_аты : базалық_класс_аты {  
    // класс тұлғасы  
}
```

Туынды кластың жариялануындағы қоснұкте : белгісінің қолданылуына назар аударыңыз. Кез келген туынды класс үшін бір ғана базалық класты көрсетуге болады. Алайда мұралау иерархиясын құрып, туынды класты басқа класс үшін базалық класс ретінде пайдалану мүмкіндігі бар.

І-мысал. Өнімнің атын, өндірушісін, бағасын сипаттайтын базалық класс берілген. Осы ақпаратқа қосымша өнімнің дүкендегі бар саны берілетін Тауар класын құру керек делік.

```
using System;  
// Өнімдерді сипаттауға арналған класс  
class Onim {  
    public string aty;  
    public string ondirushi;  
    public double bagasy;  
    public void Shygaru()  
    {  
        Console.WriteLine(ondirushi + " ondirushisining " + aty + "  
        onimining bagasy - " + bagasy);  
    }  
}  
// Өнім класынан туынды тауар класы  
class Tauar : Onim {  
    public int sany;  
    // тауар туралы ақпаратты шығару  
    new public void Shygaru() {  
        base.Shygaru(); Console.WriteLine(", al sany - " + sany);  
    }  
    // Берілген тауардың жалпы бағасы  
    public void zhalpy_baga()
```

```

    {
        Console.WriteLine("Tauardyng zhalpy bagasy: " +
            b a g a s y * s a n y ) ;
    }
}
class Program {
    static void Main() {
        Onim onim001 = new Onim();
        onim001.aty = "notebook";
        onim001.ondirushi = "Acer";
        onim001.bagasy = 200000;
        onim001.Shygaru(); Console.WriteLine();
        Tauar tauar001 = new Tauar();
        tauar001.aty = "notebook";
        tauar001.ondirushi = "Acer";
        tauar001.bagasy = 200000;
        tauar001.sany = 15;
        tauar001.Shygaru();
        tauar001.zhalpy_baga();
        Console.ReadKey();
    }
}

```

Бұл программаның орындалу нәтижесі келесідей болады:

```

Acer ondirushisining notebook onimining bagasy - 200000
Acer ondirushisining notebook onimining bagasy - 200000, al
sany - 15
Tauardyng zhalpy bagasy: 3000000

```

Program класында **Onim** және **Tauar** екі класының объектілері құрылады. Мұндағы **Tauar** класының құрамында тауардың аты, өндірушісі және бағасы туралы айнымалылар анықталмағанына қарамастан, класс объектісін құрғаннан кейін ол үшін аталған параметрлердің мәнін көрсетуге болады, бұл экземпляр айнымалылары базалық кластан мұраланады. Базалық кластың әдістерін де туынды класта қайта жарияламастан пайдалануға болады. Жоғарыдағы мысалда базалық кластың **Shygaru()** әдісі туынды класта толықтыруды қажет ететіндіктен, ол тікелей мұраланбай, қайта анықталады. Әдісті қайта анықтау арқылы оның қызметін туынды класта өзгертуге болады. Базалық кластың әдістері туынды класта **new** кілттік сөзінің көмегімен қайта анықталады, қайта анықталған әдістің құрамында базалық кластың әдісін шақыру үшін **base** кілттік сөзі қолданылады. Туынды класта **Shygaru()** әдісінен басқа, берілген тауардың дүкенде бар саны үшін жалпы бағаны анықтайтын **zhalpy_baga()** әдісі анықталған, базалық класс құрамына кірмейтін әдіс туынды класта әдеттегідей түрде анықталады.

Мысалда көрсетілгендей, **Onim** класы **Tauar** класы үшін базалық класс болғанына қарамастан, ол өз алдына тәуелсіз класс болып табылады және әдеттегі класс сияқты қолданыла алады. Бірақ **Onim** класының объектілері оның туынды кластарымен байланыспайды.

Мұралауда кластың мүшелеріне қатынасу

Өткен тарауларда көрсетілгендей, класс мүшелерін рұқсатсыз қолданудың алдын алу үшін оларды көп жағдайда жабық түрде жариялайды. Класты мұралау бұл шектеулерден тыс қалмайды, туынды класс құрамында базалық кластың жабық мүшелері қол жетімсіз болады.

Аталған шектеуді жеңудің жолдары – қорғалған (**protected**) класс мүшелерін пайдалану немесе жабық мәліметтерге қол жеткізу үшін ашық қасиеттерді қолдану. Базалық класта жарияланған әдістер сияқты, ашық қасиеттер де туынды кластарда

қолжетімді болады. Базалық класс айнымалыларына ашық қасиеттердің көмегімен қатынасудың кемшілігі – туындыкласта бұл айнымалыларды тікелей пайдалана алмаймыз.

2-мысал. Базалық кластың жабық мүшелерінің мәнін тағайындау және шығарып алу үшін ашық қасиеттерді пайдалану

```
using System;
// Өнімдерді сипаттауға арналған класс
class Onim {
    private string aty;
    private string ondirushi;
    private double bagasy;
    // Қасиеттер
    public string Aty
    {
        get { return aty;}
        set {aty = value;}
    }
    public string Ondirushi
    {
        get { return ondirushi;}
        set { ondirushi = value;}
    }
    public double Bagasy
    {
        get { return bagasy;}
        set { if(value>0) bagasy = value;
            else bagasy = 0; }
    }
    public void Shygaru()
    {
        Console.WriteLine(ondirushi + " ondirushisining " + aty + "
            onimining bagasy - " + bagasy);
    }
}
// Өнім класынан туынды тауар класы
class Tauar : Onim {
    public int sany;
    // тауар туралы ақпаратты шығару
    new public void Shygaru()
    {
        base.Shygaru(); Console.WriteLine(", al sany - " + sany);
    }
    // Берілген тауардың жалпы бағасы
    public void zhalpy_baga()
    {
        Console.WriteLine("Tauardyng zhalpy bagasy: "
            + Bagasy * sany);
    }
}
class Program {
    static void Main() {
        Onim onim001 = new Onim();
        onim001.Aty = "notebook";
        onim001.Ondirushi = "Acer";
        onim001.Bagasy = 200000;
        onim001.Shygaru(); Console.WriteLine();
        Tauar tauar001 = new Tauar();
        tauar001.Aty = "notebook";
        tauar001.Ondirushi = "Acer";
        tauar001.Bagasy = 200000;
        tauar001.sany = 15;
        tauar001.Shygaru();
        tauar001.zhalpy_baga();
    }
}
```

```

    Console.ReadKey();
}
}

```

Бұл программаның орындалу нәтижесі келесідей болады:

```

    Acer ondirushisining notebook onimining bagasy - 200000
    Acer ondirushisining notebook onimining bagasy - 200000,
al sany - 15
    Tauardying zhalpy bagasy: 3000000

```

Көрсетілген мысалда **Aty**, **Ondirushi**, **Bagasy** ашық қасиеттері кластың **aty**, **ondirushi**, **bagasy** жабық мүшелеріне қатынасуға мүмкіндік береді.

Базалық класта жарияланған жабық мүшелерді туынды класта пайдаланудың келесі тәсілі қорғалған мүшелерді пайдалану екендігі жоғарыда айтылған. Кластың қорғалған мүшесі кластар иерархиясы аясында ашық, ал иерархиядан тыс жабық болады. Кластың қорғалған мүшесі **protected** қатынасу модификаторының көмегімен құрылады.

3-мысал. **protected** қатынасу модификаторын пайдалану мысалы.

```

using System;
// Өнімдерді сипаттауға арналған класс
class Onim {
    protected string aty;
    protected string ondirushi;
    protected double bagasy;
    public void manderi(string a, string o, double b)
    {
        aty = a, ondirushi = o, magasy = b;
    }
    public void Shygaru()
    {
        Console.Write(ondirushi + " ondirushisining " + aty + "
onimining bagasy - " + bagasy);
    }
}
// Өнім класынан туынды тауар класы
class Tauar : Onim {
    private int sany;
    public int Sany
    {
        get { return sany; }
        set { if(value > 0) sany = value;
else sany = 0; }
    }
    // тауар туралы ақпаратты шығару
    new public void Shygaru()
    {
        base.Shygaru(); Console.WriteLine(", al sany - " + sany);
    }
    // Берілген тауардың жалпы бағасы
    public void zhalpy_baga()
    {
        Console.WriteLine("Tauardying zhalpy bagasy: "
+ bagasy * sany);
    }
}
class Program {
    static void Main() {

```

```

Onim onim001 = new Onim();
onim001.manderi("notebook", "Acer", 200000);
onim001.Shygaru(); Console.WriteLine();
Tauar tauar001 = new Tauar();

tauar001.manderi("notebook", "Acer", 200000);
tauar001.Sany = 15;
tauar001.Shygaru();
tauar001.zhalpy_baga();
Console.ReadKey();
}
}

```

Бұл программаның орындалу нәтижесі келесідей болады:

```

Acer ondirushisining notebook onimining bagasy - 200000
Acer ondirushisining notebook onimining bagasy - 200000,
al sany - 15
Tauardying zhalpy bagasy: 3000000

```

Protected қатынасу модификаторы тек кластар иерархиясына қолжетімді, ал кодтың басқа бөліктері үшін қолжетімсіз класс мүшелерін құру үшін ғана пайдаланған дұрыс. Егер кодтың кластар иерархиясынан тыс бөлігінде жабық мүшелердің мәнін тағайындау мүмкіндігін қарастыру қажет болса, ондай мүшелерге қатынасуды қасиеттердің көмегімен ұйымдастырған жөн.

Мұралаудағы конструкторлар

Кластар иерархиясында базалық және туынды кластардың өзіндік конструкторларын құру мүмкіндігі бар. Туынды класс үшін жеке конструктор құрған жағдайда базалық класс конструкторы объектінің базалық бөлігін, ал туынды класс конструкторы осы объектінің туынды бөлігін қалыптастыратын болады.

Базалық және туынды кластың екеуінде де конструкторлар анықталған жағдайда, туынды класс конструкторы **base** кілттік сөзінің көмегімен құрылады. Туынды класс конструкторын жариялаудың кеңейтілген формасын және **base** кілттік сөзін пайдалану арқылы туынды класта базалық класс конструкторын шақыруға болады. Конструкторды жариялаудың кеңейтілген формасы төменде көрсетілген:

```

    туынды_класс_конструкторы(параметрлер_тізімі)           :           base
(аргументтер_тізімі)
{
    // конструктор тұлғасы
}

```

мұндағы *аргументтер_тізімі* базалық класс конструкторына қажетті кез келген аргументтерді білдіреді. Бұл мүмкіндікті пайдалану ерекшеліктерін мысал арқылы қарастырайық.

4-мысал. Өнім және тауар кластар иерархиясында конструкторларды пайдалану

```

using System;
// Өнімдерді сипаттауға арналған класс
class Onim {
    private string aty;
    private string ondirushi;
    private double bagasy;
    public string Aty
    {
        get { return aty; }
        set { aty = value; }
    }
}

```

```

    }
    public string Ondirushi
    {
        get { return ondirushi;}
        set { ondirushi = value;}
    }
    public double Bagasy
    {
        get { return bagasy;}
        set { if(value > 0) bagasy = value;
            else bagasy = 0;}
    }
    public Onim(string a, string o, double b)
    {
        aty = a; ondirushi = o, bagasy = b;
    }
    public void Shygaru()
    {
        Console.Write(ondirushi + " ondirushisining " + aty + "
onimining bagasy - " + bagasy);
    }
}
// Өнім класынан туынды тауар класы
class Tauar : Onim {
    private int sany;
    public int Sany
    {
        get { return sany;}
        set { if(value > 0) sany = value;
            else sany = 0;}
    }
    public Tauar(string a, string o, double b,
        int s) : base(a, o, b)
    {
        sany = s;
    }
    // тауар туралы ақпаратты шығару
    new public void Shygaru()
    {
        base.Shygaru(); Console.WriteLine(", al sany - " + sany);
    }
    // Берілген тауардың жалпы бағасы
    public void zhalpy_baga()
    {
        Console.WriteLine("Tauardying zhalpy bagasy: "
            + Bagasy * sany);
    }
}
class Program {
    static void Main() {
        Onim onim001 = new Onim("notebook", "Acer", 200000);
        onim001.Shygaru(); Console.WriteLine();
        Tauar tauar001 = new Tauar("notebook", "Acer", 200000, 15);
        tauar001.Shygaru();
        tauar001.zhalpy_baga();
        Console.ReadKey();
    }
}

```

Бұл программаның орындалу нәтижесі төмендегідей болады:

```

    Acer ondirushisining notebook onimining bagasy - 200000
    Acer ondirushisining notebook onimining bagasy - 200000,
al sany - 15

```

Tauardyngh zhalpy bagasy: 3000000

Көрсетілген мысалда **Tauar()** конструкторы **a, o, b** параметрлері бар **base** әдісін шақырады. Бұл әрекет **Aty, Ondirushi, Bagasy** қасиеттерін **a, o, b** параметрлерінің мәндерімен инициалдайтын **Onim()** конструкторының шақырылуына алып келеді.

base кілттік сөзінің көмегімен базалық класта анықталған кез келген конструктор түрін шақыруға болады, бұл жағдайда берілген аргументтерге сәйкес келетін параметрлері бар конструктор шақырылады.

Сонымен, конструкторларды төмендегідей анықтауға болады:

<i>Базалық конструкторы</i>	<i>класс</i>	<i>Туынды класс конструкторы</i>
<code>public Onim() { aty = " ", ondirushi = " ", bagasy = 0; }</code>		<code>public Tauar() { sany = 0; } /* автоматты түрде келісім бойынша конструктор шақырылады */</code>
<code>public Onim(string a) { aty = a, ondirushi = " ", bagasy = 0; }</code>		<code>public Tauar(string a, int s): base(a) { sany = s; }</code>
<code>public Onim(string a) { aty = a, ondirushi = " ", bagasy = 0; }</code>		<code>public Tauar(string a): base(a) { sany = 0; }</code>
<code>public Onim(string a, string o, double b) { aty = a, ondirushi = o, bagasy = b; }</code>		<code>public Tauar(string a, string o, double b, int s): base(a, o, b) { sany = s; }</code>

base кілттік сөзі көпдеңгейлі иерархияда шақырушы кластан бір саты жоғары тұрған базалық класс конструкторын шақырады. Егер **base** кілттік сөзі көрсетілмесе, базалық кластың келісім бойынша конструкторы шақырылады.

Видеожазбалар:

Мұрагерлік https://youtu.be/23Q1dkZk_F8?si=4ufDmGKDddlPbswB

9 дәріс. Виртуалды әдістер және оларды қайтаанықтау. Абстрактілі кластар.

Дәрістің мақсаты: студенттерде виртуалды әдістер мен абстрактілі кластарды пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Виртуалды әдістердің қызметі бойынша түсініктерін көрсету;
- Әдісті қайта анықтау механизмі туралы түсініктерін көрсету;
- Абстрактілі кластарды пайдалану ерекшеліктері бойынша түсініктерін көрсету.

Виртуалды әдіс деп базалық класта **virtual** ретінде жарияланатын және туынды кластарда қайта анықталуы мүмкін әдісті атайды. Ендеше, әрбір туынды класта виртуалды әдістің өзіндік нұсқасы болуы мүмкін. C# тілінде виртуалды әдісті шақыру кезінде сілтеме бойынша шақыру орындалып жатқан объектінің типіне байланысты виртуалды әдістің қай нұсқасы шақырылуы керектігі анықталады. Яғни, виртуалды әдістің орындалатын нұсқасы объектіге сілтеменің типіне емес, объектінің өз типіне байланысты таңдалады. Осылайша, егер базалық класта виртуалды әдіс анықталған болса, базалық класқа сілтеме арқылы әртүрлі типтегі объектілерге қатынасу кезінде осы виртуалды әдістің әртүрлі нұсқалары орындалады.

Виртуалды әдіс базалық класта **virtual** кілттік сөзінің көмегімен жарияланады. Туынды класта виртуалды әдісті қайта анықтау **override** модификаторының көмегімен орындалады. Әдісті қайта анықтау кезінде оның аты, қайтаратын типі және сигнатурасы дәл базалық кластағы нұсқамен сәйкес келуі тиіс.

І-мысал. Виртуалды әдістерді құру және қайта анықтау

```
using System;
class Onim {
    public virtual void Shygaru()
    {
        Console.WriteLine("Onim klasymen zhumys");
    }
}
class Tauar : Onim {
    public override void Shygaru()
    {
        Console.WriteLine("Tauar klasymen zhumys");
    }
    // Берілген тауардың жалпы бағасы
}
class Program {
    static void Main() {
        Onim onim001 = new Onim();
        Tauar tauar001 = new Tauar();
        Onim Silteme;
        Silteme = onim001; Silteme.Shygaru();
        Silteme = tauar001;
        Silteme.Shygaru();
        Console.ReadKey();
    }
}
```

Бұл программаның орындалу нәтижесі келесідей болады:

Onim klasymen zhumys Tauar klasymen zhumys

Мысалда берілген **Shygaru()** виртуалдыәдісі туынды класта қайтаанықталған. **Program** класының ішінде базалық кластың **silteme** сілтемелік типті айнымалысына алдымен базалық класс типіндегі объектіге сілтеме меншіктеліп, **Shygaru()** әдісі шақырылады, содан кейін **silteme** айнымалысына туынды класс типіндегі объектіге сілтеме меншіктеледі, бұл жағдайда да **Shygaru()** әдісі шақырылады. **Shygaru()** әдісін бірінші шақырған кезде базалық класта анықталған виртуалды әдіс, ал екінші шақыру кезінде туынды класта қайта анықталған әдіс орындалады.

Виртуалды әдісті туынды класта қайта анықтау міндетті емес, бұл жағдайда туынды класс типіндегі объект арқылы базалық класта анықталған әдіс нұсқасы шақырылады. Әдістерді қайта анықтаудың көмегімен C# тілінде полиморфизм принципі қолдау табады.

Абстракттілікластар

Кейбір жағдайларда барлық туынды кластар үшін барынша жалпы формасы ғана анықталатын базалық класты құру қажет болады, мұндай кластың мазмұндылығын толықтыру туынды кластардың еншісіне беріледі. Базалық класс мазмұны нақтыболмаған жағдайда оның құрамында анықталған әдістердің туынды кластарда міндетті түрде қайта анықталғаны жөн. Бұл әрекет абстрактылы әдістердіпайдалану арқылы орындалады.

Абстрактылы әдіс abstract типіндегі модификатор көмегімен құрылады. Абстрактылы әдістің тұлғасы болмайды, сондықтан ол базалық класта жүзеге асырылмайды және туынды класта міндетті түрде қайта анықталу керек. Абстрактылы әдіс автоматты түрде виртуалды болып саналады.

Абстрактылы әдісті анықтау формасы төмендегідей:

```
abstract тип аты {параметрлер_тізімі};
```

Көріп отырғандарыңыздай, абстрактылы әдістің тұлғасы болмайды. Құрамында бір немесе бірнеше абстрактылы әдіс анықталған класс та абстрактылы болып жариялануы тиіс, ол үшін класс жариялануындағы **class** кілттік сөзінің алдында **abstract** модификаторы көрсетіледі. Абстрактылы кластың қызметі толық анықталмайтын болғандықтан, оның объектілерін құру мүмкін емес.

Абстрактылы кластан мұралайтын туынды класта базалық кластың барлық абстрактылы әдістері қайта анықталуы міндетті. Олай болмаған жағдайдатуынды класс та абстрактылы болып жариялануы керек.

2-мысал. Абстрактылыкласс мысалы

```
using System;
abstract class Progressiya {
    int a0;
    int qadam;
    // Келісім бойынша конструктор
    public Progressiya()
    {
        a0 = 1; qadam = 2;
    }
    // Параметрленген конструктор
    public Progressiya(int a, int q)
    {
        a0 = a; qadam = q;
    }
    // Қасиеттер
    public int A0
    {
        get; set;
    }
    public int Qadam
    {
        get { return qadam; }
        set { if (value > 0) qadam = value;

```

```

        else qadam = 1; }
    }
    public void Shygaru()
    {
        Console.WriteLine("Progressiyaning algashqy elementi - "
            + a0 + " , qadamy - " + qadam);
    }
    public abstract void Tip();
    // прогрессияның n-ші мүшесінің мәні
    public abstract double an(int n);
    // прогрессияның n-шы қосындысы
    public abstract double qosyndy(int n);
}
// Арифметикалық прогрессия туынды класы
class Arifm : Progressiya {
    public Arifm() { }
    public Arifm(int a, int q) : base(a, q) { }
    // Класс үшін n-мүшесінің мәнін есептеу әдісін қайта анықтау
    public override double an(int n)
    {
        return A0 + n * Qadam;
    }
    public override double qosyndy(int n)
    {
        return (n + 1) * (2*A0 + n * Qadam) / 2;
    }
    // Прогрессия түрін көрсету
    public override void Tip()
    {
        Console.WriteLine(" , arifmetikalyq progressiya ");
    }
}
// Геометриялық прогрессия туынды класы
class Geom : Progressiya {
    public Geom() { }
    public Geom(int a, int q) : base(a, q) { }
    // Класс үшін n-мүшесінің мәнін есептеу әдісін қайта анықтау
    public override double an(int n)
    {
        return A0 * Math.Pow(Qadam, n);
    }
    public override double qosyndy(int n)
    {
        double An = an(n);
        return (A0 - An * n) / (1 - n);
    }
    // Прогрессия түрін көрсету
    public override void Tip()
    {
        Console.WriteLine(" , geometriyalyq progressiya ");
    }
}
class Program {
    static void Main() {
        Progressiya[] pro = new Progressiya[4];
        int n = 5;
        pro[0] = new Arifm(1, 5); pro[1] = new Arifm();
        pro[2] = new Geom(3, 4); pro[3] = new Geom();
        for (int i = 0; i < pro.Length; i++)
        {
            pro[i].Shygaru(); pro[i].Tip();
            Console.WriteLine("Progressiyaning " + n + "-
mushesining mani = " + pro[i].an(n) + " , a1 " + n
+ "-qosyndysyning mani = " + pro[i].qosyndy(n));

```

```

    }
    Console.ReadKey();
}
}

```

Программаның нәтижесі:

```

    Progressiysnyng algashqy elementi - 1, qadamy - 5,
    arifmetikalyq progressiya
    Progressiyaning 5-mushesining mani = 25, al 5-
    qosyndysynyng mani = 75
    Progressiysnyng algashqy elementi - 1, qadamy - 2,
    arifmetikalyq progressiya
    Progressiyaning 5-mushesining mani = 10, al 5-
    qosyndysynyng mani = 30
    Progressiysnyng algashqy elementi - 3, qadamy - 4,
    geometriyalyq progressiya
    Progressiyaning 5-mushesining mani = 3072, al 5-
    qosyndysynyng mani = 3840
    Progressiysnyng algashqy elementi - 1, qadamy - 2,
    geometriyalyq progressiya
    Progressiyaning 5-mushesining mani = 32, al 5-
    qosyndysynyng mani = 40

```

Осылайша, абстрактылы кластың барынша пайдалы қасиеттерінің бірі – кез келген туындыкластарының объектілерімен цикл құрамында жұмыс жасауға мүмкіндік беруі.

Видеожазбалар

Абстракттілі класстар <https://youtu.be/RYu7jYb5SYo?si=J911cGTsv7MGEaWQ>

10 дәріс. Интерфейстер. Интерфейстік сілтемелер, интерфейстік қасиеттер мен индексторлар. Интерфейстерді мұралау.

Дәрістің мақсаты: студенттерде интерфейстермен жұмыс істеу ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Интерфейстерді абстрактілі кластардан ажырату;
- Интерфейсті мұралаудың ерекшеліктері туралы түсініктерін көрсету.

Объектіге бағытталған программалауда класс қызметті қалай орындау керектігін емес, нені атқару керектігін анықтау керек болады. Оның үстіне бұрын абстракты әдісі атанған үлгі бола алады.

Демек, бірақ оны іске асыру әдісі Қайтаратын түрі айқындалады және қолтаңба әдісін берілмейді.

Ал туынды әдіс, оның әрбір сыныптағы өзінікі-дағы абстрактілі белгілі бір базалық сыныпта іске асыру қамтамасыз етілуге тиіс. Осылайша, абстракты әдісі интерфейс, бірақ ол әдісін іске асыруды айқындайды. Әрине, абстрактілі сыныптар мен әдістері пайда болғаны белгілі, бірақ олардың негізіне салынған мүмкін принципі жақсы дамыған әрі. Жаңа кілт сөз көмегімен бөлу C # және оны іске асыру көзделген интерфейс сыныпты interface.

Абстрактілі интерфейстер синтаксистік тұрғысынан болып сыныптары. Бірақ бар интерфейске әдістерінің бірін де дене болмауы тиіс. Бұл дегеніңіз жаңа интерфейске ешқандай қатысы жоқ іске асыру беріледі. Атап айтқанда сол ғана көрсетіледі, бірақ ол онда қалай істеу керек болады. Интерфейсі сатылуы мүмкін болысымен айқындалатын болады, ол кез келген саны сынып бар. Бұдан басқа, кез келген саны бір сыныпта интерфейстерді іске асырылуы мүмкін.

Интерфейс (яғни нақты өткізу) дененің осы интерфейске сипатталған әдістерді сыныпта іске асыру үшін берілуге тиіс. Іске асыру айқындау үшін әрбір класқа өз интерфейс бөлшектердің толық еркіндік беріледі. Демек, екі бірдей интерфейс сатылуы мүмкін сыныптарында әрқилы. Дегенмен олардың әрқайсысында бірдей әдістерін жиыны осы интерфейсінң ұстануы тиіс. Ал оның қайда қалады, себебі осы объектілердің пайдаланылуы мүмкін кез келген осындай интерфейс объектілері осы екі менеджеріне арналған сынып коды, белгілі барлық біркелкі. C # бас интерфейстері қолдауы арқасында мүмкін полиморфизмді басты принцип толық көлемде іске асырылды: көп әдістерін бір интерфейс.

Кілт сөз interface интерфейстер көмегімен жарияланады. Хабарландыру төмен оңайлатылған нысаны келтірілген интерфейс.

```
interface аты {  
  
    қайтарылатын_тип әдіс_аты1 (параметрлер_тізімі);  
  
    қайтарылатын_тип әдіс_аты2 (параметрлер_тізімі);  
}
```

//.

қайтарылатын_типәдіс_атыN (параметрлер_тізімі);

}

Онда аты -бұл нақтыи - фейсатауы. Интерфейсәдістерінғанапайдаланылады жәнеоларды возвращаемый_тип хабарландыруда қолтаңба. Олар, шын мәнінде, абстрактілі әдістермен болып табылады. Жоғарыда, ешқандай интерфейске түсіндірді ретінде іске асыру мүмкін емес. Сондықтан әр сыныпта өзіне қосатын тиіс барлық әдістерді интерфейсінң іске асырылды, бұл интерфейс. Сол интерфейске анықталмағанашық деп есептеледі, сондықтан да көрсету керек емес әдістері өзінде оларға қатынасу анық.

Хабарландыру сыныбына арналған бірқатар мысал келтірілген сандар, генерациялайтын тізбекті интерфейс төмен.

```
public interface ISeries {
```

```
int GetNextO;void қалпына келтіру тәртібі бойынша қайтаруға
```

```
// келесі саны ();
```

```
// қайта іске қосу
```

```
void SetStart (int x); // бастапқы мәнді орнату
```

```
}
```

Интерфейске ISeries осы аты беріледі. I префиксі көрсетуге міндетті емес, бірақ бұл үшін интерфейсдер ажырата білу керек тәжірибеде қабылданған бағдарламалау интерфейсінң атауында бір жылғы сынып бар. Сондықтан сыныптағы сатылуы мүмкін кез келген public ретінде жарияланады интерфейсі ISeries қандай болса да бағдарламалары.

Қасиеттері, сонымен қатар, басқа әдістерін интерфейсдерге көрсетуге индекслаторы мен оқиғалар. Мүшелері интерфейсдер деректер болмауы керек. Конструкторлар болмайды, сондай-ақ оларда деструкторы немесе операторные анықтауға болады. Бұдан басқа, static интерфейс мүшелерінің бірі ретінде жариялануы мүмкін де емес.

Интерфейстердііскеасыру

Қалайғанабелгіліболады,олбірнемесебірнешесыныптарыинтерфейсісатылуымүмкін. ИнтерфейсіскеасыруушіноныңатынкөрсетужеткіліктібазалықСыныпатыныңкейін, осыған ұқсас сыныбы. Интерфейс төмен жалпы сыныпта іске асырудың нысаны келтірілген.

```
classмя_класса:имя_мнтерфемса{
```

```
Денесі//сынып
```

```
}
```

Ондаимя_интерфейсасатылатыннақтыаты-бұл интерфейс. Егер тіптіинтерфейсі сыныпта іске асырылады, онда бұл толық жасалуға тиіс. Атап айтқанда, ішінара интерфейсііскеасыруғажәнетекбөліп-бөліпболмайды.

Сыныпта бірнеше интерфейсдерді іске асыруға жол беріледі. Бұл жағдайда барлық өткізетін интерфейсдер тізімін сыныпта үтір арқылы көрсетіледі. Мұрагер болады

базалық класы және солауқытта бірнеше одаңа көп сыныпта іске асыруға интерфейс. Мұндай жағдайда аты көрсетілуге тиіс, базалық сыныпты тізімін бөлетін үтіралдында интерфейстері.

Интерфейс, *public* ретінде іске асыратын әдістері жариялануға тиіс. Мәселесінде, сондай-ақ осы әдістер интерфейске анықталмаған жобаланып жатыр, сондықтан оларды іске асыру ретінде өзінде ашық ашық болуы тиіс. Бұдан басқа, дәл сәйкес келуі тиіс және қолтаңба үлгісі мен сигнатураға возвращаемому интерфейс айқындау әдісін сатылатын қайтаратын түрі көрсетілген.

Бұрын ұсынылған бағдарламалар интерфейсін іске асырылып жатқан төмен мысал келтірілген *ISeries*. Бұл бағдарламада *ByTwos* бірізді жинайтын класы құрылады, сандар, онда бір қатар кейін жасалған әрбір көп санды екіге өткен.

```
//ISeries интерфейсін іске асыруға, сынып ByTwos: {Int, series, start; VALINT;
```

```
public ByTwos() {
    start = 0; val = 0;
}
public {val, int, GetNext() = 2; return val;
}
public void {қалпына келтіру(
    val = start;
}
public void SetStart (int x) {start = x; val = start;
```

Көріп отырғаныңыздай, *class* бойы *ByTwos* *ISeries* интерфейске айқындалатын үш әдісін іске асырылады. Тура келеді, себебі, бұл жоғары сыныптар ретінде түсіндірді интерфейсін ішінара сатуға болмайды.

Сынып коды, онда келтірілген *ByTwos* *ISeries* интерфейсін іске асырушы сынып қолдану төмен көрсетіп жатыр.

```
Сынып қолдану ByTwos // көрсетуге сатылатын интерфейс, using System;
```

```
{ {static class SeriesDemo void Main(
    ByTwos ob = new ByTwos(); /
    for (int i = 0; i < 5; i)
        Console.WriteLine("келесі саны тең" + ob.GetNext());
    Console.WriteLine("Xn Сбросить"); ob.Қалпына келтіру();
    for (int i = 0; i < 5; i)
        Console.WriteLine("келесі саны тең" + ob.GetNext());
    Console.WriteLine("Xn Начать с любого значения 100");
    ob.SetStart(100);
```

```
for(int i=0; i<5; i)
    Console.WriteLine ("келесі санымен"ob.GetNext());
}
```

Интерфейстік сілтемелерді қолдану

Айнымалы сілтеме түрін болса да бір түрлі ерсі көрінсе, бірақ жариялауға жол беріледі, яғни C# интерфейсноғайнымалы мәнді сілтемелер интерфейс. Мұндай айнымалы сілтеме жасауы мүмкін кез келген объект, оның іске асыратын интерфейс. Нысан үшін оның нұсқасы осы нысанның интерфейсноысыныпта сілтеме арқылы өткізілгенәдісін шақырған кезде орындалады.

Төменде келтірілген сақталуы тиіс бағдарламасын қолдану арқылы көрсетіп жатыр интерфейсной сілтемелер. Бұл бағдарламада айнымалы сілтемелер шақыру мақсатында объектілер үшін - екі сыныпты ByTwos және Primes интерфейсi әдістері пайдаланылады. Осы мысалда жиналған, айқын көрсетіледі және барлық бағдарламасының бөлігі үшін бірыңғай файл.

Көрсету интерфейсны // URL - using System;

Интерфейс, public interface ISeries// {анықтау

intGetNext(); void қалпына келтіру тәртібі бойыншақайтаруға// келесі саны (); // қайта іске қосу

void SetStart (int x); // бастапқы мәнді орнату

}

// // ISeries процестерін жүзеге асыру үшін бірқатар жүйелі түрде, онда көп санды екіге алдыңғыкейіннен сандардың әрбір // интерфейсi пайдалану өндіруге, сынып ByTwos: {Int iseries start; VAL INT;

val = start;

}

public voidSetStart (int x) {start = x; val = start;

}

}

// // Интерфейсі іске асыру үшін пайдалануға ISeries өндіругепроцесін жай сандар.

class Primes: {Int iseries start; VAL INT;

public Primes () {

start = 2; val = 2;

}

public int GetNextO {int i, j; bool isprime;

```

val;
for (i = val; i < 1 000 000; i) {
isprime = true; for (j = 2; i / <= j j; j) {
if ((i % j) = 0) {isprime = false; break;
}
}
if (isprime) {val = i; break;
}
}
return val;
}
public void{қалпына келтіру (
val = start;
}
public voidSetStart (int x) {start = x; val = start;
}
}
{{static class SeriesDemo2 voidMain (
ByTwostwoOb = new ByTwos();
PrimesprimeOb = new Primes ();
ISeriesob;
for (int i = 0; i < 5; i) {
ob = twoOb;
Console.WriteLine ("келесі жұп сан сияқты" ob. GetNext ()); ob = primeOb;
Console.WriteLine ("келесі жай сан" "сияқты" ob. GetNext ());

```

Міне, осы бағдарламаның орындалуын қай нәтижеге алып келеді:

Келесі жұп сан сияқты жай сан - 4 - 5 жұп санды жай сан сияқты келесі 6 жұп сан сияқты 3
2 келесі сияқты келесі келесі жұп санды жай сан сияқты келесі жұп сан 10-ға тең 8 7 келесі
келесі жай сан сияқты сияқты сияқты келесі келесі жай сан сияқты 13 11

Main () айнымалы сілтеме жасау үшін ISeries интерфейсі ob дісінде жарияланады. Ол дегеніңіз, онда сілтеме нысан сақталуы мүмкін кез келгентаптың интерфейсі іске асырушы ISeries. Бұл жағдайда ол мыналарға primeOb сыныптарын, оларда ISeries Primes ByTwos twoOb объектілері және сілтеме жасау үшін және тиісінше интерфейсі іске асырылды.

Интерфейстерді мұралау

Интерфейсін алу мүмкін бір басқа. Синтаксис бар осындай мұрагерлік интерфейстерді бірдей сынып бар. Бір кезде іске асырылады, онда барлық мүшелері екінші сыныпта интерфейс, наследующий тиіс сияқты мұрагерлік төменде келтірілген мысалда тізбекте айқындалған интерфейстерді іске асырылды.

Мысалы, мұрагерлік интерфейстерді // using System;

```
public interface IA {void Meth1 (); void Meth2 ();  
}
```

Енгізілген Meth1 Meth2 // жаңа әдістері () () және базалық интерфейсi

Ал тағы бір әдіс - туынды интерфейсi // қосылды Meth3 ().

```
public interface кеңейту: Void iA {Meth3 ();  
}
```

Осы класс // тиіс барлық әдістерді интерфейстерді іске асырылды IA және кеңейту.

```
class MyClass: { {Public void Meth1 кеңейту (  
Console.WriteLine ("іске асыруға Meth1 әдісі (.");  
}  
public void {Meth2 (  
Console.WriteLine ("іске асыруға Meth2 әдісі (.");  
}  
public void {Meth3 (  
Console.WriteLine ("іске асыруға Meth3 әдісі (.");  
}  
}
```

```
class IFExtend {  
static {void Main (  
MyClass ob = new MyClass ();  
ob.Meth1 (); ob. Meth2 (); ob. M ^ th3 ();
```

Мүдде үшін сынып бөлмесінен MyClass Meth1 әдісін () іске асыруға жойып көріңіз. Құрастыру кезінде әкеп соғатын болса, бұл қате. Бұдан бұрын, түсіндірді ретінде айқындалған, оның ішінде барлық әдістерді іске асырылды, бұл ретте сыныптағы тиіс кез келген, өткізетін интерфейсi бар және олар интерфейске другцх интерфейстерді ішінен мұраға қалдырылады..

Видеожазбалар:

Интерфейс https://youtu.be/nJN1grbmUco?si=vSGmkKLAyR_sTD8S

11 дәріс. Аластамалық жағдайларды өңдеу. Аластамаларды ұстамаудың салдары. Бірнеше catch операторларын пайдалану.

Дәрістің мақсаты: студенттерде аластамалық жағдайлардың типтері мен оларды өңдеу блоктарының қызметі туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Аластамалық жағдайлардың типтерін ажырату;
- Аластамалық жағдайларды өңдеу блоктарын ажырату.

Аластамалық жағдай, немесе жай ғана аластама, программаның атқарылуы барысында туындайды. Бағдарламаны орындау кезінде туындайтын қателіктерді жіктелген түрде #, өңдеуге және бақыланатын ауылындағы ерекше жағдайлардың пайдалана отырып өңдеу қосымша жүйесі болады. Ерекше жағдайлардың басты артықшылығы мынада, ол бұрын енгізуге тура келді, ол көп бөлігі алу кодын өңдеу бағдарламасын қателерді өңдеу үшін кез келген ірі қолмен автоматтандыруға мүмкіндік береді. Мәселен, егер бағдарламалау тілінде жазылып, онда ерекше жағдайлардың орындау сәтсіз болды, олар қолмен тексеру қажет болған кезде өңдеусіз бағдарламасы қате кодтары әдістерін әр кезде әдісін шақыру.

Еңбекті көп қажет ететін және қолайсыздыққа ұшырататын, бірақ бұл тек қателерге процесс. Қателерді пысықтау ұтымды етіп жатыр деп аталатын ерекше жағдайлардың бүкіл процесін автоматты түрде анықтауға мүмкіндік болған және выполняющийся ерекшеліктер өңдегіші код блогы өңдеу бағдарламасы қате пайда болды. Бұл қаншалықты сәтті немесе нақты әрекет сәтсіз аяқталды не қолмен тексеру әдісін шақыру қажеттігінен құтқарады. Егер қате туындаса, ол тиісті түрде Өңделген қателерді өңдегіші болады.

Ерекше маңызды, себебі мұнда C # стандартты әдеттегі қателерді көрсеткішке бөлу үшін қиыс жағдайлар анықталған жағдайлардың тағы өңдеу бағдарламалық, мысалы жиым шегінен индекс нөл немесе шығуы. Ден қою үшін осындай қателер қадағалау ж не тиісті ұйымдастырылуы тиіс бағдарламасында өңдеу

ерекше болады. Ақыр соңында C# шебер пайдалануға арналған ішкі жүйесі ерекше жағдайлардың өңдеу үшін бағдарламалау ғой табысты үйрену керек.

System.Exception класы

Жаңа C # алып тастау түрінде табыс етілуі сынып бар. C # сыныпты аттар кеңістігінің бөлігі болып табылатын туынды тиіс барлық кластар алып тастаулар жылғы келеді Exception System. Олай болса, барлық Exception класс тармағылармен сыныпты шығару болып табылады.

Ең маңызды Exception SystemException сыныбы қосалқы қатарына жатады. Дәл осыдан сыныпты шығархатын барлық алып тастаудың орындаушы C # жүйесімен туынды болып табылады (яғни жүйесімен CLR). Стандартты алып тастаулар иерархияны ұшар басына шыға келді, жай ғана Exception ештеңе сыныбы қойылатын SystemException сыныбы мүмкіндіктерін айқындайды.

Аластамалық жағдайлардың өңдеу негіздері

C # ерекше жағдайлардың өңдеу төрт кілт сөз көмегімен ұйымдастырылады: Finally, catch try, throw. Кілт сөз қолдану құрайды, онда олар өзара байланысты қосымша жүйесі бірінің қолдану білдіреді. Жоғарыда айтылған барлық кілт сөз қолдану осы тараудың өн бойына тағайындау және әрқайсысының егжей-тегжейлі қарауға болады. Бірақ алдымен олардың әрқайсысының ерекше жағдайлардың өңдеу рөлі туралы жалпы көріністі беру қажет. Сондықтан қысқа олардың іс-әрекеті принципі төменде сипатталған.

Бағдарлама операторлары бақылауға қажет ететін блогы пайда болуына алып тастаулар try жасалады. Егер айрықша жағдай туындайды, try блогының ішінде дайындайды қиыс жағдай: Бұл бағдарламаның кодында catch таңбаланушы кілт сөзбен ұтымды тәсілмен және бір оператордың көмегімен ұсталған алып тастау мүмкін өңделді. Жүйе деңгейіндегі туындайтын генерируются жүйемен автоматты түрде алып тастау, орындаушы. Ал қолмен throw өндіруге алып тастаулар кілт сөз үшін қызмет етеді. Міндетті түрде орындалуға тиіс кез келген жіберілген кодты қатарынан шыққан соң try finally блогы блок орналастырылады.

try catch кілттік сөздерін қолдану

C # саны бу try және кілт сөз ерекше жағдайлардың өңдеу негізін catch. Бұл кілт сөздерді бөлек қолданылады және бірлесіп пайдалануға болмайды. Нобай try / catch ерекше жағдайлардың блоктарын айқындау өңдеу үшін төменде келтірілген:

```
try {
```

```
Тексерілетін қателерге // код блогы.
```

```
} catch (Exception exOb
```

```
// Exception түрін шығару өңдегіші. }
```

```
catch (Exception2 exOb) {
```

```
// Exception2.} түрін шығару өңдегіші
```

онда пайда болатын түрі Exception - бұл ерекше жағдай. TRY дайындайды, ол кезде оған құрайтын екі оператор ұстап қалып жатыр, ол содан соң бұл catch шығару операторы өңдейді қиыс жағдай: Болдырмау орындалуда және түріне байланысты тиісті оператор catch. Мәселен, егер шығархатын түрлері көрсетіледі, онда дәл осы орындалады, ал қалған операторы бірдей екендігін catch ерекшеліктер және операторы өткізіледі. Шығару, ауыспалы exOb шығарылған кезде өзінің мағынасын алады ұстап қалып жатыр.

Шын мәнінде айнымалы exOb көрсетуге міндетті емес. Сөйтіп, оның нысанына қатынасты көрсетуге міндетті емес, егер бұл қажет болатын алып тастаудың фирмалардың өңдеушіге жиі кездеседі. Оның үлгісіне шығару және өңдеу үшін жеткілікті. Сондықтан да бұл тарауда келтірілген көптеген мысалдарды бағдарламалар, ауыспалы exOb түсіріледі.

Алайда, егер алып тастау керек емес екенін назарда ұстаған дайындайды, онда әдеттегідей, мен оның барлық try операторының блогы аяқталады catch операторлары өткізіледі. Ақордада catch операторының кейінгі операторының бағдарламаны орындауға және бірінші қайта басталады.

Мысал.

```
using System;
class ExcDemol {
static {void Main (
[] = new int int nums [4];
try {
Console.WriteLine ("өндіругедейін қиыс жағдайлар.");
Алып тастау шегінен шығуына байланысты // шығару индексінің жиым.
for (int i = 0; i < 10; i) {
nums [i] = i;
Console.WriteLine ("nums [{0}]: {1} ", i, nums[i]);
}
Console.WriteLine ("Не деген жатады");
}
catch (IndexOutOfRangeException) {
Ұмтылуға // қиыс жағдай:
Console.WriteLine ("индексі шегінен шықты жиым!");
}
Console.WriteLine ("Блок кейін ұстап қиыс жағдайлар.");
}
}
```

Бұл бағдарламаны орындау кезінде келесі нәтиже болып шығады.

Өндіругедейін қиыс жағдайлар.

nums [0]: ТУРАЛЫ

nums [1]: 1

nums [2]: 2

nums [3]: 3

Массив индексі шегінен шықты!

Блок кейін ұстап қиыс жағдайлар.

Осы мысалда жиым түрдегі тұрады nums int төрт элемент. Бірақ бұл кезде пайда болуына алып келеді, бұл талпынысы циклінде 0-ден 5-ке IndexOutOfRangeException болып элементіне үндеу шығару және 9 for индекстеу Жиым Жиым индексі бойынша 4.

Аластамаларды ұстамаудың салдары

Стандарттық ерекшеліктерге жоғарыдакелтірілген мысалдарды да бірінің ұстап қалу, тағы бір артықшылық береді: Ол авариялық бағдарламаны аяқтау жоққа шығарады. Кодтықалай ғана болады, ол ұсталған тиіс белгілі бір жерде бір ерекшелік сгенерировано бөлікпен бағдарламалары. Жалпы айтқанда, егер ұстап қалып жатыр, онда ол ұсталған болады емес бағдарламасындашығару орындаушы ойластырды. Мәселесонда, қате туралы хабар береді және бағдарламаны орындауға жүйесі атқарушы үзеді. Мәселен, төменде келтірілген бағдарламасын мысалға алып тастау шегінен шығуына байланысты емес индекс жиымның ұстап қалып жатыр.

C # өңдеуге мүмкіндік беруге // орындаушыжүйесі ең қате орын алды.

```
using System;
```

```
{ {static class NotHandled void Main (
```

```
[] = new int int nums [4];
```

```
Console.WriteLine ("өндіругедейін қиыс жағдайлар.");
```

```
// Шығару индексінің жиым шегінен шығуына байланысты алып тастау
```

```
for (int i = 0; i < 10; i) {
```

```
    nums [i] = i;
```

```
    Console.WriteLine ("nums [{0}]: {1} ", i, nums[i]);
```

Жиым үзіліп қате туралы осы хабарды беріледі бағдарламаны орындауға индекстеу кезде қате пайда болды.

Өңделмеген қиыс жағдай: System.IndexOutOfRangeException:

Болды, NotHandled.Main< имя_файла > жол 16 () шекарасынан тыс Массив индексі

Бұл хабар NotHandled дісінде табылғандығы туралы хабардар етеді. Main () өңделмеген қиыс жағдайлар үлгідегі System. Индекс indexoutofrangeexception, ол шегінен шығуына байланыстыжиым.

Мұндай қате туралы хабарлар жөндеу үшін пайдалы, бірақ іс жүзінде аз дәрежеде бағдарламасын пайдаланған кезде оның жағымсыз! Сондықтан да ең маңызды ерекше жағдайлардың өңдеуді ұйымдастыруғанақтыланады.

Catch бірнеше операторын қолдану

Catch try бірнеше операторларға бір оператор байланыстыруға болады. Бұл іс және іс жүзінде жиі кездеседі. Бірақ барлығы әртүрлі типті шығару catch операторлары жолдан ұстап алу тиіс. Төменде келтірілген мысал ретінде, онда мен таңбалар шегінен шығу жиымның нөл арналған бағдарламасы қатені ұстап қалып жатыр.

Бірнеше операторлар пайдалануға // catch.

```
using System;
```

```

class ExcDemo4 {
static {void Main (
Мұнда Жиым Жиым ұзынырақ numer // denom.
[]} = {int numer 4, 8, 16, 32, 64, 128, 256, 512;
[]} = {int denom 2, 0, 4, 4, 0, 8;
for (int i = 0; i < numer.Length; i) {
try {
Console.WriteLine (numer [i] "/"
[i] / denom numer denom [i] "сияқты" [i]);
}
catch (DivideByZeroException) {
Console.WriteLine ("арналған бөлунөл болмайды!");
}
catch (IndexOutOfRangeException) {
Console.WriteLine ("қолайлы элемент табылған жоқ.");
}
}
}
}
}
}

```

Міне, осы бағдарламаның орындалуын қай нәтижеге алып келеді.

4/2 сияқты 2

Арналған бөлу нөл болмайды!

4 32/4 сияқты 16/4 сияқты арналған 8 бөлісу нөл болмайды!

128 / 8 сияқты 16 қолайлы элемент табылған жоқ.

Қолайлы элемент табылған жоқ.

Жоғарыда келтірілген әр оператор өз нәтижесін ғана түрі catch кәдімгідей сезініп жатыр бірі қиыс жағдайлар.

Жалпы айтқанда, олардың сапар бағдарламасында catch операторлары тәртібі бойынша орындалады. Бірақ бұл ретте бір блокты catch түріне сәйкес келеді, онда қана орындалады шығару түрі шығархатын қиыс жағдайлар. Ал қалған catch блоктары өткізіледі.

Видеожазбалар:

try/catch <https://youtu.be/a8Re6DwpEVA?si=g4VghA5vshzRtM1I>

12 дәріс. Аластамаларды өзіндік генерациялау. Аластамалардың туынды кластары.

Дәрістің мақсаты: студенттерде аластамалардың туынды кластарын құру ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Аластамаларды өзіндік генерациялау;
- Туынды аластамалар кластарын құру.

Туынды аластама кластарын алу

Кірістірілген қомақты бөлігін қамтитын өзгеретіндігіне қарамастан барынша таралған қателерді болдырмау ерекше жағдайлардың ғана шектелмейді өңдеу бағдарламалық C # Осы қателерге. Шын мәнінде күшті жақтарын C # тәсілдің бірі болып қабылданған ерекше жағдайлардың мынада, сонымен қатар, осы тілді пайдаланушы анықтайтын ерекшеліктерді пайдалануға жол беріледі, яғни өңдеуге қойылатын тұрады кім бастап арналған программалайды #. Атап айтқанда осындай арнайы қателерді өңдеу үшін пайдалануға болады, ал олар өз шығару коды құрылады өте оңай. Бұл үшін жылғы сыныпты анықтауға жеткілікті сыныбы, туынды Exception. Енді бір ғана жұмыс түрлерінің жүйесі ретінде пайдалану үшін оларды іске асыруға міндетті түрде мұндай сыныптарында бірдеңе мүлде жеткілікті ерекшеліктер.

ЕСКЕРТПЕ

Өткен жылғы сыныпты еді, өйткені бұл иерархия сыныптарын алып тастау үшін арнайы құрылған туынды ретінде Application.Exception бастапқыда ерекшеліктер қолданбалы сипаттағы құрсауланған. Бірақ енді бұны істеудің орнына Microsoft корпорациясы, ал осы жылғы сыныпты шығару алуға ұсыным жасамаса, туынды Exception. Дәл осы себеп бойынша осы кітапта бұл тәсіл және қаралады.

Сыныптар құрылатын сипаттары мен әдістері, олар үшін белгілі бір сыныпта Exception алуға және қол жетімді пайдаланушы автоматты түрде жаңартылады. Әрине, кез келген ерекшеліктер осы мүшелердің Exception құрылатын сыныптарда сынып қайта анықтауға болады.

"Қоспау" сыныбы үшін конструкторлар сүйеді, әдетте, онда барлық кезде меншікті құрылады, онда мүмкіндігінше сыныпта айқындалған Exception. Жай арнайы сыныптарында қол жеткізу қиын емес, себебі бұл үшін қолайлы тиісті сынып бере кілт сөзді Exception конструкторға пайдалана отырып, осы ерекшеліктер жеткілікті дәлелдер base. Бірақ іс жүзінде формальды түрде конструкторлар ғана беру керек, олар бағдарламада пайдаланылады.

Мысал қарастырайық, онда арнайы үлгідегі шығару бағдарламасын пайдаланылады. Мысалы, әбден жиым, индекстелетін температура шегінде - 5 27 дейін жарамды болып есептеледі. Ал егер индекс шегінен шығып, онда бұл қатені өңдеу үшін арнайы ауыспалы RangeArray жиымның сыныпта анықталды. Мұндай ауыспалы жүгінген әрбір операциядан кейін кодында массивке использовавшем RangeArray сыныбы, нарықтық және тексерді. Әрине, мұндай көзқарас "икемсіз" және насырға шабатын қателерді өңдеуге қойылатын

қосымша қателерге. Төменде келтірілген сақталуы тиіс жақсартқан сынып RangeArray нұсқадағы қателерді өңдеу шекарасын бұзу

Жиымның ілтипатты астам көмегімен орындалады және сенімді әдіспен арнайы шығархатын қиыс жағдайлар.

Қателерді өңдеу үшін пайдалануға қойылатын арнайы алып тастау / / / массивке сыныпты жүгінген кезде RangeArray.

```
using System;
```

```
Rangearray // сыныбына арналған қиыс жағдай жасау. class RangeArrayException:Exception  
{
```

```
/* Барлық конструкторлар сыныпты іске асыруға Exception. Жай ғана мұндай  
конструкторлар құрастырушы іске асырады базалық қажет.
```

```
Ешқандай қосымша әрекеттерді болдырмау сыныбы RangeArrayException болғандықтан,  
онда ештеңе Exception қосадысыныбы қойылатын талап етілмейді. */
```

```
public RangeArrayException (): base () {}
```

```
public RangeArrayException (string str): base (str) {}
```

```
public RangeArrayException (
```

```
string str, Exceptioninner): base (str, inner) protected RangeArrayException ({}  
System.Runtime.Serialization.SerializationInfo si,
```

```
System.Runtime.Serialization.StreamingContext sc): base (si, sc) {}
```

```
// ToStringO RangeArrayException шығару сыныбы үшін алдын ала анықтау әдісі. public  
override string ToStringO {return Message;
```

```
}  
}
```

```
Жетілдірілген нұсқасы сыныпты // RangeArray. class RangeArray {
```

```
// Жабық деректер.
```

```
int [] a; Сілтеме // базалық жиым int lowerBound; неғұрлым аз индексіint // upperBound; Ең  
көп // индексі
```

```
Автоматты түрде // тек оқуға арналған сипат {int Length Length сатылатын және түсінікті,  
public get; private set; }
```

```
Жиым // жүктелген шарт бойынша салу (int low, int high) {high public RangeArray  
мөлшеріне;
```

```
if (high <= low) {
```

```
New RangeArrayException throw ("жоғарғы жол астылық аз емес.");
```

```
}
```

```
a = new int [high - low];
```

```

Length = high - low;
lowerBound= low; upperBound= - high;
}
Бұл индекстеуші // сыныбына арналған RangeArray. public {int this[int index]
Бұл аксессор // get. get {
if (ok (index)) {
return a [index - lowerBound];
} else
New RangeArrayException throw ("қате шекарасын бұзған.");
}
}
Бұл аксессор // set. set {
if (ok (index)) {
a [index - lowerBound] = value;
}
else throw new RangeArrayException ("қате шекарасын бұзған.");}
}
// // Логикалық true мәнін қайтаруға, егер бекітілген шектерде private bool ok (int index)
орналасқан, {индексі
if (index >= lowerbound & Index <= upperbound)return TRUE; return false;
}
}
{{{Try тыс, сынып RangeArrayDemo static void Main () // // көрсету жиым қолдану өз
бетінше бастап тапсырма берған индекстеу
RangeArray ra = new RangeArray (-5, 5);
RangeArray ra2 = new RangeArray (1, 10);
i
Нысанды пайдалану ретінде // га жиым.
Console.WriteLine ("жиым га ұзындығы: және - га. Length); for (int i = - 5; i <= 5; i) ra [i] = i;
Console.Write ("жиым мазмұнын ra: "); for (int i = - 5; i <= 5; i)
Console.Write (ra [i] "");
Console.WriteLine ("\n");

```

Нысанды пайдалану ретінде // ra2 жиым.

```
Console.WriteLine ("жиым ұзындығыra2: "ra2.Length); for (int i = 1; i <= 10; i) ra2 [i] = i;
```

```
Console.Write ("жиым ұзындығы ra2: "); for (int i = 1; i <= 10; i)
```

```
Console.Write (ra2 [i] "");
```

```
Console.WriteLine ("\n");
```

```
{ } catch (RangeArrayException exc
```

```
Console.WriteLine (exc);
```

```
}
```

Ал енді өңдеуді // көрсету кейбірқателер.

```
Console.WriteLine ("шығару қателері шекарасын бұзған.");
```

Құрастырғышты try // дұрыс пайдалануға берілген {

```
RangeArray ra3 = new RangeArray (100 - 10); // Қате!
```

```
{ } catch (RangeArrayException exc
```

```
Console.WriteLine (exc);
```

```
}
```

```
{
```

// Дұрыс пайдалануға берілген индексі, try

```
RangeArray ra3 = new RangeArray(100, -10); // Қате!
```

```
} catch (RangeArrayException exc) {
```

```
Console.WriteLine(exc);
```

```
}
```

// Дұрыс пайдалануға берілген индексі, try

```
RangeArray ra3 = new RangeArray(-2, 2);
```

```
for(int i = -2; i <= 2; i++) ra3[i] = i;
```

```
Console.Write("ra3 жиымы: ");
```

```
for (int i = -2; i <= 10; i++)
```

```
Console.Write(ra3[i] + " ");
```

```
} catch (RangeArrayException exc) {
```

```
Console.WriteLine(exc);
```

```
}
```

```
}
```

```
}
```

Осы бағдарламаны орындау кейін келесі нәтижеболып шығады.

Жиым га ұзындығы: 11

Жиым мазмұнын га: Ұзындығы - 5 4 3 2 1012345 жиым га2: 10

Жиым мазмұнын га2: 12345678910

Қате шығару шекарасын бұзу.

Жоғарғы жол астылық аз емес.

Жиым мазмұнын га3: - 2-1012 шекарасын бұзу қатесі.

Жиым шекарасын бұзған кезде қате пайда болды, сынып `RangeArrayException` түріндегі нысан `RangeArray` дайындайды. Келесі орындарға `RangeArray` ұш сыныпта осылай болуы мүмкін: Жаңа аксессоре индекстегіш аксессоре құрастырушыда сыныпты, `set` және `get` индекстегіш `RangeArray`. "Қоспау" түрінің нысандарын тиіс деп ұстап алу үшін осы сияқты, мүмкін оның `RangeArray` құрастырылған және қол жетімді `true` блогын көрсеткен жоғарыда нақтыланады. Қателер туралы хабарлау, енді бірі ретінде пайдалана отырып әрекет етеді, сондықтан ол `RangeArray` сыныбы үшін арнайы алып тастау мүмкін қателерді өңдеу механизміне бағдарламасында деректер түрлерінің толық интеграцияланған кіріктірілген жаңа `C #`, табылатын.

Жай ғана өз дәлелдерін береді, бірақ оның орнына олар жоқ екенін ескеріңіз конструкторларды сыныпты денесіндегі ерекшеліктер `RangeArrayException` операторлары `Exception` кілт сөз `base` пайдалана отырып, қандай да бір класы. Бұрын ретінде түсіндірді жағдайларда фирмалардың толықтырады, бүкіл `Exception` конструкторларға сыныпты ерекшеліктер құру үдерісі базалық сыныпты туынды сыныбы функциялары тапсыруға болады. "Қоспау" сыныбы толықтыруға міндетті түрде бір жылғы сыныпты мұра етхатын тиіс функцияларды туынды ғой мүлде `Exception`.

Жоғарыда келтірілген бастап одан әрі кітап оқуға көшу көріңіз бұрын аздап поэкспериментировать ұсынады. Атап айтқанда, Түсініктеме қылу үшін алдын ала анықтау әдісін `()` және бақылаулармен нәтижелерімен `ToString` көріңіз. Бұдан басқа, қараңызшы, бұл ретте стандартты құралдарымен де хабар әдепкі құрастырғышты пайдалана отырып, адамның қолына алып тастау жасап көріңіз.

Видеожазбалар:

Generic class <https://youtu.be/L5we5smQPBw?si=t8mWM1ns8gL1VpKe>

13 дәріс. Делегаттар. Топтық адрестеу. Оқиғалар.

Дәрістің мақсаты: студенттерде делегаттар мен оқиғаларды пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Делегаттарды пайдалану ерекшеліктері туралы түсініктерін көрсету;
- Оқиғаларды пайдалану ерекшеліктері туралы түсініктерін көрсету;

Делегат түсінігінің анықтамасынан бастайық. Қарапайым тілмен айтқанда, *делегат әдіске сілтеме жасайтын объект* болып саналады. Сондықтан, делегат құру нәтижесінде әдіске сілтеме жасайтын объект пайда болады.

Сонымен бірге, әдісті осы сілтеме арқылы шақыруға болады. Басқаша айтқанда, делегат өзі сілтеме жасап тұрған әдісті шақыруға мүмкіндік береді. Төменде осындай қағиданың әрекеттер орындауға қаншалықты ыңғайлы болатыны көрсетіледі.

Программаның орындалу барысында бір ғана делегаттың өзі әртүрлі әдістерді шақыруға қолданыла алады, ол үшін делегат сілтеме жасап тұрған әдісті өзгерту жеткілікті. Сонымен, делегат шақыратын әдіс компиляция кезінде емес, орындалу кезеңінде анықталады. Сондықтан делегаттың басты артықшылығы осында.

ЕСКЕРТУ

Егерде сіздің C/C++ тілдерінде программалаудан тәжірибеңіз бар болса, онда C# тіліндегі делегат C/C++ тіліндегі функцияға нұсқауышқа ұқсас болып келеді.

Делегат типі `delegate` түйінді сөзі арқылы жарияланады. Төменде делегаттың жалпы жазылу формасы көрсетілген:

```
delegate қайтарылатын_типi аты(параметрлер_тізімі);
```

мұндағы *қайтарылатын_типi* – делегатпен шақырылатын әдістермен қайтарылатын мәннің типін білдіреді; *аты* – делегаттың нақты аты; *параметрлер_тізімі* – делегат арқылы шақырылатын әдіс үшін қажетті параметрлер. Делегаттың экземплярлары құрылғаннан соң, ол бірден әдістерді шақырып сілтеме жасай алады, мұндағы әдістердің қайтаратын типі мен параметрлері делегаттың жариялануында көрсетілген мәндермен сәйкес болуы тиіс.

Ең бастысы, делегатты қайтарылатын типі мен сигнатурасы сәйкес келетін кез келген әдісті шақыру үшін қолдануға болады. Сонымен бірге, шақырылатын әдіс жеке объектімен байланысқан экземпляр әдісі немесе нақты класпен байланысқан статикалық әдіс болуы мүмкін. Бір ғана шартты қанағаттандырса болғаны: қайтарылатын тип пен әдістің сигнатурасы делегатты жариялау кезінде көрсетілген мәндермен сәйкес болуы керек.

Делегаттың жұмысын көрсету үшін алдымен оның қолданылуынан қарапайым мысал қарастырайық.

```
// Делегатты пайдаланудың қарапайым мысалы.
```

```
using System;
```

```
// Делегат типін жариялау.  
delegate string StrMod(string str);
```

```
class DelegateTest  
{  
    // Бос орындарды дефиспен алмастыру.
```

```

static string ReplaceSpaces(string s) {
    Console.WriteLine("Замена пробелов дефисами.");
    return s.Replace(' ', '-');
}

// Босорындарды өшіру.
static string RemoveSpaces(string s) {
    string temp = "";
    int i;

    Console.WriteLine("Удаление пробелов.");
    for (i = 0; i < s.Length; i++)
        if (s[i] != ' ') temp += s[i];
    return temp;
}

// Тіркесті кері жазу.
static string Reverse(string s) {
    string temp = "";
    int i, j;

    Console.WriteLine("Обращение строки. ");
    for (j = 0, i = s.Length - 1; i >= 0; i--, j++)
        temp += s[i];
    return temp;
}

static void Main()
{
    // Делегаты конструкциялау.
    StrMod strOp = new StrMod(ReplaceSpaces);
    string str;

    // Делегат арқылы әдісті шақыру.
    str = strOp("Это простой тест.");
    Console.WriteLine("Результирующая строка: " + str);
    Console.WriteLine();

    strOp = new StrMod(RemoveSpaces);
    str = strOp("Это простой тест.");
    Console.WriteLine("Результирующая строка: " + str);
    Console.WriteLine();

    strOp = new StrMod(Reverse);
    str = strOp("Это простой тест.");
    Console.WriteLine("Результирующая строка: " + str);
    Console.ReadKey();
}

```

Бұл кодты орындау келесідей нәтиже береді.

Замена пробелов дефисами.

Результирующая строка: Это-простой-тест.

```

file:///D:/MyDocs/Visual Studio 2...
Замена пробелов дефисами.
Результирующая строка: Это-простой-тест.

Удаление пробелов.
Результирующая строка: Этопростойтест.

Обращение строки.
Результирующая строка: .тсет йотсорп отЭ

```

Удаление пробелов.

Результирующая строка: Этопростойтест.

Обращение строки.

Результирующая строка: .тсет йотсорп отЭ

Топтық адрестеу

Делегаттың ең тамаша қасиеттерінің бірі топтық адрестеуді сүйемелдеу болып табылады. Қысқаша айтқанда, *топтық адрестеу* – бұл делегатты пайдалану кезінде, автоматты түрде шақырылатын әдістер үшін жаңа *тізім* немесе *шақырулар тізібегін* жасау деген сөз. Мұндай тізім құру қиын емес. Ол үшін бір делегат экземплярын жасап алып, сонан соң `+` немесе `+=` операторы көмегімен сол тізімге әдістер косу жеткілікті. Тізімдегі бір әдісті өшіру үшін `-` (азайту) немесе `-=` операторы қолданылады. Егерде делегат мән қайтаратын болса, онда ол шақырулар тізіміндегі ең соңғы әдіс қайтаратын мән болып табылады. Сондықтан, топтық адрестеу қолданылатын делегаттың қайтарылатын типі көбінесе `void` болады.

Төменде топтық адрестеу мысалы келтірілген. Бұл алдыңғы мысалдардың қайта өзгертілген нұсқасы, мұнда тіркестерді өңдейтін әдістер қайтаратын мәндердің типі `void` болып өзгертілген, ал өңделген сөз тіркесін кодтың шақыратын бөлігіне қайтару үшін `ref` типіндегі параметр қолданылады. Осыған орай бұл әдістер топтық адрестеуге бейімделген түрде болады.

```
// Топтық адрестеуді көрсету.
```

```
using System;
```

```
// Делегат типін жариялау
```

```
delegate void StrMod(ref string str);
```

```
class MultiCastDemo
```

```
{
```

```
    // Дефистерді бос орындармен алмастыру
```

```
    static void ReplaceSpaces(ref string s)
```

```
    {
```

```
        Console.WriteLine("Замена пробелов дефисами.");
```

```
s = s.Replace(' ', '-');
```

```
    }
```

```
    // Бос орындарды өшіру
```

```
    static void RemoveSpaces(ref string s)
```

```
    {
```

```
        string temp = "";
```

```
        int i;
```

```
        Console.WriteLine("Удаление пробелов.");
```

```
        for (i = 0; i < s.Length; i++)
```

```
            if (s[i] != ' ') temp += s[i];
```

```
s = temp;
```

```
    }
```

```
    // Тіркесті кері жазу.
```

```
    static void Reverse(ref string s)
```

```
{
```

```

        string temp = "";
        int i, j;

        Console.WriteLine("Обращение строки.");
        for (j = 0, i = s.Length - 1; i >= 0; i--, j++)
            temp += s[i];

        s = temp;
    }

    static void Main()
    {
        //Делегатты конструкциялау.
        StrMod strOp;
        StrMod replaceSp = ReplaceSpaces;
        StrMod removeSp = RemoveSpaces;
        StrMod reverseStr = Reverse;
        string str = "Это простой тест.";

        // Топтық адресстеуді ұйымдастыру
        strOp = replaceSp;
        strOp += reverseStr;
        //Делегатқа топтық адресстеуді қолдану.
        strOp (ref str);
        Console.WriteLine("Результирующая строка: " + str);
        Console.WriteLine();
        //Босорындарды алмастыру әдісін өшіріп,босорындарды өшіретін әдіс қосу.
        strOp -= replaceSp;
        strOp += removeSp;
        str = "Это простой тест."; // бастапқы жолды қалпына келтіру

        // Топтық адресстеуі бар делегатты пайдалану.
        strOp (ref str);
        Console.WriteLine("Результирующая строка: " + str);
        Console.WriteLine();
    }
}

```

Бұл кодтың орындалуы келесі нәтижеге алып келеді.

Замена пробелов дефисами.

Обращение строки.

Результирующая строка: .тсет-йотсорп-отЭ

Обращение строки.

Удаление пробелов.

Результирующая строка: .тсетйотсорпотЭ

Қарастырылып отырған кодта Main() әдісінде делегаттың төрт экземпляры құрылады. Олардың біріншісі – strOp бос болып құрылады, ал қалған үшеуі сөз тіркестерін өзгертетін нақтыәдістерге сілтеме жасайды. Содан кейін RemoveSpaces() и Reverse() әдістерін шақыру үшін топтық адресстеу ұйымдастырылады.

Делегаттардың атқаратын қызметі

Алдыңғы мысалдарда делегаттар қызметінің ішкі механизмі көрсетілген болатын, бірақ бұл мысалдар олардың шынайы атқаратын қызметін айқын бере алмайды.

Көбінесе, делегаттар екі түрлі себеппен қолданылады. Біріншіден, бұдан бұрын айтылғандай, делегаттар оқиғаларды сүйемелдейді. Екіншіден, делегаттар программаны орындалуы кезінде компиляция барысында, олар туралы ешнәрсені анық білмесе де, әдістерді шақыра алады. Бұл жекелеген программалық компоненттерді қосуға мүмкіндік беретін базалық конструкцияны құру кезінде өте қолайлы.

Мысал ретінде стандартты Windows Paint сервистік программасына ұқсас графикалық программаны қарастырайық. Делегат көмегімен қолданушыға арнайы түрлі түсті фильтрлер мен бейнелер анализаторын қосатын мүмкіндіктер беруге болады. Сонымен қатар, қолданушы осы фильтрлер мен анализаторларды пайдалана отырып, олардан бүтін бір тізбекті бейнелер құрастыра алады. Делегаттарды қолдану арқылы программалардың осы сияқты мүмкіндіктерін қамтамасыз ету қиынемес.

Оқиғалар

C# тіліндегі делегаттарға негізделіп жасалған тағы бір құрал – оқиға. Оқиға, негізінде, қандай да бір әрекеттің болғанын автоматты түрде хабарлайды. Оқиғалар келесі принцип бойынша әрекет жасайды: оқиғаға қызығушылық білдіретін объект, сол оқиғаның өңдеуішін тіркеуге алады. Оқиға орындалғанда, сол оқиғаның барлық тіркелген өңдеуіштері шақырылады. Оқиға өңдеуіштері әдетте делегаттар түрінде беріледі. Оқиғалар кластың мүшелері болып табылады да, `event` түйінді сөзі көмегімен жарияланады. Көбінесе олар мынадай формада қолданылады:

`event оқиға_делегаты оқиға_аты;`

мұндағы `оқиға_делегаты` оқиғаны сүйемелдеу үшін қолданылатын делегат атын белгілейді, ал `оқиға_аты` — жарияланатын оқиғаның нақты бір объектісі.

Алдымен қарапайым мысал қарастырайық.

// Оқиға жұмысын көрсететін өте қарапайым мысал

```
using System;
```

// Оқиға үшін делегат типін анықтау.

```
delegate void MyEventHandler();
```

// Оқиғаны қамтитын классты жариялау

```
class MyEvent {
    public event MyEventHandler SomeEvent;
    // бұл әдіс оқиғаны іске қосу үшін қолданылады
    public void OnSomeEvent() {
        if (SomeEvent != null)
            SomeEvent();
    }
}

class EventDemo {
    // Оқиғаны өңдеуіш
    static void Handler() {
        Console.WriteLine("Произошло событие");
    }
}
```

```
static void Main() {
    MyEvent evt = new MyEvent();
    // Handler() әдісін оқиға тізіміне қосу,
```

```

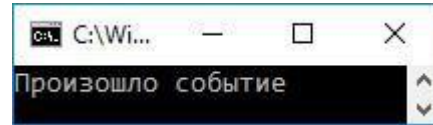
evt.SomeEvent += Handler;
    // Оқиғаны жіберу

evt.OnSomeEvent();
}
}

```

Кодты орындағанда шығатын нәтиже:

Произошло событие



Өзінің қарапайымдылығына қарамастан, берілген мысалда оқиғаны өңдеуге қажет барлық негізгі элементтер бар. Ол төменде көрсетілгендей, оқиғаны өңдеуші үшін қажетті делегаттың типін жариялаудан басталады.

```
delegate void MyEventHandler();
```

Барлық оқиғалар делегаттар көмегімен іске асырылады. Сондықтан, оқиға делегатының типі сол оқиға үшін қайтарылатын тип пен сигнатураны анықтайды. Мұндағы жағдайда оқиғаның параметрлері жоқ, бірақ оларды көрсетуге рұқсат етіледі.

Ары қарай `MyEvent` оқиға класы құрылады. Бұл класта келесі жолдағы `SomeEvent` оқиғасы жарияланады.

```
public event MyEventHandler SomeEvent;
```

Бұл жарияланудың синтаксисіне назар аударыңыз. `Event` түйінді сөзі компиляторға оқиға жарияланатыны туралы мәлімет береді.

Сонымен қатар, `MyEvent` класында оқиғаның іске қосылатыны жайлы сигнал беру үшін шақырылатын `OnSomeEvent()` әдісі жарияланады. Бұл оның оқиға орындалуы кезінде шақырылатынын білдіреді. `OnSomeEvent()` әдісінде `SomeEvent` делегаты көмегімен оқиға өңдеуші шақырылады.

```

if (SomeEvent != null)
    SomeEvent();

```

Көріп тұрғаныңыздай, `SomeEvent` оқиғасы бос болмаған жағдайда ғана оқиға өңдеуші шақырылады. Ал оқиғаға қызығушылық программаның басқа бөліктерінде тіркелуі керек болғандықтан, ол туралы мәлімет алу үшін, кез келген оқиға өңдеуші тіркелгенге дейін `OnSomeEvent()` әдісі шақырылуы керек. Бірақ бос сілтеме бойынша шақыруды болдырмас үшін, оқиға делегатының бос емес екендігіне көз жеткізу мақсатында, оқиға делегаты тексерілуі тиіс.

`EventDemo` класында `Handler()` оқиға өңдеушісі құрылады. Берілген осы қарапайым мысалда оқиға өңдеушісі жай ғана хабарлама шығарады, бірақ басқа өңдеушілер мұнан гөрі күрделірек функцияларды орындай алады. Ары қарай, `Main()` әдісінде `MyEvent` оқиға класының объектісі құрылады, ал `Handler()` тізімге қосылатын осы оқиғаның өңдеушісі ретінде тіркеледі.

```

MyEvent evt = new MyEvent();
// Handler() әдісін оқиға тізіміне қосу.

evt.SomeEvent += Handler;

```

Өңдеушінің `+=` операторы арқылы тізімге қосылатынына назар аударыңыздар. Оқиғаларды тек `+=` және `-=` операторлары ғана сүйемелдейді. Мұндағы жағдайда `Handler()`

статикалық әдіс болып табылады, бірақ оқиға өңдеуіштері ретінде экземпляр әдістері де қолданыла алады.

Сонымен, ең соңында оқиға төменде көрсетілгендей түрде іске қосылады.

```
// Оқиғаны іске қосу  
evt.OnSomeEvent();
```

OnSomeEvent() әдісін шақыру өңдеуіш тіркеген барлық оқиғаларды шақыруға әкеп соқтырады. Мұндағы жағдайда тек бір өңдеуіш тіркелген, бірақ олар, келесі бөлімде көрсетілетіндей, бірнешеу болулары да мүмкін.

Видеожазбалар:

Делегаттар <https://youtu.be/rT0bRkLbEu8?si=mCuQCik0Fg62m7J3>

14 дәріс. Анонимдік функциялар. Анонимдік әдістер және лямбда-өрнектер.

Дәрістің мақсаты: студенттерде анонимдік функцияларды пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Анонимдік әдістерді пайдалану ерекшеліктері туралы түсініктерін көрсету;
- Лямбда-өрнектерді пайдалану ерекшеліктері туралы түсініктерін көрсету.

Анонимдік функциялар

Делегат сілтеме жасап тұратын әдіс көбінесе осы мақсат үшін қолданылады. Басқаша айтқанда, әдістің қолданылуының негізгі себебі ол тек қана делегат арқылы шақырылады, ал әдістің өзі еш уақытта шақырылмайды. Осындай жағдайларда, жеке әдіс құрмай-ақ, *анонимдік* (жасырын) *функцияны* қолдануға болады. Анонимдік функция, шын мәнінде, делегат конструкторына берілетін атаусыз кодтық блок болып табылады. Ашып айтар болсақ, анонимдік функцияның артықшылығы оның қарапайымдылығында. Соның арқасында, ең басты қызметі делегатқа берілуі болып табылатын жеке әдісті жарияламауға болады.

C# тілінің 3.0 нұсқасын бастап, анонимдік функцияның екі түрі қарастырылды: *анонимдік әдістер* және *лямбда-өрнектер*. Анонимдік әдістер C# тілінің 2.0 нұсқасына енгізілсе, лямбда-өрнектер оның 3.0 нұсқасында пайда болған еді. Жалпы айтқанда, лямбда-өрнек анонимдік әдістің жұмыс істеу қағидасын жақсартады және қазіргі уақытта анонимдік функциясын құру үшін ол қолайлырақ болып табылады. Бірақ, анонимдік әдістер C# тілі кодтарында кең қолданылады, сондықтан да олар C# тілінің маңызды құрамдас бөлігі болып табылады.

Анонимдік әдістер лямбда-өрнектерден бұрын пайда болғандықтан, оларды қарастыру лямбда-өрнектердің ерекшелігін дұрыс түсінуге мүмкіндік береді. Сонымен қатар, анонимдік әдістерді, лямбда-өрнектерді қолдануға болмайтын көптеген жағдайларда, пайдалануға болады. Сондықтан да осы тарауда анонимдік әдістер де және лямбда-өрнектер де қатар қарастырылады.

Анонимдік әдістер

Анонимдік әдіс — делегаттың нақты экземплярымен байланысты кодтың атаусыз блогын құрудың бір тәсілі.

Анонимдік әдісті құру үшін `delegate` түйінді сөзінен кейін код блогын көрсету жеткілікті. Мұның қалай құрылатынын нақты мысалмен көрсетейік. Төменде көрсетілген программада анонимдік әдіс 0-ден 5-ке дейін санауды ұйымдастыру үшін қолданылады.

```
// Анонимдік әдістің қолданылуы
```

```
using System;
```

```
// Делегат типін жариялау.  
delegate void CountIt();
```

```
class AnonMethDemo  
{  
    static void Main()  
    {
```

```
        // Ары қарай анонимдік әдіс ретінде делегатқа берілетін
```

```

        // сандарды санайтын код орналасады.
        CountIt count = delegate
    {
        // Бұл кодтық блок делегатқа беріледі.
        for (int i = 0; i <= 5; i++)
            Console.WriteLine(i);
    } ;    // нүктелі үтірге назар аударыңыз

    count ( ) ;
}
}

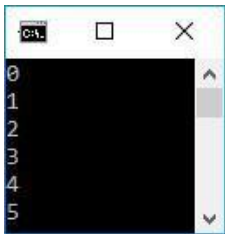
```

Бұл программада, алдымен, void типін қайтаратын, параметрсіз CountIt делегат типі жарияланады. Содан соң Main() әдісінде CountIt делегатының count экземпляры құрылады, оған delegate түйінді сөзінен кейін орналасқан код блогы беріледі. Дәл осы код блогы count делегатын қолданған кезде орындалатын анонимдік әдіс болып табылады. Код блогынан кейін тұрған нүктелі үтірге назар аударыңыз, ол жариялау операторын нақты түрде аяқтайтын белгі болып табылады. Төменде жоғарыдағы программа нәтижесі келтірілген.

```

0
1
2
3
4
5

```



Аргументтердің анонимдік әдіске берілуі

Анонимдік әдіске бір немесе бірнеше аргумент беруге болады. Ол үшін delegate түйінді сөзінен кейін жақша ішінде параметрлер тізімін көрсету жеткілікті, ал мұнан кейін делегат экземплярын қолданғанда, оған тізімге сәйкес аргументтер беріледі. Төменде мысал ретінде алдыңғы программаның өзгертілген нұсқасы келтірілген, онда аргумент ретінде саналуға тиіс мәндер берілген.

```

//Аргумент қабылдайтын анонимдік әдістің қолданылуын көрсету

using System;

// Енді CountIt делегатының параметрі бар екеніне назар аударыңыз
delegate void CountIt(int end);

class AnonMethDemo2 {

    static void Main() {

        // Есептің соңғы шешімі анонимді әдіске беріледі.
        CountIt count = delegate(int end)
        {
            for (int i = 0; i <= end; i++)
                Console.WriteLine(i);
        } ;

        count ( 3 ) ;
        Console.WriteLine();
        count ( 5 ) ;
    }
}

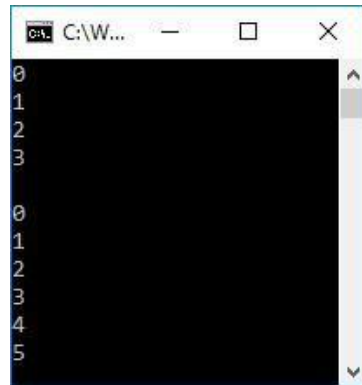
```

}

Бұл программа нұсқасында CountIt делегаты бүтін сан түріндегі аргументті қабылдайды. Анонимдік әдісті құрғанда параметрлер тізімі delegate түйінді сөзінен кейін көрсетілетініне назар аудару керек. End параметрі аты бар әдісті құрғандағы сияқты анонимді әдісте де код үшін қолжетімді болады. Төменде программаның орындалу нәтижесі көрсетілген.

0
1
2
3

0
1
2
3
4
5



Анонимдік әдістен мән қайтару

Анонимдік әдіс мән қайтара алады. Ол үшін аты бар әдістердегі сияқты return операторы қолданылады. Қайтарылған мәннің типі делегатты жариялағанда көрсетілген қайтарылатын типпен сәйкес келуі керек. Мысал ретінде төменде қосындылау арқылы санау жүргізіп, нәтиже қайтаратын программа коды көрсетілген.

//Мән қайтаратын анонимдік әдістің қолданылуы.

```
using System;
```

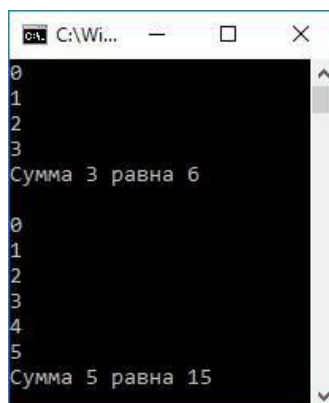
```
// Бұл делегат мән қайтарады  
delegate int CountIt(int end);
```

```
class AnonMethDemo3 {  
  
    static void Main() {  
        int result;  
  
        // Санауға арналған мәндер анонимдік әдіске беріледі.  
  
        // Саналған мәндер қосындысы қайтарылады  
        CountIt count = delegate (int end) {  
            int sum = 0;  
  
            for(int i=0; i <= end; i++) {  
                Console.WriteLine (i);  
                sum += i;  
            }  
            return sum; // анонимдік әдістен мән қайтарылады  
        } ;  
  
        result = count(3);  
        Console.WriteLine("Сумма 3 равна " + result);  
        Console.WriteLine ();  
  
        result = count(5);  
        Console.WriteLine("Сумма 5 равна " + result);  
    }  
}
```

Программаның бұл нұсқасында қосындының мәні count делегатының экземплярымен байланысты код блогымен қайтарылады. Мұндағы анонимдік әдісте return операторының аты бар әдістегідей қолданылатынына назар аударыңыз. Төменде осы кодтың нәтижесі көрсетілген.

```
0  
1  
2  
3  
Сумма 3 равна 6
```

```
0  
1  
2  
3  
4  
5  
Сумма 5 равна 15
```



3

4

5

Сумма 5 равна 15

Лямбда-өрнектер

Анонимдік әдістердің құндылығына қарамастан, олардың орнына мүмкіндіктері мол лямбда-өрнектер келді. Лямбда-өрнек C# тілінің 1.0 нұсқасынан бастап, енгізілген мәні зор жаңалықтардың бірі болды. Ол жаңа синтаксистік элементке негізделіп жасалған және анонимдік әдіске қарағанда, анағұрлым тиімді балама (альтернатива) болып табылады. Лямбда-өрнек артықшылықтары LINQ-мен (ол туралы 19-тарауда) жұмыс жасағанда жақсы білінгенмен, олар делегаттармен де және оқиғалармен де бірге жиі қолданылады. Енді осы туралы айтатын боламыз.

Лямбда-өрнек – бұл анонимдік функцияларды құрудың жаңа түрі. (Оның анонимдік әдіс ретінде қолданылуы бұдан бұрын айтылды.) Сол себепті ол делегатқа меншіктеле алады. Лямбда-өрнек оған ұқсас анонимдік әдіске қарағанда, тиімді болғандықтан, көбінесе осы өрнекті қолдану жиі ұсынылып жүр.

Лямбда-оператор

Барлық лямбда-өрнектерде лямбда-өрнекті екі бөлікке бөлетін жаңа $\Rightarrow$ лямбда-оператор қолданылады. Оның сол жақ бөлігінде кіріс параметрі (немесе бірнеше параметр), ал оң жақ бөлігінде – лямбда-өрнектің тұлғасы (өз денесі) көрсетіледі. Кейде $\Rightarrow$ операторы «ауысады» немесе «болып шығады» деген сөздерімен сипатталады.

C# тілінде лямбда-өрнектің, өз ішкі мазмұнына байланысты, екі түрі сүйемелденеді. Егер лямбда-өрнектің денесі бір өрнектен ғана тұратын болса, *жеке лямбда-өрнек* құрылады. Мұндай жағдайда өрнек жүйелі (фигуралы) жақшаға алынбайды. Ал егер лямбда-өрнектің денесі жүйелі жақшаларға алынған операторлар блогынан тұрса, онда *блоктық лямбда-өрнек* құрылады. Мұндайда блоктық лямбда-өрнек бірнеше операторлар тізбегінен: циклдерден, if шартты операторларынан және әдістерді шақырулардан да тұратын болды. Енді лямбда-өрнектердің осы екі түрін жеке-жеке қарастырып шығамыз.

Жеке лямбда-өрнектер

Жеке лямбда-өрнектегі $\Rightarrow$ операторының оң жағындағы бөлігі сол жағында көрсетілген параметрге (бірнеше параметрлерге) әсер етеді. Мұндай өрнектің есептеліп қайтарылатын нәтижесі лямбда-оператордың орындалу нәтижесі болып табылады.

Төменде жалғыз параметрді қабылдайтын жеке лямбда-өрнектің жалпы жазылу формасы көрсетілген.

параметр $\Rightarrow$ өрнек

Егер бірнеше параметрді көрсету керек болса келесі форма қолданылады:

(параметрлер_тізімі) $\Rightarrow$ өрнек

Осылайша екі немесе одан көп параметрді көрсету үшін, оларды жақшаға алу қажет. Егер өрнек параметрді қажет етпесе, іші бос жақшаларды қолдану керек.

Төменде жеке лямбда-өрнектің қарапайым мысалдары көрсетілген:

`count  $\Rightarrow$  count+2`

Бұл өрнекте count айнымалысы `count + 2` өрнегіәсер ететін параметр болып табылады, осыныңнәтижесінде count операторыныңмәні екіге артады. Төменде жеке лямбда-өрнектің тағы бір мысалы көрсетілген.

```
n => n % 2 == 0
```

Мұндағы жағдайда егер n параметрінің сандық мәні жұп сан болса, өрнек true логикалық мәнін қайтарады, кері жағдайда false логикалық мәнін қайтарады.

Лямбда-өрнек екі кезеңде қолданылады. Алдымен лямбда-өрнекпен үйлесімді болып келетін делегат типі жарияланады, сонан соң лямбда-өрнек меншіктелетін делегат экземпляры жарияланады. Оның есептелген нәтижесі қайтарылатын мән болып табылады.

Төмендегі программа мысалында екі жеке лямбда-өрнектің қолданылуы көрсетілген. Алдымен программада делегаттың екі типі жарияланады. Олардың біріншісі int типті аргументті қабылдап алып, сол типтегі нәтижені қайтаратын Incr делегаты болыптабылады. Ал, екіншісі int типті аргументті қабылдайтын, бірақ bool типті нәтиже қайтаратын IsEven делегаты болып келеді. Сонан кейін осы делегаттар экземплярларына жеке лямбда-өрнектер меншіктеледі. Соңында лямбда-өрнек делегаттардың соларға сәйкес экземплярларының көмегімен есептеледі.

```
// Екі жеке лямбда-өрнекті пайдалану.
```

```
using System;
```

```
// int типін қабылдайтын және int типін қайтаратын делегат жариялау.  
delegate int Incr(int v);
```

```
// int типін қабылдайтын және bool типін қайтаратын делегат жариялау.  
delegate bool IsEven(int v);
```

```
class SimpleLambdaDemo {
```

```
    static void Main() {
```

```
        // Лямбда-өрнегіне сілтеме жасап, өз параметрін 2-ге арттыратын  
        // Incr делегатын құру.  
        Incr incr = count => count + 2;
```

```
        // Ал енді incr лямбда-өрнегіне пайдалану.
```

```
        Console.WriteLine("Использование лямбда-выражения incr: ");
```

```
        int x = -10;
```

```
        while (x <= 0)
```

```
{
```

```
    Console.Write(x + " ");
```

```
x = i n c r ( x ) ; // x мәнін 2-ге арттыру
```

```
}
```

```
    Console.WriteLine("\n");
```

```
    // Лямбда-өрнекке сілтеме жасайтын IsEven делегат экземплярын құру,  
    // егер оның параметрі жұп сан болса, true мәнін қайтарады,  
    // әйтпесе false мәнін қайтарады
```

```
    IsEven isEven = n => n % 2 == 0;
```

```

//Ал енді isEven лямбда-өрнегін пайдаланамыз.
Console.WriteLine("Использование лямбда-выражения isEven: ");

for (int i = 1; i <= 10; i++)
    if (isEven(i)) Console.WriteLine(i + " четное.");
}
}

```

Программаның орындалуы төмендегідей нәтиже береді.

Использование лямбда-выражения incr:

-10 -8 -6 -4 -2 0

Использование лямбда-выражения isEven:

2 четное.

4 четное.

6 четное.

8 четное.

10 четное.

```

C:\Windows\system32\cmd.exe
Использование лямбда-выражения incr:
-10 -8 -6 -4 -2 0
Использование лямбда-выражения isEven:
2 четное.
4 четное.
6 четное.
8 четное.
10 четное.

```

Программадағы келесі жарияланған жолдарға назар аударыңыз.

Incr incr = count => count +2;

IsEven isEven = n => n % 2 == 0;

Видеожазбалар:

Анонимді функциялар https://youtu.be/_LTpO_cT0vc?si=QgWGLS43d2zsVXSp

15 дәріс. Жалпыламалар.

Дәрістің мақсаты: студенттерде жалпыламалардың қызметі және оларды пайдалану ерекшеліктері туралы түсініктерін көрсетуге қабілет қалыптастыру.

Осы дәрісті меңгеру нәтижесінде студенттер келесі қабілеттерге ие болады:

- Жалпыланған кластарды пайдалану ерекшеліктері туралы түсініктерін көрсету;
- Жалпыланған әдістерді пайдалану ерекшеліктері туралы түсініктерін көрсету.

Жалпылама термині параметрленген типті білдіреді. Олар ерекше ролі параметрленген кластар, құрылымын, оларды жасауға мүмкіндік береді, оларда Өңделетін деректер түрлерінің екендігі тұрады, әдістері мен делегаттар интерфейстер параметр түрінде көрсетіледі. Болады, мысалы, автоматты түрде өңдеу үшін жарамды болады, ол бірыңғай сыныбы жалпылауларды көмегімен жасау әртүрлі болады. Делегат оперирующий параметри зированным деп аталады, мысалы, сыныбы, құрылымы, интерфейс, әдісті немесе деректер түрімен немесе жалпылама жалпыланған сыныбы қорытылған дісі.

С # әрқашан ерекше атап өткен жөн, бұл түрінің коды, операция жасап object жалпыланған байланыстар жасау мүмкіндігіне болды. Қалған барлық түрі объект жүгінуге болады, онда кез келген үлгідегі object сілтеме бойынша базалық болғандықтан object сыныбы үшін сынып болып табылады. Осылайша, операция жасау үшін пайдаланылған сілтемелер осы мақсат үшін түрінің коды, онда қызмет объектілері жалпылауларды пайда болғанға дейін разнотипными бағдарламаларында жалпыланған object.

Бірақ мәселе мынада, мұндай қиын болды, себебі осы түрді түрлендіру object нақты деректер түрі түрлерінің коды үлгі қауіпсіздігін сақтауға келтіру қажет болды. Ал бұл қателерді келтіруге себепші болды, бұл әлеуетті көзі болуы мүмкін түрлерінің анадан қасақана емес выколнено дұрыс емес. Бұл мүмкіндік береді, оған да қауіпсіздігін қамтамасыз ете отырып, бұрын оларды қорыту үлгі алар еді. Сонымен қатар бүкіл процесін орындау қажеттігін болдырмайды немесе басқа үлгідегі түрлендіру үшін түрлер келтіру топтауға көмектеседі, себебі, қорыту объектінің өңделетін болады. Осылайша, қорыту қайта пайдалану мүмкіндігін кеңейтеді кодының және сенімді және қарапайым мұны істеуге мүмкіндік береді.

Қарапайым мысал жалпылауларды

Қарапайым мысал бастаймыз жалпылауларды қарау бастап болғанын білуіміз қажет. Төменде келтірілген бағдарламасы осы екі класс айқындалады. Олардың алғашқысы болып табылады, онда Gen сыныбы Demo Generics сыныбы, екінші - жалпыланған Gen сыныбы пайдаланылады.

Қарапайым мысал // болғанын білуіміз сынып, using System;

Төменде келтірілген нақты түрін жасау кезінде деректер түрімен Gen. class Gen < T > түріндегі нысан ауыстырылады // в сыныптағы Gen параметр T // {

T ob; Айнымалы жариялау // түрі T

Осы параметр түрі бар екеніне ескеріңіз құрастырушы // у = {ob public Gen(T o) туралы;

}

Айнымалы қайтаруға // ob, ол {return ob T. public T GetObO данасын түріне жатады;

}

T. public void { // ShowTypeO түрін көрсету

Console.WriteLine ("K T түріне жатады" typeof (T));

}

}

```

{{Static Main () void сыныбы, classGenericsДемоболғанын білуімізқолдану // көрсету

Айнымалы сілтеме түрін жасау // Gen объектісіне int.
Gen < int > iob;
// Түріндегі нысан жасау және оған сілтеме айнымалы iob iob = new gen < int > Gen <
int > тағайындау (102);
Айнымалы iOb // сақталған деректерді түрін көрсету. iOb.ShowType();
Анымалының мәнін алу // iOb. int v = iOb.GetOb ();
Console.WriteLine ("мән: "v);
Console.WriteLine ();
Жолдар үшін // Gen түріндегі нысан жасау.
Gen < string > strob = new Gen<string>("Жалпыламалар тиімділікті арттырады.");

Айнымалы strOb // сақталған деректерді түрін көрсету. strOb.ShowType();
Анымалының мәнін алу // strOb. string str = strOb.GetOb ();
Console.WriteLine ("мән: "str);
Бұл бағдарлама келесі нәтиже береді.
Т относится' түріне System.Int32 мән: 102
Т System.String түріне жатады
Мән: Қорыту тиімділігі арттырады.
Бұл бағдарламаны мұқият талдаймыз. Ең алдымен осы жолда сынып хабарландыру
Gen назар аударыңыз төменде келтірілген код:
class Gen < T > {
    Онда Т - бұл параметр атауы түрі. Бұл атау ретінде қызмет етеді, ол нақты үлгідегі
    толтырғыш белгілері объектісі құрылған кезде сыныпты Демек, әр жолы параметрі талап
    етілетін түрдегі сыныпта пайдаланылады Gen Gen. Т атауы көрсетіледі. Бұрыштық
    жақшалар екенін ескеріңіз Т атауы болып табылады (< >). Бұл синтаксис қорыту болады:
    жақша ішінде хабарланады, ол кез келген кезде, бір бұрышы параметр түрі көрсетіледі.
    Параметр түрі болғандықтан сыныпта пайдаланылады, онда мұндай сыныбы Gen
    жалпылама болып есептеледі.

    Дәстүр бойынша қолданылатын басқа да кез келген түрін көрсетуге болады, бірақ
    сынып хабарландыруда Gen параметр атауы неғұрлым қатарына жатады және Е Сіз, әрине,
    еркін пайдалануға және одан да көп түрінің аттар таңдайды аты Т. V сипаттама аттар,
    мысалы TValue параметрлерін немесе TKey. Бірақ бұл жағдайда түрін көрсетуге
    қабылданған параметр атындағы бірінші бас әріптен Т.

```

Лабораториялық жұмыс1. Қолданушы функцияларын құру.

Лабораториялық жұмыстың мақсаты: Студенттерде қолданушы функцияларын құру дағдысын қалыптастыру.

Лабораториялықжұмысты орындаунәтижесінде студенттер келесі қабілеттерге ие болады:

- Қолданушы функцияларына параметрлер беру;
- Функциялардан мән қайтару.

Тапсырма: Төменде көрсетілген мысалдармен танысып, Абрамянның программалау бойынша тапсырмалар жинағының «Функциялар және процедуралар» бөлімінде берілген тапсырмаларды нұсқаңызға сәйкес орындаңыз.

Күрделі әрі көлемді программалар жазу кезінде, программаны*функцияларға* – шағын бөліктерге бөлу арқылы жұмысты жеңілдетуге және бастапқы код көрнекілігін арттыруға болады. Мысалы, ағымдағы бухгалтерлікшоттармен жұмыс істейтін программа жазу қажет болсын делік. Мұнда кітап операцияларын орындайтын басты бір функция жазып, шоттарды төлеу ісін атқаратын тағы бір функция, сонан соң қабылдап алынатын шоттарды реттейтін функция және баланстық есеп беру жұмысын орындайтын функциялар қарастырылса, программаны оқу, түсіну жеңіл болар еді. Егер осы әрекеттерді орындайтын барлық операторлар **Main** ішінде орналасқан болса, онда программа өте көлемді болып кетеді және оны түсіну де қиын болар еді. Оның үстіне, программа көлемі мен күрделілігі артқан сайын, онда қате жіберу мүмкіндігі де ұлғаяды.

Берілген бір тапсырманы атқаратын атаулы (өзіндік аты бар) операторлар жиыны *қолданушыфункциясы* болып табылады. Мысалы, **hello_world** деп аталатын келесі функция **Console.WriteLine()** операторы арқылы экранға мәлімет жазып шығарады:

```
static void hello_world(void)
{
    Console.WriteLine("Hello world!!!");
}
```

Көбінесе функциялар программаға өз есептеулерінің нәтижесін қайтарады. Егер функция ешқандай да мән қайтармайтын болса, онда ол функция атының алдында **void** түйінді сөзі көрсетілуі қажет. Функция атынан кейін жақша ішінде оның параметрлері көрсетіледі.*Параметр–бұлфункцияғаөңдеуүшінберілетінмәлімет* . Егер функцияның параметрлері болмаса, онда дөңгелек жақшалар ішіне **void** түйінді сөзі жазылады немесе ештеңе көрсетілмейді. Функцияны пайдалануды функцияны шақыру деп атайды. Келесі программада **hello_world** функциясы пайдаланылады.

```
using System;
namespace Example
{
    class Program
    { static void hello_world()
      {
        Console.WriteLine("Hello_world()!!!");
      }
    static void Main(string[] args)
    {
```

```

    Program.hello_world();
    Console.ReadKey();
}
}
}

```

Программаны орындау нәтижесі:

Hello world!!!

Сонымен, программаны іске қосқан кезде бірінші болып **Main** функциясы орындалады. Жоғарыдағы программада **hello world** функциясын шақыру **Main** функциясында атқарылады. Егер программаны орындау кезінде функцияны шақыру әрекеті кезіксе, онда басқару бірден функцияға беріледі де, оның операторлары ең алғашқы жолдан бастап орындала бастайды. Функцияның ең соңғы операторы орындалып болған соң, басқару негізгі программадағы функцияны шақыру жолынан кейін тұрған операторға беріледі.

Функцияларды пайдалану программалаушыларға модульдік программалауды пайдалану мүмкіндігін береді, яғни программаны модульдерден құрастыруға болады. Әрбір функция белгілі бір тапсырманы (есепті) орындауы тиіс және функция аты сол тапсырманың мазмұнына сәйкес келуі керек. Сонда ғана бұл функцияны басқа программаларды жазу кезінде де қолдануға болады.

Функцияның мәнін қайтаруы

Үнемі функция нақты бір жұмысты атқаруды (мысалға, қандай да бір хабарламаға қорытынды), және де шақыру функциясының есептеу нәтижесін қайтаруды талап етеді. Шақыру кодының нәтижесін қайтаруды қамтамасыз ету үшін функцияда **return** операторын пайдалану керек.

Функцияның типі оның (**int**, **float**, **char**, т.б.) мағынасының қайтарылу типімен анықталады. Мысалы, егер функция **int** типін қайтаратын болса, онда сол типті функция идентификаторы алдында көрсету қажет:

```

int some_function(int value)
{
    // функция операторлары
}

```

Келесі **icube** функциясы параметр ретінде берілген санның кубын қайтарады. Мысалы, функцияға 5 саны берілсе, онда **icube** функциясы $5 \cdot 5 \cdot 5$ мәнін, яғни 125 мәнін қайтарады:

```

int icube(int value)
{
    return(value * value * value);
}

```

Сонымен, функция шақырылған соң, оның жұмыс нәтижесін қайтару үшін **return** операторы қолданылады (төмендегіні қараңыз). Программада функцияның қайтарған мәні айнымалыға меншіктеледі немесе басқа бір функцияның параметрі ретінде келесідей түрде көрсетілуі мүмкін:

```

result = icube(5);

```

немесе

```

Console.WriteLine("5 cubed is {0}", icube(5));

```

Енді толық программа қарастырайық.

```
using System;
namespace Example
{
    class Program
    {
        static int icube (int value)
        {
            return (value * value * value);
        }
        static void Main(string[] args)
        {
            Console.WriteLine("3 cubed is " + Program.icube(3));
            Console.WriteLine("5 cubed is " + Program.icube(5));
            Console.WriteLine("7 cubed is " + Program.icube(7));
            Console.ReadKey();
        }
    }
}
```

Программаны орындау нәтижелері:

```
3 cubed is 27
5 cubed is 125
7 cubed is 343
```

Функцияға берілетін мән

оның параметрі типіне сәйкес болу керек. Егер, мысалы, нақты (жылжымалы нүктелі сан) санның үшінші дәрежесін есептеу керек болса, онда типі басқа **fcube** функциясын құру керек:

```
float fcube(float value)
{
    return(value * value * value);
}
```

C# тілі **return** сөзі кездескенде, бірден функция жұмысын аяқтап, нәтижесін негізгі программаға (оны шақырушы функцияға) қайтарады. Функцияны орындау кезінде **return** операторынан кейін тұрған кез келген операторлар еленбей қалады. Программа өз жұмысын функцияны шақырған жолдан ары қарай жалғастыра береді.

Нақты және формальды(көрсетпелі) параметрлер

Формальды параметрлер деп функция анықталуында көрсетілетін параметрлер атауларын айтамыз. Мысалы, мынадай функцияда:

```
void job_information (int age, float salary, int number_job)
{
    // функциялар операторлары
}
```

age, salary, number_job атаулары берілген функция үшін формальды немесе көрсетпелі параметрлер болып саналады.

Функцияны программдан немесе басқа бір функциядан шақырған кезде, функцияға берілетін мәндер нақты немесе қойылмалы параметрлер болып табылады.

Төмендегі мысалда берілген 30, 42000.00 және 321 мәндері **job information** функциясын шақыру кезіндегі көрсетпелі параметрлерді алмастырып солардың орнына қойылатын нақты параметрлер болып табылады:

```
job_information (30,42000.00,321);
```

Функцияға тасымалданып қойылатын параметрлер сандық мән немесе айнымалы түрінде болуы мүмкін. Оларға қойылатын жалғыз талап – қойылмалы нақты параметрдің (немесе айнымалының) типі көрсетпелі параметр типімен бірдей болуы тиіс. Келесі код фрагменті қойылмалы параметр орнына айнымалыны пайдалану тәсілінкөрсетеді:

```
int workers_age = 30;  
float workers_salary = 42000.00;  
int job_number = 321; ,  
job_information {workers_age, workers_salary, job_number};
```

Функциялар мен айнымалылар атауларының әрекет ету аймағы

Программадағы функциялар мен айнымалылар атауларының өз әрекет ету (немесе көріну) аймағы, яғни оларды пайдалануға болатын, мәндері белгілі болып келетін программаның жұмыс аймағы болады.

Келесі программаны қарастырайық.

```
using System;  
namespace Example  
{  
    class Program  
    {  
        static void pr(int pr_count)  
        {  
            int count;  
            for (count = 1; count<=pr_count; count++)  
                Console.WriteLine('A');  
        }  
        static void Main()  
        {  
            int count;  
            for (count = 1; count <= 3; count++)  
            {  
                Console.WriteLine("Your computer should print {0}  
time(s)", count);  
                pr(count);  
            }  
            Console.WriteLine();  
        }  
    }  
}
```

Программаның орындалу нәтижелері:

```
Your computershould print 1 time(s)  
A  
Your computer should print 2 time(s)  
AA  
Your computer should print 3 time(s)  
AAA
```

Мұнда **pr** және **Main** функцияларының әрқайсысында **count** айнымалысы пайдаланылады, бірақ бұл айнымалылардың екеуі екі түрлі болып табылады, өйткені олардың әрқайсысының өз жұмыс аймағы бар. **Pr** функциясындағы **count** айнымалысы тек функцияның орындалуы кезінде ғана белгілі (көріну аймағы функция ішінде ғана) болып саналады. Сол сияқты екінші **count** айнымалысы тек **Main** функциясының орындалуы кезінде ғана жұмыс істей алады. Сондықтан **for** цикліндегі **count** айнымалысының өзгеру нәтижесі **Main** функциясындағы **count** айнымалысына ешқандай әсерін тигізбейді.

Айнымалының жұмыс істеу аймағын қарастыру кезінде локальдық және глобальдық айнымалылар түсінігі жиі пайдаланылады.

Локальдық (жергілікті) айнымалы – бұл белгілі бір функцияның ішкі операторларында ғана белгілі болып табылатын, әрекет ету аймағы шектеулі айнымалы.

Глобальды (ауқымды) айнымалы – бұл программаның барлық аймақтарында толық белгілі болып саналатын айнымалы.

Жоғарыла мысалда көрсетілген **count** айнымалысы тек қарастырылған функция ішінде ғана анықталған локальдық айнымалы болатын.

Лабораториялық жұмыс 2. Кластарды құру және оларға әдістері қосу

Лабораториялық жұмыстың мақсаты: Студенттерде класты сипаттау мен әдістерді құрумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Кластың өрістерін сипаттау;
- Кластың әдістерін жүзеге асыру.

Тапсырма: 1-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрыш қабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. үшбұрыш периметрін анықтау;
2. үшбұрыш ауданын анықтау;

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі әдістерді жүзеге асырыңыз:

1. прогрессияның таңдалған мүшесінің мәнін есептеу;
2. прогрессияның таңдалған қосындысын есептеу;

3-нұсқа

Жалақы класын құрыңыз. Класта келесідей өрістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), жұмысқа орналасу жылы, көтерме пайызы, табыс салығы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны, белгілі бір үшін есептелген жалақы мөлшері (жұмыс істеген күндерінің санына байланысты) және ұсталған ақша мөлшерлері. Келесі әдістерді жүзеге асырыңыз:

1. есептелген жалақы мөлшерін анықтау; ұсталған ақша мөлшерін анықтау;
2. қолға берілетін ақша көлемін және жұмыс өтілін анықтау.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі әдістерді жүзеге асырыңыз:

1. уақытқа секундтар қосу; уақыттан секундтарды азайту;
2. уақытты секундтарға айналдыру; уақытты минуттарға айналдыру (толық минутқа дейін дөңгелектей отырып).

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі әдістерді жүзеге асырыңыз:

1. бағаларды пайыздық көрсеткіштен балдық көрсеткішке ауыстыру (0-4 балл, академиялық саясат ережелеріне байланысты);
2. студенттің GPA балын есептеу;

6-нұсқа

Есепшот класын сипаттаңыз. Кластың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы мөлшері, шоттағы ақша көлемі. Келесі әдістерді жүзеге асырыңыз:

1. есепшоттан ақша сомасын алу; есепшотқа ақша сомасын салу;
2. белгілі бір мерзім үшін жинақталған сыйақы мөлшерін есептеу.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі әдістерді жүзеге асырыңыз:

1. берілген бөлшек санды бүтін санға көбейту;
2. объектіні әдеттегі бөлшек сан түрінде экранға шығару;

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс k , n екі бүтін сандық өрістен тұру керек ($k < n$). Төмендегі өрнек бойынша мәнді есептеп шығаратын әдіс құрыңыз:

1. $C(n, k) = n! / ((n - k)! \times k!)$
2. факториалды есептеу жеке әдіс ретінде өрнектеледі;

9-нұсқа

Тауар класын сипаттаңыз. Кластың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі әдістерді жүзеге асырыңыз:

1. тауар бағасын өзгерту; тауар санын өзгерту (кеміту немесе арттыру);
2. белгілі бір тауардың жалпы бағасын анықтау;

10-нұсқа

Телефон класын сипаттаңыз, ол телефон арқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, жеңілдіктер (пайызбен, мысалы, 10 минуттан артық сөйлесу үшін), сөйлесу уақыты (минутпен), төлемге есептелген ақша сомасы. Келесі әдістерді жүзеге асырыңыз:

1. сөйлесу бағасы мен жеңілдікті өзгерту;
2. төлемге арналған соманы есептеу;

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденең координаталары, терезе биіктігімен ені. Келесі әдістерді жүзеге асырыңыз:

1. терезені көлденең немесе тік бағытта жылжыту (терезе экран шекарасынан шығып кетпеуін бақылаңыз);
2. терезенің ұзындығымен енін өзгерту;

Лабораториялық жұмыс 3. Конструкторларды құру. Класс мүшелеріне жабық қатынасуды жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде класс конструкторларын құрумен және жабық класс мүшелеріне қатынасумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Класстың келісім бойынша конструкторын құру;
- Класстың параметрленген конструкторын құру;
- Класқа қасиеттер қосу.

Тапсырма: 2-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрыш қабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

3-нұсқа

Жалақы класын құрыңыз. Класста келесідей өрістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), жұмысқа орналасу жылы, көтерме пайызы, табыс салығы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны, белгілі ай үшін есептелген жалақы мөлшері (жұмыс істеген күндерінің санына байланысты) және ұсталған ақша мөлшерлері. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

6-нұсқа

Есепшот класын сипаттаңыз. Кластың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы мөлшері, шоттағы ақша көлемі. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс k, n екі бүтін сандық өрістен тұру керек ($k < n$). Келесі әдістерді құрыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

9-нұсқа

Тауар класын сипаттаңыз. Кластың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

10-нұсқа

Телефон класын сипаттаңыз, ол телефон арқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, жеңілдіктер (пайызбен, мысалы, 10 минуттан артық сөйлесу үшін), сөйлесу уақыты (минутпен), төлемге есептелген ақша сомасы. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденеңкоординаталары, терезе биіктігімен ені. Келесі әдістерді жүзеге асырыңыз:

1. класс өрістерін инициалдайтын конструкторлар құру;
2. класс өрістерін жабық түрде жариялап, оларға қатынасуға мүмкіндік беретін қасиеттерді құру.

Лабораториялық жұмыс 4. Объектілерді әдістерге беруді жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде класс әдістеріне параметр ретінде объектілерді беру арқылы жұмыс істеумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Кластың әдістеріне объектілерді параметр ретінде беру;
- Әдіс құрамында класс объектілерін өңдеу.

Тапсырма: 3-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрыш қабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. екі үшбұрыш аудандарының айырмасын анықтау.

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі әдістерді жүзеге асырыңыз:

1. екі геометриялық прогрессияның сәйкес мүшелерінің мәнін салыстыру.

3-нұсқа

Жалақы класын құрыңыз. Класста келесідей өрістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), жұмысқа орналасу жылы, көтермепайызы, табыс салығы, есептеуді жүргізу айы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны. Келесі әдістерді жүзеге асырыңыз:

1. белгілі бір қызметкердің екі айдағы жалпы жалақы мөлшерін есептеу.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі әдістерді жүзеге асырыңыз:

1. екі уақыт мезетінің арасындағы толық секундтар санын есептеу.

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі әдістерді жүзеге асырыңыз:

1. екі студенттің GPA балдарын салыстыру.

6-нұсқа

Есепшот класын сипаттаңыз. Кластың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы мөлшері, шоттағы ақша көлемі. Келесі әдістерді жүзеге асырыңыз:

1. бір есепшоттан екіншісіне ақша сомасын аудару.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі әдістерді жүзеге асырыңыз:

1. екі бөлшек санды қосу.

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс k , n екі бүтін сандық өрістен тұру керек ($k < n$). Төмендегі өрнек бойынша мәнді есептеп шығаратын әдіс құрыңыз: $C(n, k) = \frac{n!}{(n-k)! \times k!}$

1. екі үйлестіру мәнін салыстыру.

9-нұсқа

Тауар класын сипаттаңыз. Кластың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі әдістерді жүзеге асырыңыз:

1. екі тауардың жалпы бағасын салыстыру.

10-нұсқа

Телефон класын сипаттаңыз, ол телефонарқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, жеңілдіктер (пайызбен, мысалы, 10 минуттан артық сөйлесу үшін), сөйлесу уақыты (минутпен), төлемге есептелген ақша сомасы. Келесі әдістерді жүзеге асырыңыз:

1. екі сөйлесудің қосынды төлем сомасын есептеу.

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденең координаталары, терезе биіктігімен ені. Келесі әдістерді жүзеге асырыңыз:

1. екі терезенің қиылысу аймағының ауданын есептеу.

Лабораториялық жұмыс 5. Әдістерден объектілерді қайтаруды жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде класс әдістерінен объектілерді қайтарумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Кластың әдісінің құрамында класс объектілерін құру;
- Әдістен объектіні негізгі программаға қайтару.

Тапсырма: 4-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрыш қабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. Берілген үшбұрыштың ең ұзын қабырғасының ұзындығын `l` таңып, қабырғаларының ұзындығы `l` таң мәніне тең болатын тең қабырғалы үшбұрышты құруға арналған әдіс жазыңыз.

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі әдістерді жүзеге асырыңыз:

1. екі геометриялық прогрессияның ішінде екінші қосындысының мәні кіші болатынын қайтаратын әдісті құрыңыз.

3-нұсқа

Жалақы класын құрыңыз. Класта келесідей өрістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), табыс салығы, есептеуді жүргізу айы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны. Келесі әдістерді жүзеге асырыңыз:

1. қызметкерлер массивінің ішінен ағымдағы ай үшін есептелген ақша мөлшері ең аз болатын жұмысшыны қайтаратын әдіс құрыңыз.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі әдістерді жүзеге асырыңыз:

1. үш уақыт мезетінің ішіндегі өсуі бойынша екіншісін қайтаратын әдіс құрыңыз.

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі әдістерді жүзеге асырыңыз:

1. екі студенттің ішіндегі GPA балы артық болған студентті қайтаратын әдіс құрыңыз.

6-нұсқа

Есепшот класын сипаттаңыз. Кластың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы мөлшері, шоттағы ақша көлемі. Келесі әдістерді жүзеге асырыңыз:

1. екі есепшоттан ортақ бір есепшот құратын әдіс жазыңыз, есепшот параметрлері бастапқы ақша сомасы үлкен болған есепшоттан меншіктеледі, ал қорытынды ақша сомасы екі есепшоттың қосынды ақшасы ретінде анықталады.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі әдістерді жүзеге асырыңыз:

1. екі бөлшек санды азайту нәтижесін бөлшек сан ретінде қайтару.

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс k , n екі бүтін сандық өрістен тұру керек ($k < n$). Төмендегі өрнек бойынша мәнді есептеп шығаратын әдіс құрыңыз: $C(n, k) = n! / ((n-k)! \times k!)$

1. екі үйлестірудің кішісін қайтаратын әдіс құрыңыз.

9-нұсқа

Тауар класын сипаттаңыз. Кластың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі әдістерді жүзеге асырыңыз:

1. жалпы бағасы үлкен болатын тауарды қайтаратын әдісті құрыңыз

10-нұсқа

Телефон класын сипаттаңыз, ол телефонарқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, жеңілдіктер (пайызбен, мысалы, 10 минуттан артық сөйлесу үшін), сөйлесу уақыты (минутпен), төлемге есептелген ақша сомасы. Келесі әдістерді жүзеге асырыңыз:

1. екі сөйлесудің ішіндегі көбірек жеңілдікке ие болғанын қайтаратын әдіс құрыңыз

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденең координаталары, терезе биіктігімен ені. Келесі әдістерді жүзеге асырыңыз:

1. екі терезені қамтитын жаңа терезені қайтаратын әдіс құрыңыз.

6 лабораториялық жұмыс. Әдістерді асыражүктеуді жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде класс әдістерін асыра жүктеумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Асыра жүктелген әдістерді құру;
- Көшіруконструкторын құру.

Тапсырма: 5-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрышқабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. үшбұрышпараметрлерін еселеудің әртүрлі нұсқаларын әдістерді асыра жүктеу жолымен құру.

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. геометриялық прогрессияның n -ші мүшесінің мәнін анықтаудың екі әдісін асыра жүктеңіз: n санын аргументарқылы анықтай отырып есептеу және параметрсіздік көмегімен 10-шы мүшесінің мәнін есептеу.

3-нұсқа

Жалақы класын құрыңыз. Класта келесідей өрістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), жұмысқа орналасу жылы, есептеу орындалған ай, табыс салығы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. белгілі бір ай үшін қолға берілетін жалақы мөлшерін есептеудің әдісін асыра жүктеу: сыйақы тағайындау арқылы және сыйақы тағайындаусыз.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. екі уақыт мерзімі арасындағы айырманы табу әдісін асыра жүктеңіз: а) берілген екі уақыт мерзімінің айырмасын табу; б) берілген уақыт мерзімінің ағымдағы уақыттан айырмасын анықтау.

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. студенттің орташа балын есептеу әдісін асыра жүктеңіз: а) студенттің 5 пән үшін орташа балын параметрсіздік көмегімен есептеу;
б) студенттің берілген пәндер саны үшін орташа балын параметрлі әдіс көмегімен есептеу.

6-нұсқа

Есепшот класын сипаттаңыз. Кластың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы

мөлшері, шоттағы ақша көлемі. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. есепшоттағы келешек ақша мөлшерін есептеу әдісін асыра жүктеңіз:
 - а) параметрсізәдістің көмегімен есепшоттағы бір жылдан кейін жиналатын ақша мөлшерін есептеу;
 - б) параметрлі әдістің көмегімен есепшоттағы ақша мөлшерін белгілі айлар саны үшін есептеу.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. бөлшек санды бүтін санға және басқа бөлшек санға қосу әдістерін асыра жүктеу арқылы құрыңыз.

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс k , n екі бүтін сандық өрістен тұру керек ($k \leq n$). Төмендегі өрнек бойынша мәнді есептеп шығаратынәдіс құрыңыз: $C(n,k) = n! / ((n-k)! \times k!)$

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. үйлестіруді салыстыру әдісін асыра жүктеңіз: а) екі үйлестіру объектісінің айырымын есептеу; б) берілген үйлестіру мәнін нақты санмен салыстыру.

9-нұсқа

Тауар класын сипаттаңыз. Кластың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі әдістердіжүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. жалпы бағаны есептеу әдісін асыра жүктеңіз: а) параметрлі әдіс берілген тауардың жалпы бағасын есептейді; б) параметрсізәдіс қоймадағы бар тауарлардыңжалпы бағасын есептейді.

10-нұсқа

Телефон класын сипаттаңыз, ол телефонарқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, сөйлесу уақыты (минутпен).Келесі әдістерді жүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. сөйлесу бағасын есептеу әдісін асыра жүктеңіз: а) жеңілдіксіз;
б) жеңілдік тағайындау арқылы.

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденең координаталары,терезе биіктігімен ені. Келесі әдістердіжүзеге асырыңыз:

1. класс үшін бос, параметрлі және көшіру конструкторларын құру;
2. терезе параметрлерін өзгерту әдісін асыра жүктеңіз: бүтін санға көбейту арқылы және нақты санға көбейту арқылы масштабтау.

7 лабораториялық жұмыс. Операторларды асыра жүктеуді жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде класс құрамында операторларды асыра жүктеумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Арифметикалық операторларды асыра жүктеу;
- Қатынастық операторларды асыра жүктеу;
- True/false операторларын асыра жүктеу.

Тапсырма: 6-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқа бойынша орындаңыз.

Тапсырмалар

1-нұсқа

Үшбұрыш класын құрыңыз. Класс үшбұрышқабырғаларының ұзындығын анықтайтын 3 өрістен тұрады. Келесі операторларды асыра жүктеңіз:

1. үшбұрыштардың салыстыру ($>$, $<$) операторларын асыра жүктеңіз, ол үшбұрыштар аудандарын салыстыру арқылы жүзеге асырылатын болады;
2. үшбұрыш параметрлерін еселеуді көбейту ($*$) операторын асыра жүктеу арқылы (бүтін санға көбейту) орындаңыз.

2-нұсқа

Геометриялық прогрессия класын сипаттаңыз. Өрістері: прогрессияның алғашқы элементі және тұрақты еселік. Келесі операторларды асыра жүктеңіз:

1. екі геометриялық прогрессияны теңдікке салыстыру ($==$, $!=$) операторларын асыра жүктеңіз. Егер екі прогрессияның 10-шы қосындысы тең болса, онда прогрессиялар тең болып саналады деп тұжырымдаңыз.
2. геометриялық прогрессия класы үшін бірге арттыру ($++$) операторын асыра жүктеңіз, операция нәтижесінде объектінің алғашқы мүшесімен еселігінің мәндері бірге арту керек.

3-нұсқа

Жалақы класын құрыңыз. Класта келесі дейерістер болу керек: қызметкердің аты-жөні, жалақы сомасы (оклад), жұмысқа орналасу жылы, есептеу орындалған ай, табыс салығы, көтерме пайызы, ай ішіндегі жалпы жұмыс күндерінің саны, ай ішіндегі жұмысқа келген күндерінің саны. Келесі операторларды асыра жүктеңіз:

1. жалақыны өсіру үшін ($++$) операторын асыра жүктеңіз, жалақыны өсіру әдісінің нәтижесінде жалақы көтермепайызға арту керек;
2. жалақы объектілерін салыстыру үшін ($>$, $<$) операторларын асыра жүктеңіз, операция екі объект үшін жұмысқа келген күндерінің санын ескере отырып, қолға берілетін жалақы мөлшерлерін салыстырады деп тұжырымдаңыз.

4-нұсқа

Уақыт класын сипаттаңыз. Ол үш бүтін сандық өріс арқылы беріледі: сағат, минут, секунд. Келесі операторларды асыра жүктеңіз:

1. екі уақыт мерзімін салыстыру үшін ($>$, $<$) операторларын асыра жүктеңіз;
2. уақытты терістеу үшін унарлы ($-$) операторын асыра жүктеңіз, ол күннің алғашқы бөлігіндегі уақытты екінші бөлігіне және керісінше ауыстырады деп тұжырымдаңыз (мысалы, 02:00:00 = 14:00:00, 15:00:00 = 03:00:00).

5-нұсқа

Студентті сипаттайтын класс құрыңыз. Өрістері: жеке нөмірі, тегі, 5 пән бойынша бағасы (0-100%), 5 пән бойынша кредит саны. Келесі операторларды асыра жүктеңіз:

1. студент класы үшін **true/false** операторларын асыра жүктеңіз, егер студент ешбір пән бойынша 50-ден төменбалл жинамаған болса, онда ол «ақиқат студент», кері жағдайда «жалған студент» болып есептеледі;
2. екі студенттің үлгерімін салыстыру үшін (**>**, **<**) операторларын асыра жүктеңіз, «жақсырақ» бағалар саны артық болған студенттің үлгерімі жоғары болып есептеледі деп тұжырымдаңыз.

6-нұсқа

Есепшот класын сипаттаңыз. Класстың 4 өрісі болу керек: есепшот иесінің тегі, есепшот нөмірі, сыйақы мөлшері, шоттағы ақша көлемі. Келесі операторларды асыра жүктеңіз:

1. есепшот класы үшін (**++**) операторын асыра жүктеңіз, операция нәтижесінде бір ай өткеннен кейінгі шоттағы ақша көлемі есептеледі деп тұжырымдаңыз;
2. есепшот класы үшін (**-**) операторын асыра жүктеңіз, операция нәтижесінде бір есепшотта бар ақша көлемі екіншісіне толықтай аударылады деп тұжырымдаңыз.

7-нұсқа

Бөлшек сан класын сипаттаңыз. Сан бүтін және бөлшек бөліктерін беретін екі бүтін сандық өріс арқылы беріледі. Келесі операторларды асыра жүктеңіз:

1. екі бөлшек санның қосындысын есептеу үшін (**+**) операторын асыра жүктеңіз;
2. бөлшек сан объектісінің ақиқат/жалған болуын тексеру үшін **true/false** операторларын асыра жүктеңіз, бөлімі нөлге тең болатын объект жалған болады деп тұжырымдаңыз.

8-нұсқа

Үйлестіру класын сипаттаңыз. Класс **k, n** екі бүтін сандық өрістен тұру керек (**k < n**). Төмендегі өрнек бойынша мәнді есептеп шығаратын әдіс құрыңыз: $C(n, k) = n! / ((n - k)! \times k!)$. Келесі операторларды асыра жүктеңіз:

1. екі үйлестіру объектісін салыстыру үшін (**>**, **<**) операторларын асыра жүктеңіз, операция нәтижесінде екі объект үшін есептелген өрнек мәндері салыстырылады деп тұжырымдаңыз;
2. үйлестіру класы үшін унарлы (**-**) операторын асыра жүктеңіз, операция нәтижесінде **k** айнымалысының мәні **n - k** тең болады деп тұжырымдаңыз.

9-нұсқа

Тауар класын сипаттаңыз. Класстың өрістері: тауар аты, тауар бағасы, қоймадағы тауар саны. Келесі операторларды асыра жүктеңіз:

1. класс үшін салыстыру (**>**, **<**) операторларын асыра жүктеңіз. Операция екі тауардың берілген саны үшін жалпы ақша сомасын салыстыру керек;
2. класс үшін (***** - объектіні бүтін санға көбейту) операторын асыра жүктеңіз, операция нәтижесінде тауар бағасы көрсетілген пайызға артады деп тұжырымдаңыз.

10-нұсқа

Телефон класын сипаттаңыз, ол телефон арқылы сөйлесудің бағасын белгілейді. Оның өрістері: телефон нөмірі, 1 минут сөйлесу бағасы, сөйлесу уақыты (минутпен). Келесі операторларды асыра жүктеңіз:

1. телефон класы үшін **true/false** операторларын асыра жүктеңіз, сөйлесу уақыты 1 минуттан кем болатын объект жалған деп тұжырымдаңыз;
2. телефон класы үшін (**+** екі телефон объектісін қосу) операторын асыра жүктеңіз, операция нәтижесінде екі объект үшін есептелген сөйлесу бағасының қосындысы есептеледі деп тұжырымдаңыз.

11-нұсқа

Терезе класын сипаттаңыз. Өрістері: терезе тақырыбы, сол жақ жоғарғы төбесінің тік және көлденең координаталары, терезе биіктігі мен ені. Келесі операторларды асыра жүктеңіз:

1. екі терезенің қосындысын анықтау (**+**) операторын асыра жүктеңіз. Операция нәтижесінде екі терезенің орналасу ауданын толық қамтитын жаңа терезе құрылу керек;
2. терезе параметрлерін өзгерту әдісін асыра жүктеңіз: бүтін санға көбейту арқылы және нақты санға көбейту арқылы масштабтау.

Лабораториялық жұмыс 8. Кеңейтуші әдістерді жүзеге асыру

Лабораториялық жұмыстың мақсаты: Студенттерде класс қызметін кеңейтуші әдісті құрумен байланысты дағдыларды қалыптастыру.

Лабораториялықжұмысты орындаунәтижесінде студенттер келесі қабілеттерге ие болады:

- Мұралауды рұқсат етпейтін класты құру;
- Класты кеңейтуші әдісті құру;
- Кеңейтуші әдісті пайдалану.

Тапсырма: 7-дәріс материалдарын қайта қарастырып, 1-7 лабораториялық жұмыстарда берілген әдістердің бірін кеңейтуші әдіс ретінде жүзегеасырыңыз.

9 лабораториялық жұмыс. Мұралауды жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде кластарды мұралаумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Базалық класты құру;
- Базалық кластан мұралайтын туынды класты құру.

Тапсырма: 8-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқаңызға сәйкес орындаңыз.

Тапсырмалар

1-нұсқа

Рaіg базалық класын анықтаңыз, өрістері: екі бүтін сан. Өрістерді теңдікке тексеру және $(a,b)-(c,d)=(a-b,c-d)$ ережесі бойынша екі объектінің айырмасын табу әдістерін жүзеге асырыңыз. Туынды Рационал сан класын құрыңыз, құрамында объектілерді $(a,b)+(c,d)=(ad+bc,bd)$ ережесі бойынша қосу әдісін анықтап, объектілерді азайту әдісін $(a,b)-(c,d)=(ad-bc,bd)$ ережесіне сәйкес қайта анықтаңыз.

2-нұсқа

Радиус өрісі арқылы анықталатын Шеңбер базалық класын құрып, оның құрамында ауданды есептеуге және шеңбер радиусының мәнін еселеуге арналған виртуалды әдістерді анықтаңыз. Бұл кластан туынды Сақина класын құрыңыз, оның құрамында қосымша өріс – ішкі радиус жарияланады және базалық кластың виртуалды әдістері қайта анықталады, мұндағы сақина ауданын сыртқы және ішкі радиустардың айырмасы ретінде анықтаңыз, ал сақинаны еселеу функциясы екі радиусты қатарымен еселейтін болсын.

3-нұсқа

Рaіg класын құрыңыз, өрістері: екі бүтін сандық өріс. Өрістердің мәндерін өзгертуге арналған әдісті құрыңыз, екі объектіні салыстыруға арналған $>$, $<$ операцияларын асыра жүктеңіз (егер $\text{obj1.p1} > \text{obj2.p1}$ немесе $\text{obj1.p1} = \text{obj2.p1}$ және $\text{obj1.p2} > \text{obj2.p2}$). Туынды Fraction класын анықтаңыз, өрістері: санның бүтін бөлігі және бөлшегі, екі объектіні салыстыру әдістерін қайта анықтаңыз.

4-нұсқа

Ұзындығы мен енін білдіретін өрістер арқылы анықталатын Тіктөртбұрыш базалық класын құрып, оның құрамында ауданы мен периметрін есептеуге арналған виртуалды әдістерді анықтаңыз. Бұл кластан туынды Трапеция класын құрыңыз, оның құрамында қосымша екі қабырғасын білдіретін өрістер беріледі және базалық кластың виртуалды әдістері қайта анықталады.

5-нұсқа

Үштік базалық класын құрыңыз, өрістері: үш бүтін сан. Екі үштікті салыстыру әдісін құрыңыз (өріс мәндерінің қосындысы артық болған үштік үлкен болып саналады). Туынды Уақыт класын құрыңыз, екі уақытты салыстыру әдістерін қайта анықтаңыз.

6-нұсқа

Үлкен және кіші жартылай өстерінің ұзындықтарын білдіретін жабық өрістер арқылы анықталған Эллипс базалық класын құрып, оның құрамында ауданды есептеуге және эллипс өстерінің ұзындықтарын параллель еселеуге арналған әдістерді анықтаңыз. Базалық кластың жабық мүшелеріне қатынас құруға мүмкіндік беретін виртуалды қасиеттерді жазыңыз. Эллипс класынан туынды Шеңбер класын құрыңыз, оның құрамындағы қасиеттерді екі жартылай өстерінің ұзындықтарын бір-біріне тәуелді түрде тағайындайтындай (мәндері тең болу үшін) етіп қайта анықтаңыз.

7-нұсқа

Раіг класын құрыңыз, өрістері: екі бүтін сандық өріс. Сандарды көбейту әдісін құрыңыз. Туынды Тіктөртбұрышты класын анықтаңыз, өрістері катеттері. Үшбұрыштың гипотенузасы мен ауданын табу әдістерін құрыңыз.

8-нұсқа

Ұзындығы мен енін білдіретін өрістер арқылы анықталатын Тіктөртбұрыш базалық класын құрып, оның құрамында ауданы мен периметрін есептеуге арналған виртуалды әдістерді анықтаңыз. Бұл кластан туынды Дөңгелектенген_төртбұрыш класын құрыңыз, оның құрамында қосымша өріс – дөңгелектеу радиусы (r) беріледі және базалық кластың виртуалды әдістері қайта анықталады. Дөңгелектенген тіктөртбұрыштың периметрі $- 8 \cdot + 2 \cdot$, ал радиусы $- 4 \cdot +$ өрнектері арқылы есептеледі, мұндағы r – әдеттегі тіктөртбұрыштың периметрі, S – әдеттегі тіктөртбұрыштың ауданы.

9-нұсқа

Үштік класын құрыңыз, өрістері: 3 бүтін сан. Өрістердің мәндерін бірге арттыру әдісін анықтаңыз. ТуындыДата класын құрыңыз, өрістері: жыл, ай, күн. Өрістердің мәндерін бірге арттыру әдісін қайта анықтаңыз және мерзімді күнге арттыру әдісін құрыңыз.

10-нұсқа

Студент базалық класын құрыңыз, оның өрістері: аты-жөні, мамандығы, курс, GPA балы. Бұл класс үшін келесі виртуалды әдістерді анықтаңыз: GPA балы 2.0-ден артық болса, курсты бірге арттыратын келесі курсқа көшіру әдісі және оның орта балы бойынша алатын стипендиясының көлемін теңгемен көрсететін әдіс. Студент класынан туынды Ақылы_оқуда класын құрыңыз, оның құрамында оқу ақысының төленгенін / төленбегенін көрсететін қосымша өрісті анықтаңыз. Туынды класс құрамында базалық кластың виртуалды әдістерін қайта анықтаңыз: келесі курсқа көшіру әдісінде студенттің GPA балы және оқу ақысының төленуі тексеріледі, ал стипендия мөлшерін анықтайтын әдіс әрқашан 0 теңгені қайтарады.

11-нұсқа

Адам класын құрыңыз, өрістері: аты, жасы, салмағы. Өрістердің мәндерін қайта тағайындау әдістерін құрыңыз. ТуындыСтудент класын анықтаңыз, өрістері: курсы. Студенттің курсын жоғарылату әдісін анықтаңыз.

12-нұсқа

Раіг класын құрыңыз, өрістері: екі бүтін сан. Өрістердің мәндерін өзгерту және сандардың көбейтіндісін табу әдістерін құрыңыз. Туынды Тіктөртбұрыш класын құрыңыз, өрістері: қабырғаларының ұзындықтары. Тіктөртбұрыштың периметрі мен ауданын есептеу әдістерін құрыңыз.

10 лабораториялықсабақ. Абстрактілі кластарды құру.

Лабораториялықжұмыстың мақсаты: Студенттерде абстрактілі кластардан мұралаумен байланысты дағдыларды қалыптастыру.

Лабораториялықжұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Абстрактілі базалық класты құру;
- Базалық кластан мұралайтын туынды класты құру.

Тапсырма: 9-дәріс материалдарын қайта қарастырып, төменде берілген тапсырмаларды нұсқаңызғасәйкес орындаңыз.

Тапсырмалар

1-нұсқа

Фигура абстрактылы базалық класын құрып, оның құрамында аудан мен периметрді есептейтін абстрактылы әдістерді анықтаңыз. Бұл кластан туынды Шеңбер және Трапеция кластарын құрып, олардың құрамында аудан мен периметрді есептеу әдістерін қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

2-нұсқа

Ағғау абстрактылы базалық класын құрыңыз. Құрамында қосындыны табу және жиымды элементтік өңдеу әдістерін жариялаңыз. AndArray және OrArray туынды кластарын құрыңыз. AndArray класында қосындыны табу операциясы жиындардың қиылысуы ретінде, ал өңдеу әдісі квадрат түбірді табу арқылы жүзеге асырылады. OrArray класында қосындыны табу операциясы жиындардың бірігуі ретінде, ал өңдеу әдісі логарифмді табу арқылы жүзеге асырылады.

3-нұсқа

Валюта абстрактылы базалық класын құрып, оның құрамында теңгеге аудару және экранға шығару абстрактылы әдістерін анықтаңыз. Бұл кластан туынды Доллар және Рубль кластарын құрып, олардың құрамында теңгеге аудару және экранға шығару әдістерін қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

4-нұсқа

Pair абстрактілі базалық класын анықтап, құрамында + және – операцияларын жариялаңыз. Fuzzy және Fraction туынды кластарын құрыңыз, Fuzzy класының өрістері: $\langle A-a1, A, A+a2 \rangle$ үштігі, операцияларды жүзеге асыру ережелері: қосу: $A+B=(A+B-a1-b1, A+B, A+B+a2+b2)$; азайту: $A-B=(A-B-a1-b1, A-B, A-B+a2+b2)$. Fraction класының өрістері: санның алымы мен бөлімін білдіретін екі бүтін сан, қосу және азайту операцияларын жүзеге асырыңыз.

5-нұсқа

Үштік (үш бүтін сандық өрістен тұратын класс) абстрактылы базалық класын құрып, оның құрамында әрбір өрістің мәнін бірге арттыратын жеке абстрактылы әдістерді анықтаңыз. Бұл кластан туынды Мерзім (Дата) және Уақыт кластарын құрып, олардың құрамында әрбір өріс мәнін бірге арттыратын әдістерді қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

6-нұсқа

Прогрессия абстрактылы базалық класын құрыңыз, құрамында n-ші элементті және n-ші қосындыны табу операцияларын жариялаңыз. Туынды кластары: арифметикалық және геометриялық. Туынды кластарда операциялардың жүзеге асырылу ережелерін анықтаңыз.

7-нұсқа

Түбір абстрактылы базалық класын құрып, оның құрамында түбірлерді есептеп, нәтижені экранға шығару абстрактылы әдістерін анықтаңыз. Бұл кластан туынды Сызықтық және Квадрат кластарын құрып, олардың құрамында сәйкесінше, сызықтық және квадрат теңдеулер түбірлерін табу және экранға шығару әдістерін қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

8-нұсқа

Рационал абстрактілі базалық класын анықтап, құрамында $+$ және $-$ операцияларын жариялаңыз. Қаражат және Комплекс туынды кластарын құрыңыз, Қаражат класының өрістері: теңге мен тиын түріндегі қаражат, қосу және азайту операцияларын жүзеге асырыңыз. Комплекс класының өрістері: санның нақты және жорамал бөліктері, қосу және азайту операцияларын келесі ережелер бойынша жүзеге асырыңыз: қосу: $(a,b)+(c,d)=(a+b,c+d)$; азайту: $(a,b)-(c,d)=(a-b,c-d)$;

9-нұсқа

Сан абстрактылы базалық класын құрып, оның құрамында негізгі арифметикалық операцияларды орындайтын абстрактылы әдістерді анықтаңыз. Бұл кластан туынды Бүтін және Бөлшек (санның алымы мен бөлімін білдіретін екі бүтін сандық өрістен тұрады) кластарын құрып, олардың құрамында арифметикалық операцияларды орындау әдістерін қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

10-нұсқа

Үштік абстрактілі базалық класын құрыңыз. Құрамында өрістердің мәндерін бірге арттыру әдістерін жариялаңыз. Туынды Дата және Уақыт кластарын құрып, өрістер мәндерін бірге арттыру әдістерін анықтаңыз.

11-нұсқа

Норма абстрактылы базалық класын құрып, оның құрамында норманы және модульді есептеуге арналған абстрактылы әдістерді анықтаңыз. Бұл кластан туынды Комплекс_сан және Үшөлшемді_вектор кластарын құрып, олардың құрамында норманы және модульді есептеуге арналған әдістерді қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

12-нұсқа

Геометриялық_дене абстрактылы базалық класын құрып, оның құрамында толық бетінің ауданын және көлемін есептеуге арналған абстрактылы әдістерді анықтаңыз. Бұл кластан туынды Цилиндр және Текше кластарын құрып, олардың құрамында денелердің толық бетінің ауданын және көлемін есептеуге арналған әдістерді қайта анықтаңыз. Аталған кластар құрамында қажетті өрістерді және оларды базалық немесе туынды класта анықтаудың дұрыстығы туралы шешімді өзіңіз қабылдаңыз.

11 лабораториялық сабақ. Интерфейстерді құру және жүзеге асыру.

Лабораториялықжұмыстың мақсаты: Студенттерде интерфейссті құрумен және жүзеге асырумен байланысты дағдыларды қалыптастыру.

Лабораториялықжұмысты орындаунәтижесінде студенттер келесі қабілеттерге ие болады:

- Интерфейсті құру;
- Интерфейсті жүзеге асыратын класты құру.

Тапсырма: 10-дәріс материалдарын қайта қарастырып, абстрактілі кластарға байланыстыберілген тапсырмаларды интерфейс түрінде қайта орындаңыз.

12 лабораториялық сабақ. Аластамалық жағдайларды өңдеу.

Лабораториялық жұмыстың мақсаты: Студенттерде аластамалық жағдайларды өңдеумен байланысты дағдыларды қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Кітапханалық аластамаларды өңдеу;
- Catch блогын пайдаланудың әртүрлі нұсқаларын қарастыру.

Тапсырма: 11-дәріс материалдарын қайта қарастырып, өзіңіздің нұсқаңызға сәйкес кластың алдын ала анықталған аластамаларын өңдеңіз.

13 лабораториялық сабақ. Туынды аластамалар кластарын құру.

Лабораториялық жұмыстың мақсаты: Студенттерде аластамалық жағдайларды өңдеумен байланысты дағдыларды қалыптастыру.

Лабораториялықжұмысты орындаунәтижесінде студенттер келесі қабілеттерге ие болады:

- Туынды аластамалар класын құру;
- Туынды аластамаларды өңдеу.

Тапсырма: 12-дәріс материалдарын қайта қарастырып, өзіңіздің нұсқаңызға сәйкес кластың құрамында анықталатын, бірақ аластамалар кеңістігінде анықталмаған аластамаларды өңдеуге арналған класты құрып, туындыаластамаларды өңдеңіз.

14 лабораториялық сабақ. Оқиғаларды жүзеге асыру.

Лабораториялық жұмыстың мақсаты: Студенттерде оқиғаларды жүзеге асырумен байланысты дағдыларды қалыптастыру.

Лабораториялықжұмысты орындаунәтижесінде студенттер келесі қабілеттерге ие болады:

- Класс құрамында оқиғаны құру;
- Оқиғаны жүзеге асыру.

Тапсырма: 13-дәріс материалдарын қайта қарастырып, өзіңіздің нұсқаңызға сәйкес кластың құрамында оқиғаны құрып, өңдеңіз.

15 лабораториялық сабақ. Анонимдік функцияларды құру

Лабораториялық жұмыстың мақсаты: Студенттерде лямбда-өрнектерді пайдалану дағдыларын қалыптастыру.

Лабораториялық жұмысты орындау нәтижесінде студенттер келесі қабілеттерге ие болады:

- Әдістерді лямбда-өрнектер түрінде жүзеге асыру;
- Лямбда-өрнектерде параметрлерді пайдалану.

Тапсырма: 14-дәріс материалдарын қайта қарастырып, өзіңіздің нұсқаңызға сәйкес кластың құрамындағы қарапайым әдістерді лямбда-өрнек түрінде жүзеге асырыңыз.

БІЛІМДІ ТЕКСЕРУ МАТЕРИАЛДАРЫ

Тест тапсырмалары.

1–10. C# тілі және бағдарламалау ортасы

1. C# бағдарламасын орындайтын .NET ортасының негізгі құрамдасы қалай аталады?

- A) HTML
- B) CLR
- C) SQL
- D) CSS

2. C# бастапқы коды сақталатын файлдың кеңейтімі қандай?

- A) .cs
- B) .exe
- C) .html
- D) .dll

3. C# кітапханасы компиляцияланған кезде қандай файл жасалуы мүмкін?

- A) .docx
- B) .jpg
- C) .dll
- D) .css

4. C# жобасын құрастыру үшін .NET CLI-де қандай команда қолданылады?

- A) dotnet data
- B) dotnet draw
- C) dotnet translate
- D) dotnet build

5. C# тілінде бас және кіші әріптер ажыратыла ма?

- A) Иә, ажыратылады
- B) Жоқ, ажыратылмайды
- C) Тек түсініктемелерде ажыратылады
- D) Тек файл кеңейтімінде ажыратылады

6. C# тілінде біржолдық түсініктеме қалай басталады?

- A) <!--
- B) //
- C) **
- D) ##

7. Көпжолдық түсініктеменің дұрыс жазылуын көрсетіңіз.

- A) <comment>...</comment>
- B) //...//
- C) /* ... */
- D) #...#

8. C# тілінде көптеген қарапайым операторлар қандай таңбамен аяқталады?

- A) :
- B) ,
- C) .
- D) ;

9. C# тіліндегі классикалық кіру нүктесінің дұрыс жазылуы қандай?

- A) static void Main(string[] args)
- B) public main(String args)
- C) static int start()
- D) void Main[]

10. C# кодын жазу, іске қосу және жөндеу үшін қолданылатын IDE қайсысы?

- A) Microsoft Paint
- B) Visual Studio

- C) VLC
- D) Adobe Reader

11–20. Мәліметтер типтері және айнымалылар

11. Бүтін санды сақтайтын C# типін көрсетіңіз.

- A) string
- B) bool
- C) int
- D) void

12. Логикалық мәндерді сақтайтын C# типі қандай?

- A) char
- B) double
- C) long
- D) bool

13. char типі қандай мәнді сақтайды?

- A) Бір UTF-16 код бірлігін
- B) Бүтін сандар массивін
- C) Кез келген ұзындықтағы мәтінді
- D) Класс атауын

14. double типінің негізгі қызметі қандай?

- A) Логикалықмәнді сақтау
- B) Жылжымалы нүктелі санды сақтау
- C) Бір символды сақтау
- D) Әдісті анықтау

15. Қайсысы C# тіліндегі мәндік типке жатпайды?

- A) int
- B) char
- C) string
- D) bool

16. Айнымалыны дұрыс жариялау нұсқасын көрсетіңіз.

- A) age int = 20;
- B) int = age 20;
- C) integer age := 20;
- D) int age = 20;

17. const int MAX = 100; жарияланымының мағынасы қандай?

- A) MAX мәнін кейін өзгертуге болмайды
- B) MAX міндетті түрде нөлге тең
- C) MAX тек мәтін сақтайды
- D) MAX автоматты түрде массивке айналады

18. int типінің көлемі қанша бит?

- A) 16
- B) 32
- C) 64
- D) 8

19. double x = 7.9; int y = (int) x; орындалғаннан кейін y мәні қандай?

- A) 8
- B) 7.9
- C) 7
- D) 0

20. Жергілікті айнымалыға мән тағайындамай, оның мәнін оқуға әрекет жасалса не болады?

- A) Ол автоматты түрде 1 болады
- B) Ол міндетті түрде null болады

C) Бағдарлама қалыпты орындалады

D) Компиляция қатесі пайда болады

21–30. Мәліметтерді енгізу және шығару

21. Мәтінді шығарып, келесі жолға көшетін әдіс қандай?

A) Console.WriteLine()

B) Console.Write()

C) Console.ReadLine()

D) Console.Print()

22. Console.Write("C#"); командасы не істейді?

A) Мәтінді файлға сақтайды

B) Мәтінді шығарады, соңына жаңа жол қоспайды

C) Мәтінді өшіреді

D) Пайдаланушыдан мәтін сұрайды

23. Консольден енгізілген жолды бүтін санға түрлендіру үшін қандай әдіс қолданылады?

A) Math.Sqrt()

B) Thread.Sleep()

C) int.Parse()

D) object.ToString()

24. Console класын пайдалануға мүмкіндік беретін кеңістікті қосу директивасы қандай?

A) include System;

B) import System;

C) package System;

D) using System;

25. Пернетақтадан бір жол мәтін оқу үшін қандай өрнек қолданылады?

A) Console.ReadLine()

B) Console.WriteLine()

C) new Console()

D) Console.ReadText()

26. Мәтіннен int мәнін алатын әдіс қандай?

A) string.Read()

B) int.Parse()

C) bool.Read()

D) char.Next()

27. Консольден толық мәтін жолын оқитын әдіс қандай?

A) Console.ReadInt()

B) Console.GetInteger()

C) Console.ReadLine()

D) Console.GetString()

28. Console.WriteLine("Нәтиже: " + 5); қандай мәтін шығарады?

A) Нәтиже: + 5

B) 5

C) Нәтиже:

D) Нәтиже: 5

29. Жолдық литералдағы \n нені білдіреді?

A) Жаңа жолға көшуді

B) Бос орын қосуды

C) Тырнақша шығаруды

D) Кері қиғаш сызық шығаруды

30. Console.WriteLine("{0}", 12); кодындағы {0} не үшін қолданылады?

A) Мәтінді өшіру үшін

B) Бірінші аргумент мәнін пішімдеп шығару үшін

C) Логикалық мәнді өзгерту үшін

D) Жаңа жолды жою үшін

31–40. Операторлар және математикалық есептеулер

31. $17 \% 5$ өрнегінің нәтижесі қандай?

A) 3

B) 5

C) 2

D) 12

32. $7 / 2$ өрнегінің нәтижесі қандай?

A) 3.5

B) 4

C) 2

D) 3

33. $7.0 / 2$ өрнегінің нәтижесі қандай?

A) 3.5

B) 3

C) 4

D) 2.0

34. `int x = 5; x++;` орындалғаннан кейін `x` мәні қандай?

A) 4

B) 6

C) 5

D) 10

35. Мән тағайындау операторы қайсысы?

A) `==`

B) `!=`

C) `=`

D) `<=`

36. Екі примитивті сандық мәnniң теңдігін тексеретін оператор қандай?

A) `=`

B) `<>`

C) `:=`

D) `==`

37. Қысқартылған есептеу қолданылатын логикалық «ЖӘНЕ» операторы қандай?

A) `&&`

B) `||`

C) `!`

D) `%`

38. `!true` өрнегінің нәтижесі қандай?

A) `true`

B) `false`

C) `1`

D) `null`

39. `Math.Sqrt(81)` қандай мән қайтарады?

A) 81.0

B) 8.0

C) 9.0

D) 6561.0

40. `Math.Pow(2, 3)` қандай мән қайтарады?

A) 6.0

B) 9.0

- C) 5.0
- D) 8.0

41–50. Шартты операторлар

41. Шартқа байланысты код блогын орындайтын оператор қандай?

- A) if
- B) import
- C) package
- D) return

42. if шартындағы өрнектің типі қандай болуы керек?

- A) string
- B) bool
- C) double
- D) char

43. else блогы қашан орындалады?

- A) Әрқашан
- B) Шарт ақиқат болғанда
- C) Тиісті if шарты жалған болғанда
- D) Тек цикл аяқталғанда

44. `int x = 8; if (x > 5) x = 1; else x = 2;` орындалғаннан кейін x мәні қандай?

- A) 8
- B) 5
- C) 2
- D) 1

45. Санның жұп екенін тексеретін шарт қайсысы?

- A) `n % 2 == 0`
- B) `n / 2 == 0`
- C) `n + 2 == 0`
- D) `n * 2 == 0`

46. x санының 10 мен 20 аралығында, шекараларын қоса алғанда, жатқанын тексеріңіз.

- A) `x >= 10 || x <= 20`
- B) `x >= 10 && x <= 20`
- C) `x < 10 && x > 20`
- D) `x == 10 && x == 20`

47. Тернарлық шартты оператордың дұрыс үлгісі қандай?

- A) `шарт :: мән1 :: мән2`
- B) `шарт -> мән1 -> мән2`
- C) `шарт ? мән1 : мән2`
- D) `шарт ; мән1 ; мән2`

48. `int a = 4, b = 9; int max = a > b ? a : b;` нәтижесінде max неге тең?

- A) 4
- B) 13
- C) 0
- D) 9

49. Дәстүрлі switch операторында сәйкес case табылмаса, қай белгіге тиесілі код орындалуы мүмкін?

- A) default
- B) finally
- C) extends
- D) catch

50. Дәстүрлі switch ішінде break операторының қызметі қандай?

- A) Жаңа нұсқа қосады
- B) switch орындалуын аяқтайды

C) Шартты өзгертеді

D) Айнымалыны өшіреді

51–60. Циклдік операторлар

51. Қайталау саны белгілі болғанда жиі қолданылатын цикл қандай?

A) if

B) switch

C) for

D) catch

52. while циклінде шарт қашан тексеріледі?

A) Тек бағдарлама аяқталғанда

B) Тек бірінші қайталаудан кейін

C) Ешқашан тексерілмейді

D) Әр қайталаудың алдында

53. Қай циклдің денесі кемінде бір рет орындалады?

A) do...while

B) while

C) for

D) Ешқайсысы

54. for (int i = 0; i < 5; i++) циклінің денесі қанша рет орындалады?

A) 4

B) 5

C) 6

D) 0

55. break операторының цикл ішіндегі қызметі қандай?

A) Келесі қайталауды бастайды

B) Айнымалыны арттырады

C) Өзі қатысты циклді тоқтатады

D) Әдіс құрады

56. continue операторы не істейді?

A) Бағдарламаны толық аяқтайды

B) Циклді толық тоқтатады

C) Барлық айнымалыны өшіреді

D) Ағымдағы қайталаудың қалған бөлігін өткізіп, келесі қайталауға көшеді

57. int s = 0; for (int i = 1; i <= 3; i++) s += i; соңында s қандай?

A) 6

B) 3

C) 4

D) 9

58. Ішкі циклдің сыртқы цикл денесінде орналасуы қалай аталады?

A) Шартты таңдау

B) Кірістірілген цикл

C) Әдісті қайта анықтау

D) Типті түрлендіру

59. int i = 0; while (i < 3) i++; аяқталғанда i қандай мәнге тең?

A) 0

B) 2

C) 3

D) 4

60. int[] a = {2, 4, 6}; foreach (int x in a) Console.Write(x + " "); нәтижесі қандай?

A) 0 1 2

B) 6 4 2

- C) 2 4
- D) 2 4 6

61–70. Массивтер және рекурсия

61. C# массивіндегі бірінші элементтің индексі қандай?

- A) 0
- B) 1
- C) -1

D) Массив ұзындығына тең

62. Үш элементтен тұратын бүтін сандар массивін дұрыс жариялаңыз.

- A) `int a = {1, 2, 3};`
- B) `int[] a = {1, 2, 3};`
- C) `array int a(1, 2, 3);`
- D) `int[3] a = 1, 2, 3;`

63. `int[] a = {4, 7, 9};` массивіндегі `a[1]` мәні қандай?

- A) 4
- B) 9
- C) 7
- D) 1

64. C# массивінің ұзындығын анықтайтын өрнек қандай?

- A) `a.size()`
- B) `a.length()`
- C) `length(a)`
- D) `a.Length`

65. `new int[4]` арқылы жасалған массив элементтерінің бастапқы мәні қандай?

- A) 0
- B) 1
- C) null
- D) Кездейсоқ сан

66. Екі өлшемді бүтін сандар массивін дұрыс жасаңыз.

- A) `int[] m = new int[2,3];`
- B) `int[,] m = new int[2,3];`
- C) `int m = new int[2,3];`
- D) `matrix m = int(2,3);`

67. `int[,] m = {{1, 2}, {3, 4}};` үшін `m[1,0]` мәні қандай?

- A) 1
- B) 2
- C) 3
- D) 4

68. Рекурсия дегеніміз не?

- A) Массивтісұрыптау
- B) Айнымалыны жою
- C) Файлды ашу
- D) Әдістің тікелей немесе жанама түрде өзін шақыруы

69. Рекурсивті алгоритмдегі базалық жағдай не үшін қажет?

- A) Рекурсивті шақыруларды тоқтату үшін
- B) Барлық айнымалыны мәтінге айналдыру үшін
- C) Әдістің атауын өзгерту үшін
- D) Массивті үлкейту үшін

70. Факториалдың дұрыс рекурренттік қатынасы қандай?

- A) $n! = n + (n - 1)!$
- B) $n! = n \times (n - 1)!$, мұнда $0! = 1$

C) $n! = n - (n - 1)!$

D) $n! = n / (n - 1)!$, мұнда $0! = 0$

71–80. Әдістер, тізімдер, жиындар және сөздіктер

71. C# әдісінде void нені білдіреді?

A) Әдіс міндетті түрде бүтін сан қайтарады

B) Әдіс шақырылмайды

C) Әдіс мән қайтармайды

D) Әдіс тек массив қабылдайды

72. Әдістен мән қайтару үшін қандай оператор қолданылады?

A) break

B) continue

C) import

D) return

73. Әдісті асыра жүктеу дегеніміз не?

A) Бір атаулы әдістерді әртүрлі параметрлер тізімімен анықтау

B) Бір әдістің кодын өшіру

C) Әдістің атауын міндетті түрде өзгерту

D) Әдісті тек бір рет шақыру

74. `int add(int a, int b) { return a + b; }` әдісіне `add(3, 4)` берілсе, нәтиже қандай?

A) 12

B) 7

C) 1

D) 34

75. `List<T>` қандай құрылым?

A) Тек бір элемент сақтайтын айнымалы

B) Қайталанбайтын элементтер жиыны

C) Өлшемі өзгертін тізім

D) Тек кілттер сақтайтын құрылым

76. `List<string>` list тізіміне элемент қосатын әдіс қандай?

A) `list.Put("C#")`

B) `list.InsertKey("C#")`

C) `list.AppendValue("C#")`

D) `list.Add("C#")`

77. `List<T>` ішіндегі элементтер санын анықтайтын қасиет қандай?

A) Count

B) Length

C) CapacityValue

D) CountAll

78. Қай C# коллекциясы қайталанатын элементтерді сақтауға жол бермейді?

A) `List<T>`

B) `HashSet<T>`

C) `Dictionary<TKey, TValue>`

D) `Queue<T>`

79. Кілт–мән жұптарын сақтайтын C# коллекциясы қандай?

A) `HashSet<T>`

B) `List<T>`

C) `Dictionary<TKey, TValue>`

D) `Queue<T>`

80. `Dictionary<string, int>` map құрылымына жұпты дұрыс қосыңыз.

A) `map.AddValue("C#", 10);`

B) `map.SetKey("C#", 10);`

C) map.Append("C#", 10);

D) map.Add("C#", 10);

81–90. Объектіге бағытталған бағдарламалау

81. Класс дегеніміз не?

A) Объектілердің күйі мен әрекетін сипаттайтын үлгі

B) Тек сан сақтайтын айнымалы

C) Тек циклдің бір түрі

D) Міндетті түрде деректер қорының кестесі

82. Объект дегеніміз не?

A) Бағдарламаның түсініктемесі

B) Кластың нақты данасы

C) Кез келген оператор

D) Файл кеңейтімі

83. Конструктор шақырып, жаңа объект жасау үшін қандай кілттік сөз қолданылады?

A) class

B) void

C) new

D) return

84. C# конструкторына қатысты дұрыс тұжырымды көрсетіңіз.

A) Оның атауы әрқашан Main болады

B) Ол міндетті түрде int қайтарады

C) Ол тек цикл ішінде жазылады

D) Оның атауы класс атауымен бірдей және қайтарылатын типі көрсетілмейді

85. Инкапсуляцияның негізгі мақсаты қандай?

A) Объектінің ішкі деректеріне қолжетімділікті басқару

B) Барлық өрісті міндетті түрде ашық ету

C) Барлық класты бір массивке біріктіру

D) Цикл санын көбейту

86. private мүшесіне тікелей қолжетімділік әдетте қай жерде беріледі?

A) Кез келген пакетте

B) Өзі жарияланған кластың ішінде

C) Тек main әдісінде

D) Тек туынды класта

87. C# тілінде кластың басқа кластан мұрагерлігін көрсету үшін қандай таңба қолданылады?

A) implements

B) import

C) :

D) throws

88. Полиморфизмнің мысалы қайсысы?

A) Айнымалы атауын өзгерту

B) Массивті нөлдермен толтыру

C) Файлды өшіру

D) Базалық типтегі сілтеме арқылы туынды объектінің қайта анықталған әдісін шақыру

89. this кілттік сөзі нені білдіреді?

A) Ағымдағы объектіге сілтемені

B) Кез келген базалық класты

C) Барлық статикалық әдістерді

D) Жаңа пакетті

90. static әдісіне қатысты дұрыс тұжырымды көрсетіңіз.

A) Ол тек конструктор ішінде анықталады

B) Ол классқа тиесілі және оны объект жасамай шақыруға болады

- C) Ол міндетті түрде абстрактті болады
- D) Ол тек мәтін қайтарады

91–100. Пакеттер, кітапханалар және ерекшеліктерді өңдеу

91. C# файлына атаулар кеңістігін жариялау үшін қандай кілттік сөз қолданылады?

- A) import
- B) extends
- C) namespace
- D) return

92. Басқа атаулар кеңістігіндегі типтерді қысқа атаумен қолдануға мүмкіндік беретін директива қандай?

- A) throws
- B) new
- C) private
- D) using

93. `Math.abs(-12)` нәтижесі қандай?

- A) 12
- B) -12
- C) 0
- D) 144

94. `Math.max(5, 9)` нәтижесі қандай?

- A) 5
- B) 9
- C) 14
- D) 4

95. Орындалу барысында ерекшелік туындауы мүмкін код қай блокқа орналастырылады?

- A) package
- B) class
- C) try
- D) import

96. Ерекшелікті ұстап, өңдейтін блок қандай?

- A) for
- B) switch
- C) extends
- D) catch

97. `finally` блогына қатысты дұрыс тұжырымды көрсетіңіз.

- A) `try` блогынан шығу кезінде, ерекшелік туындаса да, туындамаса да, әдетте орындалады
- B) Тек компиляцияқатесі болғанда орындалады
- C) Тек цикл ішінде орындалады
- D) Ерекшелікті міндетті түрде қайта тастайды

98. Ерекшелікті тікелей тастау үшін қандай кілттік сөз қолданылады?

- A) throws
- B) throw
- C) catch
- D) finally

99. `int z = 0; int x = 10 / z;` орындалғанда қандай ерекшелік туындайды?

- A) `NullReferenceException`
- B) `IOException`
- C) `DivideByZeroException`
- D) `ClassNotFoundException`

100. `int[] a = {1, 2}; Console.WriteLine(a[2]);` орындалғанда не болады?

- A) 2 саны шығады
- B) 0 саны шығады

- C) Массив автоматты түрде кеңейеді
D) IndexOutOfRangeException туындайды

Жауап кілті

№	Жауап	№	Жауап	№	Жауап	№	Жауап	№	Жауап
1	B	21	A	41	A	61	A	81	A
2	A	22	B	42	B	62	B	82	B
3	C	23	C	43	C	63	C	83	C
4	D	24	D	44	D	64	D	84	D
5	A	25	A	45	A	65	A	85	A
6	B	26	B	46	B	66	B	86	B
7	C	27	C	47	C	67	C	87	C
8	D	28	D	48	D	68	D	88	D
9	A	29	A	49	A	69	A	89	A
10	B	30	B	50	B	70	B	90	B
11	C	31	C	51	C	71	C	91	C
12	D	32	D	52	D	72	D	92	D
13	A	33	A	53	A	73	A	93	A
14	B	34	B	54	B	74	B	94	B
15	C	35	C	55	C	75	C	95	C
16	D	36	D	56	D	76	D	96	D
17	A	37	A	57	A	77	A	97	A
18	B	38	B	58	B	78	B	98	B
19	C	39	C	59	C	79	C	99	C
20	D	40	D	60	D	80	D	100	D

ҚОРЫТЫНДЫ

«C# тілінде объектіге бағытталған бағдарламалау» электрондық оқу құралы білім алушылардың C# бағдарламалау тілінің негіздерін меңгеруіне және бағдарламалық қосымшалар әзірлеу дағдыларын қалыптастыруына арналған. Оқу құралында теориялық материалдар, бағдарламалық мысалдар, зертханалық жұмыстар, бейнесабақтарға сілтемелер және білімді тексеруге арналған тест тапсырмалары өзара байланыста ұсынылған.

Оқу материалдарында мәліметтер типтері, айнымалылар, деректерді енгізу және шығару құралдары, математикалық операциялар, шартты және циклдік операторлар қарастырылды. Массивтер, әдістер, рекурсия, тізімдер, жиындар және кілт–мән құрылымдары арқылы деректерді ұйымдастыру мен өңдеу тәсілдері түсіндірілді. Сонымен қатар C# тілінің синтаксистік ерекшеліктері, .NET платформасының мүмкіндіктері және бағдарламаларды әзірлеу ортасының негізгі құралдары қамтылды. Бұл тақырыптар есептің шартын талдау, шешу алгоритмін құру және оны C# тілінде іске асыру үшін қажетті негіз қалыптастырады.

Объектіге бағытталған бағдарламалау бөлімінде кластар мен объектілерді құру, инкапсуляция, мұрагерлік, полиморфизм және интерфейстерді пайдалану қағидалары қамтылды. Атаулар кеңістігін (namespace) ұйымдастыру, кітапханаларды қолдану және ерекше жағдайларды өңдеу тәсілдері бағдарламаның құрылымын реттеуге, кодтың қайта қолданылуын қамтамасыз етуге және оның жұмысын жетілдіруге бағытталған. Қорытынды дәрісте осы ұғымдарды шағын қосымша әзірлеу барысында бірге қолдану үлгісі берілді.

Зертханалық жұмыстар теориялық білімді тәжірибеде бекітуге мүмкіндік береді. C# тілінде бағдарламаларды құру, компиляциялау, іске қосу, нәтижелерін салыстыру және қателерін түзету барысында білім алушылардың логикалық ойлауы мен практикалық есептерді шешу дағдылары дамиды. Бейнесабақтар тапсырмалардың орындалу ретін түсінуге көмектеседі, ал тест тапсырмалары меңгерілген білімді бағалауға жағдай жасайды.

Оқу құралын жүйелі меңгеру нәтижесінде білім алушылар C# тілінде бағдарламалар жазуға, деректер құрылымдарын таңдауға, кластардың жауапкершілігін анықтауға және объектілердің өзара әрекеттесуін ұйымдастыруға үйренеді. Сонымен қатар олар .NET платформасының негізгі мүмкіндіктерін пайдалануға, бағдарламалық кодты құрылымдауға және қарапайым консольдік қосымшаларды әзірлеуге қажетті дағдыларды қалыптастырады. Алынған білім кейінгі кәсіби пәндерді игеруге, оқу жобаларын орындауға және бағдарламалық жүйелерді әзірлеу негіздерін тереңдетуге тірек болады.

C# тілінде бағдарламалау шеберлігі тұрақты тәжірибе арқылы жетіледі. Сондықтан

ПАЙДАЛАНЫЛҒАН ӘДЕБИЕТТЕР ТІЗІМІ

1. Albahari J. *C#12 in a Nutshell: The Definitive Reference*. — O'Reilly Media, 2024.
2. Price M. J. *C#12 and .NET 8—Modern Cross-Platform Development Fundamentals*. — 8th ed. — Packt Publishing, 2023.
3. Troelsen A., Japikse P. *Pro C#10 with .NET 6: Foundational Principles and Practices in Programming*. — Apress, 2022.
4. Skeet J. *C# in Depth*. — 4th ed. — Manning Publications, 2019.
5. Richter J. *CLR via C#*. — 4th ed. — Microsoft Press, 2012.
6. Freeman A. *Pro ASP.NET Core 7*. — 10th ed. — Manning Publications, 2023.
7. Griffiths I. *Programming C#10: Build Cloud, Web, and Desktop Applications*. — O'Reilly Media, 2022.
8. Stellman A., Greene J. *Head First C#*. — 4th ed. — O'Reilly Media, 2020.
9. Nakov S. et al. *Fundamentals of Computer Programming with C#*. — Telerik Software Academy, 2013.
10. Hejlsberg A., Wiltamuth S., Golde P. *The C# Programming Language*. — 4th ed. — Addison-Wesley Professional, 2010.
11. Microsoft. *C# Documentation* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/>
12. Microsoft. *C# Language Reference* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/>
13. Microsoft. *Introduction to C#* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>
14. Microsoft. *C# Type System* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/>
15. Microsoft. *C# Operators and Expressions* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/>
16. Microsoft. *Selection Statements (C# Reference)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements>
17. Microsoft. *Iteration Statements (C# Reference)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements>
18. Microsoft. *Arrays (C# Programming Guide)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/arrays>
19. Microsoft. *Methods (C# Programming Guide)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/methods>
20. Microsoft. *Classes, Structs, and Records in C#* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/>
21. Microsoft. *Inheritance (C# Programming Guide)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/inheritance>
22. Microsoft. *Polymorphism (C# Programming Guide)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/object-oriented/polymorphism>

23. Microsoft.*Interfaces(C#ProgrammingGuide)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/types/interfaces>
24. Microsoft.*ExceptionHandling(C#ProgrammingGuide)* [Электрондық ресурс].— URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/exceptions/>
25. Microsoft.*Collections(C#)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/collections>
26. Microsoft.*List<T>Class* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.list-1>
27. Microsoft.*Dictionary<TKey,TValue>Class* [Электрондық ресурс].— URL: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.dictionary-2>
28. Microsoft.*HashSet<T>Class* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.hashset-1>
29. Microsoft.*LINQ(LanguageIntegratedQuery)* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/csharp/linq/>
30. Microsoft..*NETDocumentation* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/dotnet/>
31. Microsoft..*NETFundamentals* [Электрондық ресурс].— URL: <https://learn.microsoft.com/en-us/dotnet/fundamentals/>
32. Microsoft..*VisualStudioDocumentation* [Электрондық ресурс].— URL: <https://learn.microsoft.com/en-us/visualstudio/>
33. Microsoft..*Get StartedwithC#* [Электрондық ресурс]. — URL: <https://learn.microsoft.com/en-us/training/paths/get-started-c-sharp-part-1/>
34. Gamma E., Helm R., Johnson R., Vlissides J..*DesignPatterns:ElementsofReusable Object-OrientedSoftware*. — Addison-Wesley Professional, 1994.
35. Martin R.C..*CleanCode: AHandbookofAgileSoftwareCraftsmanship*. — Prentice Hall, 2008.